

# x86-64 Assembly Language Programming with Ubuntu-1

남궁명수

『x86-64 Assembly Language Programming with Ubuntu』를 기반으로 어셈블리 언어를 학습하였으며, 개념 이해 및 학습 정리를 목적으로 내용을 정리하였다.

|                                                   |    |
|---------------------------------------------------|----|
| Chapter1. Introduction.....                       | 8  |
| [1-1] 전제 조건(Prerequisites).....                   | 8  |
| [1-2] 어셈블리 언어란.....                               | 9  |
| [1-3] 어셈블리 언어를 배우는 이유.....                        | 10 |
| [1-3-1] 아키텍처 이슈에 대한 이해 향상.....                    | 10 |
| [1-3-2] 툴 체인에 대한 이해.....                          | 11 |
| [1-3-3] 알고리즘 개발 능력 향상.....                        | 11 |
| [1-3-4] 함수/프로시저에 대한 이해 향상.....                    | 12 |
| [1-3-5] 입출력(I/O) 버퍼링에 대한 이해.....                  | 12 |
| [1-3-6] 컴파일러의 역할과 범위 이해.....                      | 12 |
| [1-3-7] 멀티프로세싱 개념 소개.....                         | 13 |
| [1-3-8] 인터럽트 처리 개념 소개.....                        | 13 |
| Chapter2. 아키텍처 개요.....                            | 14 |
| [2-1] 아키텍처 개요.....                                | 14 |
| [2-1-1] CPU(Central Processing Unit).....         | 14 |
| [2-1-2] 주기억장치(RAM).....                           | 15 |
| [2-1-3] 보조 저장 장치(Secondary Storage).....          | 15 |
| [2-1-4] 프로그램 실행 과정.....                           | 15 |
| [2-1-5] 버스(BUS).....                              | 15 |
| [2-2] 데이터 저장 크기.....                              | 16 |
| [2-2-1] 왜 2의 거듭제곱인가?.....                         | 16 |
| [2-2-2] 배열과 리스트.....                              | 16 |
| [2-2-3] 고급 언어와의 관계.....                           | 17 |
| [2-3] 중앙 처리 장치(Central Processing Unit, CPU)..... | 17 |
| [2-3-1] CPU 레지스터.....                             | 17 |
| [2-3-2] 캐시 메모리.....                               | 23 |
| [2-4] 주기억장치(Main Memory).....                     | 24 |
| [2-4-1] 메모리는 연속된 바이트의 집합.....                     | 24 |
| [2-4-2] 리틀 엔디언(Little-Endian) 저장 방식.....          | 24 |
| [2-4-3] 예제: 5,000,000 저장 과정.....                  | 25 |
| [2-4-4] 중요한 개념 정리.....                            | 25 |
| [2-4-5] 왜 이 개념이 중요한가.....                         | 26 |
| [2-4-6] 레지스터와 메모리의 관계.....                        | 26 |
| [2-5] 메모리 배치(Memory Layout).....                  | 27 |
| [2-5-1] 예약 영역(Reserved Section).....              | 27 |

|                                                     |    |
|-----------------------------------------------------|----|
| [2-5-2] 텍스트(Text) 영역/ 코드(Code) 영역.....              | 27 |
| [2-5-3] 데이터(Data) 영역 .....                          | 28 |
| [2-5-4] 초기화되지 않은 데이터 영역(BSS Section).....           | 28 |
| [2-5-5] 힙(Heap).....                                | 29 |
| [2-5-6] 스택(Stack) .....                             | 29 |
| [2-6] 메모리 계층 구조(Memory Hierachy).....               | 30 |
| [2-6-1] 메모리 계층 구조의 기본 개념 .....                      | 30 |
| [2-6-2] 메모리 계층 구조의 형태 .....                         | 31 |
| [2-6-3] 일반적인 메모리 계층별 특성 .....                       | 31 |
| [2-6-4] 왜 메모리 계층 구조가 필요한가?.....                     | 31 |
| [2-6-5] 상대적 속도 차이의 중요성.....                         | 32 |
| Chapter3. 데이터 표현(Data Representation) .....         | 33 |
| [3-1] 정수 표현(Integer Representation).....            | 33 |
| [3-1-1] 2의 보수(Two's Complement) .....               | 36 |
| [3-1-2] 바이트 예제(8 비트).....                           | 37 |
| [3-1-2] 워드 예제(16 비트) .....                          | 38 |
| [3-1-3] 2의 보수의 본질적 의미.....                          | 39 |
| [3-2] 부호 없는 덧셈과 부호 있는 덧셈 .....                      | 40 |
| [3-2-1] 동일한 연산, 다른 해석 .....                         | 40 |
| [3-2-2] 동일한 비트 패턴의 또 다른 해석.....                     | 41 |
| [3-2-3] CPU는 “해석”하지 않는다 .....                       | 42 |
| [3-2-4] 오버플로우와 캐리의 차이.....                          | 42 |
| [3-2-5] 왜 데이터 크기와 타입 정의가 중요한가.....                  | 43 |
| [3-3] 부동소수점 표현(Floating-point Representation) ..... | 43 |
| [3-3-1] IEEE 754 32 비트 표현 방식 .....                  | 44 |
| [3-3-3] IEEE 64 비트 표현 방식(Double Precision) .....    | 49 |
| [3-3-4] 숫자가 아님(Not a Number, NaN).....              | 50 |
| [3-4] 문자와 문자열(Characters and Strings) .....         | 50 |
| [3-4-1] 문자 표현.....                                  | 50 |
| [3-4-2] 문자열 표현 .....                                | 52 |
| Chapter4. 프로그램 형식.....                              | 54 |
| [4-1] 주석(Comments) .....                            | 54 |
| [4-2] 숫자 값(Numeric Values) .....                    | 55 |
| [4-2-1] 10 진수(Decimal) .....                        | 55 |
| [4-2-2] 16 진수(Hexadecimal).....                     | 55 |

|                                            |    |
|--------------------------------------------|----|
| [4-2-3] 8 진수(Octal) .....                  | 55 |
| [4-3] 상수 정의 .....                          | 55 |
| [4-3-1] 상수의 특징 .....                       | 56 |
| [4-3-2] 크기와의 관계 .....                      | 56 |
| [4-4] 데이터 섹션(Data Section)(*) .....        | 56 |
| [4-4-1] 데이터 섹션의 역할 .....                   | 56 |
| [4-4-2] 변수 이름 규칙 .....                     | 56 |
| [4-4-3] 변수 정의 형식 .....                     | 57 |
| [4-4-4] 지원되는 데이터 타입 .....                  | 57 |
| [4-4-5] 초기화된 배열 .....                      | 57 |
| [4-4-6] 문자열 선언 .....                       | 58 |
| [4-4-7] 중요한 제약 조건 .....                    | 58 |
| [4-5] BSS 섹션 .....                         | 58 |
| [4-5-1] BSS 섹션의 역할 .....                   | 58 |
| [4-5-2] 변수 이름 규칙 .....                     | 59 |
| [4-5-3] 변수 정의 형식 .....                     | 59 |
| [4-5-4] 지원되는 데이터 타입 .....                  | 59 |
| [4-5-5] 예제 .....                           | 60 |
| [4-5-6] 중요한 특징 .....                       | 60 |
| [4-5-7] .data 와 .bss 의 차이 비교 .....         | 60 |
| [4-6] 텍스트(Text) 섹션 .....                   | 60 |
| [4-6-1] 텍스트 섹션의 역할 .....                   | 60 |
| [4-6-2] 명령어 작성 규칙 .....                    | 61 |
| [4-6-3] 프로그램의 시작점 정의 .....                 | 61 |
| [4-6-4] 프로그램 종료 처리 .....                   | 61 |
| [4-6-5] 전체 구조 예시 .....                     | 62 |
| [4-7] 예제 프로그램 .....                        | 62 |
| [4-7-1] 전체 구조 개요 .....                     | 64 |
| [4-7-2] Data Section(section .data) .....  | 64 |
| [4-7-3] Code Section(section .text) .....  | 66 |
| [4-7-4] 연산 예제 분석 .....                     | 66 |
| [4-7-5] 프로그램 종료 .....                      | 66 |
| [4-7-6] 대괄호([])의 의미 .....                  | 66 |
| [4-7-7] 왜 byte, word 같은 크기 지정이 필요한가? ..... | 67 |
| Chapter5. 툴 체인(Tool Chain) .....           | 68 |

|                                                 |    |
|-------------------------------------------------|----|
| [5-1] Assemble → Link → Load 개요 .....           | 68 |
| [5-1-1] 전체 흐름 요약 .....                          | 68 |
| [5-1-2] 단계별 책임 비교 .....                         | 70 |
| [5-2] Assembler .....                           | 70 |
| [5-2-1] Assemble 명령어 .....                      | 71 |
| [5-2-2] 리스트 파일(List File) .....                 | 71 |
| [5-2-3] Two-Pass Assembler(2 패스 어셈블러) .....     | 73 |
| [5-2-4] 어셈블러 지시자(Assembler Directives) .....    | 75 |
| [5-3] Linker .....                              | 78 |
| [5-3-1] 여러 파일을 링크하는 경우 .....                    | 78 |
| [5-3-2] 링킹 과정의 핵심 개념 .....                      | 79 |
| [5-3-3] 동적 링크(Dynamic Linking) .....            | 80 |
| [5-4] 어셈블 / 링크 스크립트 .....                       | 81 |
| [5-4-1] Bash 기반 어셈블 / 링크 스크립트 예제 .....          | 81 |
| [5-4-2] 스크립트 구조 분석 .....                        | 82 |
| [5-4-3] 스크립트 사용 예 .....                         | 83 |
| [5-4-4] 빌드 자동화의 확장 .....                        | 84 |
| [5-5] Loader(로더) .....                          | 84 |
| [5-5-1] 로더의 주요 기능 .....                         | 84 |
| [5-5-2] 스케줄러와의 관계 .....                         | 84 |
| [5-5-3] 로더는 언제 호출되는가? .....                     | 85 |
| [5-5-4] 출력이 없는 프로그램의 경우 .....                   | 85 |
| [5-6] Debugger(디버거) .....                       | 85 |
| [5-6-1] 왜 디버거가 필요한가? .....                      | 86 |
| [5-6-2] 디버깅 정보를 포함해야 하는 이유 .....                | 86 |
| [5-6-3] 사용되는 디버거 .....                          | 86 |
| [5-7] ELF(Executable and Linkable Format) ..... | 87 |
| [5-7-1] ELF 파일의 전체 구조 .....                     | 87 |
| [5-7-2] ELF Header .....                        | 87 |
| [5-7-3] Program Header Table .....              | 88 |
| [5-7-4] Section Header Table .....              | 88 |
| [5-7-5] 실행 시 메모리 배치 과정 .....                    | 89 |
| [5-7-6] Section vs Segment 정리 .....             | 89 |
| [5-7-7] 디버깅과 ELF의 관계 .....                      | 89 |
| Chapter 6. DDD 디버거(DDD Debugger) .....          | 90 |

|                                                |     |
|------------------------------------------------|-----|
| Chapter 7. 명령어 집합 개요(Instruction Set Overview) | 91  |
| [7-1] 표기 규칙(Notational Conventions)            | 91  |
| [7-1-1] 피연산자 표기법(Operand Notation)             | 92  |
| [7-1-2] 두 피연산자 명령어의 의미                         | 94  |
| [7-1-3] 메모리 연산 제약                              | 94  |
| [7-1-4] 수 체계 표기                                | 95  |
| [7-1-5] 명령어 실행의 기본 모델                          | 95  |
| [7-2] 데이터 이동                                   | 95  |
| [7-2-1] mov 명령어의 의미                            | 95  |
| [7-2-2] 왜 메모리 → 메모리가 안되는가?                     | 96  |
| [7-2-3] 목적지는 즉시값이 될 수 없다.                      | 96  |
| [7-2-4] 크기 일치 규칙                               | 97  |
| [7-2-5] 32 비트 레지스터 쓰기의 특별 동작                   | 97  |
| [7-2-6] 데이터 선언과 CPU 의 관계                       | 98  |
| [7-2-7] 메모리 저장 시 크기 지정이 필요한 이유                 | 98  |
| [7-2-8] 크기 생략이 가능한 이유                          | 99  |
| [7-2-9] 실전 예제                                  | 99  |
| [7-3] Addresses and Values(주소 vs 값)            | 100 |
| [7-3-1] 대괄호 []의 의미                             | 100 |
| [7-3-2] 예제로 보는 주소 vs 값                         | 100 |
| [7-3-3] 왜 어셈블러는 에러를 내지 않는가?                    | 101 |
| [7-3-4] lea 명령어(Load Effective Address)        | 101 |
| [7-3-5] mov 와 lea 의 차이                         | 102 |
| [7-3-6] 직관적으로 이해하기                             | 102 |
| [7-4] 변환 명령어(Conversion Instructions)          | 103 |
| [7-4-1] 축소 변환(Narrowing Conversion)            | 103 |
| [7-4-3] 확장 변환(Widening Conversion)             | 104 |
| [7-5] 정수 산술 명령어                                | 109 |
| [7-5-1] 덧셈(add)                                | 109 |
| [7-5-2] 증가 명령어(inc)                            | 111 |
| [7-5-3] 캐리를 포함한 덧셈(adc)                        | 111 |
| [7-5-4] 뺄셈                                     | 114 |
| [7-5-5] 감소 명령어(dec)                            | 116 |
| [7-5-6] 정수 곱셈                                  | 117 |
| [7-5-4] 정수 나눗셈                                 | 122 |

|                                                                |     |
|----------------------------------------------------------------|-----|
| [7-6] 논리 명령어(Logical Instructions) .....                       | 127 |
| [7-6-1] 논리 연산.....                                             | 127 |
| [7-7] 논리 연산 vs 산술 연산 차이 .....                                  | 130 |
| [7-7-1] 실전 활용 예제.....                                          | 130 |
| [7-8] RFLAGS 레지스터란? .....                                      | 131 |
| [7-8-1] CF - Carry Flag(자리올림 플래그).....                         | 132 |
| [7-8-2] ZF - Zero Flag(제로 플래그) .....                           | 132 |
| [7-8-3] SF - Sign Flag(부호 플래그).....                            | 133 |
| [7-8-4] OF - Overflow Flag(부호 오버플로우).....                      | 133 |
| [7-8-5] PF - Parity Flag .....                                 | 134 |
| [7-8-6] AF - Auxiliary Carry.....                              | 134 |
| [7-9] Shift Operations(시프트 연산).....                            | 134 |
| [7-9-1] Logical Shift(논리 시프트).....                             | 134 |
| [7-9-2] Arithmetic Shift(산술 시프트) .....                         | 136 |
| [7-9-3] Rotate Operations(회전 연산) .....                         | 137 |
| [7-10] Control Instructions(제어 명령어) .....                      | 137 |
| [7-10-1] Labels(레이블) .....                                     | 138 |
| [7-10-2] Unconditional Control Instructions(무조건 제어 명령어) .....  | 138 |
| [7-10-3] Conditional Control Instructions(조건 제어 명령어).....      | 139 |
| [7-10-4] Jump Out of Range(점프 범위 초과) .....                     | 142 |
| [7-10-5] Iteration(반복) .....                                   | 146 |
| [7-11] Example Program - Sum of Squares(제곱합 구하기 예제 프로그램) ..... | 148 |
| [7-11-1] 전체 프로그램 .....                                         | 148 |

## Chapter1. Introduction

이 책의 목적은 대학 수준의 어셈블리 언어 및 시스템 프로그래밍 과목을 위한 참고서를 제공하는 것이다.

다루는 대상은 Ubuntu64 비트 운영체제에서 실행되는 x86-64 계열 프로세서의 명령어 집합이다.

여기서 중요한 포인트는 두 가지다.

첫째, 아키텍처 대상이 명확하다는 것 → x86-64(64 비트 인텔/AMD 계열 CPU)

둘째, 운영체제 환경이 Ubuntu22.04 LTS(64 비트)라는 것

코드는 이 환경에서 테스트되었으며, 이론적으로는 다른 리눅스 64 비트 OS 에서도 동작한다.

[x86-64 는 어떤 구조인가?]

x86-6 는 CISC(Complex Instruction Set Computer, 복합 명령어 집합 컴퓨팅) 구조이다.

CISC 의 특징은 다음과 같다.

- 명령어 종류가 매우 많다.
- 서로 기능이 겹치는 명령어도 존재한다.
- 명령어 길이가 가변적이다.
- 주소 지정 방식(Addressing Mode)이 다양하다.

이 용어는 RISC(Reduced Instruction Set Computer)와 대비되기 위해 나중에 만들어진 개념이다.

| 구분     | CISC     | RISC     |
|--------|----------|----------|
| 명령어 수  | 많음       | 적음       |
| 명령어 길이 | 가변적      | 고정적      |
| 철학     | 복잡하지만 유연 | 단순하지만 빠름 |

[1-1] 전제 조건(Prerequisites)

이 책은 프로그래밍을 처음 배우는 사람을 위한 책이 아니다.

저자는 독자가 이미 다음과 같은 능력을 갖추었다고 가정한다.

- C/C++/Java 같은 컴파일 언어 경험
- 기본적인 프로그래밍 개념 이해

구체적으로 이미 알고 있어야 하는 개념은

- 선언
- 산술 연산
- 제어 구조
- 반복



- 함수 호출
- 함수
- 간접 참조(즉, 포인터)
- 변수 범위(scope) 문제

즉, 이 책은 “어셈블리를 처음 배우는 사람”을 위한 것이지, “프로그래밍을 처음 배우는 사람”을 위한 것은 아니다.

## [1-2] 어셈블리 언어란

학생들이 가장 흔히 묻는 질문은 “왜 어셈블리를 배워야 하나요?”이다.

이 질문에 답하기 전에 먼저, 어셈블리 언어가 정확히 무엇인지 이해하는 것이 중요하다.

어셈블리 언어는 기계 종속적인 언어이다. 즉, 특정 프로세서 아키텍처에 맞춰 작성된다. 예를 들어, x86-64 프로세서를 대상으로 작성된 어셈블리 코드는 ARM 기반의 RISC 프로세서(스마트폰이나 태블릿에서 흔히 사용됨)에서는 실행될 수 없다. 이는 어셈블리 언어가 하드웨어 구조에 직접적으로 결합되어 있기 때문이다.

어셈블리 언어는 흔히 저수준(low-level)언어라고 불린다. 이는 프로그래머가 프로세서의 명령어에 거의 직접적으로 접근할 수 있기 때문이다. 어셈블리 언어는 프로그래머가 사용할 수 있는 범위 안에서, 프로세서에 가장 가까운 형태의 언어이다.

고급 언어(C, C++, Java 등)로 작성된 프로그램은 결국 프로세서가 실행할 수 있는 형태로 변환되어야 한다. 이 과정에서 컴파일러는 고급 언어 코드를 어셈블리 코드로 변환하고, 이후 기계어로 번역한다. 따라서 고급 언어는 인간이 이해하기 쉽게 만들어진 추상화 계층이며, 실제 하드웨어 명령과 프로그래머 사이를 중재하는 역할을 한다.

이러한 이유로 “어셈블리 언어는 죽었다”라는 말은 사실이 아니다. 모든 고급 언어 프로그램은 결국 어셈블리 수준의 명령으로 변환되어 실행되기 때문이다. 즉, 어셈블리는 사라진 것이 아니라, 단지 보이지 않는 곳에서 항상 사용되고 있다.

어셈블리 언어의 가장 큰 특징은 시스템 자원에 대한 직접적인 제어권을 제공한다는 점이다. 프로그래머는 레지스터의 값을 직접 설정할 수 있고, 특정 메모리 위치에 접근할 수 있으며, 하드웨어와 직접 상호작용할 수 있다. 하지만 이러한 직접 제어는 동시에 더 큰 책임을 요구한다. 프로세서 내부 구조, 레지스터 동작 방식, 메모리 구조 등에 대한 깊은 이해가 필요하기 때문이다.

### [1-3] 어셈블리 언어를 배우는 이유

이 책의 목표는 어셈블리 언어 프로그래밍에 대한 포괄적인 입문서를 제공하는 것이다. 그러나 어셈블리 언어를 배우는 목적은 대규모 상용 프로그램을 개발하기 위함이 아니다. 핵심 목적은 컴퓨터가 실제로 어떻게 동작하는지를 이해하는 것에 있다.

어셈블리 언어는 기계 종속적이기 때문에, 다른 아키텍처로 쉽게 옮길 수 없다는 한계가 있다. 이러한 이식성 부족은 실제 대형 소프트웨어 개발에서는 큰 제약이 된다. 그럼에도 불구하고 어셈블리를 배우는 이유는, 컴퓨터의 저수준 동작을 직접 다뤄볼 수 있기 때문이다.

어셈블리 언어 학습 과정에서는 단순한 코드 예제를 넘어서, 비교적 복잡한 저수준 프로그램을 작성하게 된다. 여기에는 다음과 같은 요소들이 포함된다.

- 산술 연산 수행
- 함수 호출 메커니즘 이해
- 스택을 활용한 동적 지역 변수 처리
- 운영체제와의 상호작용(입출력 등)

이러한 과정은 프로그래머가 프로그램 실행의 내부 구조를 체계적으로 이해하도록 돕는다.

단순히 몇 개의 작은 어셈블리 예제를 읽는 것만으로는 충분하지 않다. 중요한 것은, 컴퓨터 시스템의 근본 원리를 직접 경험하는 것이다. 장기적인 관점에서 보면, 어셈블리를 포함한 기초 원리를 이해하는 사람과 그렇지 않은 사람에는 큰 차이가 생긴다.

변화하는 언어 문법이나 프레임워크에만 익숙한 코딩 기술자는 기술이 바뀌면 적응하기 어렵다. 반면, 시스템의 근본 원리를 이해하는 사람은 새로운 기술이 등장해도 그 기반 구조를 빠르게 이해하고 적용할 수 있다. 이 차이가 단순한 “코더”와 지속적으로 성장하는 “컴퓨터 과학자”를 구분 짓는다.

다음 절들에서는 어셈블리 언어를 배워야 하는 보다 구체적인 이유들을 더 자세히 다루게 된다.

#### [1-3-1] 아키텍처 이슈에 대한 이해 향상

어셈블리 언어 수준에서 학습하고 실제로 코드를 작성해보는 경험은, 컴퓨터의 기저에 있는 아키텍처에 대해 훨씬 깊은 이해를 제공한다. 고급 언어에서는 보이지 않던 내부 구조가 어셈블리에서는 그대로 드러난다.

이러한, 이해에는 기본 명령어 집합, 프로세서 레지스터의 역할, 메모리 주소 지정 방식, 하드웨어와의 인터페이싱, 그리고 입출력(Input/Output) 처리 방식 등이 포함된다. 즉, 프로그램이 실제로 실행될 때 내부에서 어떤 일이 벌어지는지를 직접 다루게 된다.

고급 언어에서는 단순한 연산 한 줄이 실제로 여러 개의 어셈블리 명령어로 변환되어 실행된다. 어셈블리를 배우면 이 과정을 이해하게 되고, 특정 연산이 왜 빠른지, 왜 느린지, 왜 복잡한지에 대한 감각을 기를 수 있다.

궁극적으로 모든 프로그램은 어셈블리 수준에서 실행된다. 어떤 언어로 작성했든지, 최종적으로는 기계 명령어로 변환되어 CPU 에서 수행된다. 따라서 어셈블리의 능력과 제약을 이해하는 것은 “무엇이 가능한가?”, “무엇이 효율적인가?”, “어떤 작업이 비용이 많이 드는가?”에 대한 중요한 통찰을 제공하난.

이는 단순한 문법 학습이 아니라, 시스템 설계와 성능 최적화의 기초가 된다.

### [1-3-2] 툴 체인에 대한 이해

툴 체인(tool chain)이란 사람이 작성한 소스 코드를 컴퓨터가 직접 실행할 수 있는 형태로 변환하는 전체 과정을 의미한다. 이 과정에는 컴파일러(또는 어셈블리의 경우 어셈블러), 링커, 로더, 그리고 디버거가 포함된다.

초보 프로그래머들은 보통 “이렇게 컴파일하면 됩니다.”라는 식으로 사용법만 배우는 경우가 많고, 그 내부에서 어떤 일이 일어나는지 깊이 있게 배우지 않는다. 그러나 실제로는 소스 코드가 기계어로 변환되고, 여러 오브젝트 파일이 연결되며, 실행 파일이 메모리에 적재되고, 실행 중 디버깅이 이루어지는 복잡한 단계들이 존재한다.

어셈블리 언어를 배우고 저수준에서 작업하는 과정은 이러한 툴 체인의 각 단계가 어떤 역할을 하는지 명확히 이해하도록 도와준다. 이는 단순한 “사용자”에서 벗어나, 도구의 원리를 이해하는 프로그래머로 성장하는 기반이 된다.

### [1-3-3] 알고리즘 개발 능력 향상

어셈블리 언어는 고급 언어보다 훨씬 더 많은 사고와 세밀한 주의를 요구한다. 자동으로 처리해주던 기능들이 사라지기 때문에, 모든 연산과 데이터 흐름을 직접 설계해야 한다.

이 과정은 프로그래머의 알고리즘 개발 능력을 향상시킨다. 문제를 더 세분화하여 생각하게 만들고, 각 단계가 실제로 어떤 연산으로 이루어지는지를 고민하게 만든다.

또한, 프로그램이 처음부터 올바르게 동작하지 않는 경우(사실 꽤 자주 발생한다), 어셈블리 언어 디버깅은 매우 좋은 훈련이 된다. 단순히 출력문을 추가하는 방식이 어렵기 때문에, 레지스터 값과 메모리 상태를 직접 추적해야 하며, 이는 보다 정교하고 체계적인 접근을 요구한다.

이 과정은 디버거를 적극적으로 활용하는 능력을 길러주며, 이는 어떤 언어를 사용하든 매우 중요한 기술이 된다.

### [1-3-4] 함수/프로시저에 대한 이해 향상

어셈블리 언어로 작업하면 함수(또는 프로시저) 호출이 내부적으로 어떻게 동작하는지를 명확히 이해할 수 있다. 특히 함수 호출 프레임(call frame), 또는 활성 레코드(activation record)의 구조를 직접 다루게 된다.

활성 레코드에는 상황에 따라 다음과 같은 요소들이 포함될 수 있다.

- 스택 기반 인자
- 보존된 레지스터
- 스택 동적 지역 변수

이러한 구조는 고급 언어에서는 보이지 않지만, 실제로는 모든 함수 호출에서 생성되고 제거된다.

특히 스택 기반 지역 변수와 관련해서는 중요한 구현 문제와 보안적 영향이 존재한다. 예를 들어, 스택 오버플로우와 같은 취약점은 이러한 구조를 이해하지 못하면 제대로 파악하기 어렵다. 따라서 저수준에서의 학습은 보안 개념을 이해하는 데도 큰 도움이 된다. 물론, 이러한 지식은 항상 선한 목적을 위해 사용되어야 한다.

또한 스택과 호출 프레임의 동작 원리를 이해하는 것은 재귀의 핵심을 이해하는 데 필수적이다. 재귀 함수는 결국 스택 프레임이 반복적으로 쌓이고 제거되는 과정이기 때문이다.

### [1-3-5] 입출력(I/O) 버퍼링에 대한 이해

고급 언어에서는 입출력 동작이 마치 마법처럼 보일 수 있다. printf 한 줄이면 화면에 출력이 되고, 파일 입출력도 간단한 함수 호출로 이루어진다. 그러나 그 내부에는 버퍼링과 운영체제 서비스 호출이 포함되어 있다.

어셈블리 언어 수준에서 작업하면 이러한 입출력 동작이 실제로 어떻게 이루어지는지 이해하게 된다. 특히 다음과 같은 차이를 명확히 알 수 있다.

- 대화형 입출력(interactive I/O)
- 파일 입출력(file I/O)
- 이에 연관된 운영체제 시스템 호출

버퍼는 왜 필요한지, 언제 실제로 출력이 이루어지는지, 운영체제가 어떤 역할을 하는지를 이해하게 되면, 프로그램의 동작 방식에 대한 통찰이 훨씬 깊어진다.

### [1-3-6] 컴파일러의 역할과 범위 이해

고급 언어를 먼저 학습한 이후 어셈블리 언어로 프로그래밍하는 것은, 컴파일러가 실제로 어떤 일을 수행하는지, 그리고 어떤 일은 수행하지 않는지를 명확히 이해하도록 돕는다.

컴파일러는 고급 언어 코드를 기계가 실행할 수 있는 명령어로 변환해 주지만, 모든 것을 “마법처럼” 해결해주는 것은 아니다. 어셈블리 수준에서 프로그램을 작성해 보면, 고급 언어에서 자동으로 처리되던 부분들 - 예를 들어, 함수 호출 구조, 스택 프레임 구성, 레지스터 사용, 데이터 이동 방식 - 이 실제로 어떻게 구현되는지를 직접 확인할 수 있다.

이를 통해 프로그래머는 컴파일러의 최적화 범위, 아키텍처 의존적 제약, 그리고 성능에 영향을 미치는 요소들을 보다 정확히 이해하게 된다. 결국 이는 단순히 코드를 작성하는 수준을 넘어, 코드가 실제 하드웨어 위에서 어떻게 실행되는지를 이해하는 단계로 나아가게 만든다.

### [1-3-7] 멀티프로세싱 개념 소개

이 책은 멀티프로세싱에 대한 기본적인 개념도 소개한다. 여기에는 분산 처리(distributed processing)와 멀티코어 프로그래밍의 일반적인 개념이 포함되며, 특히 공유 메모리(shared memory)와 스레드 기반(thread-based) 처리에 중점을 둔다.

현대의 컴퓨터 시스템은 대부분 멀티코어 구조를 가지고 있으며, 여러 스레드가 동시에 실행된다. 이 과정에서 공유 메모리에 동시에 접근하게 되면 경쟁 상태(race condition)와 같은 미묘한 문제가 발생할 수 있다.

저자는 이러한 스레딩 관련 문제들이 저수준에서 가장 쉽게 이해된다고 본다. 왜냐하면 어셈블리 언어에서는 메모리 접근과 명령 실행 순서를 직접 확인할 수 있기 때문이다. 따라서 동시성 문제를 보다 근본적인 수준에서 이해할 수 있다.

### [1-3-8] 인터럽트 처리 개념 소개

현대의 다중 사용자 컴퓨터 시스템은 인터럽트(interrupt)를 기반으로 동작한다. 인터럽트는 외부 또는 내부 사건이 발생했을 때, 현재 실행 중인 작업을 잠시 중단하고 특정한 처리 루틴으로 제어를 이동시키는 메커니즘이다.

저수준에서 작업하는 것은 이러한 인터럽트 기반 동작 원리를 이해하기에 가장 적합하다. 여기에는 다음과 같은 핵심 개념들이 포함된다.

- 인터럽트 처리(interrupt handling)
- 인터럽트 서비스 핸들러(interrupt service handler)
- 벡터 인터럽트(vector interrupt)

이러한 개념을 이해하면 운영체제가 어떻게 하드웨어 이벤트를 처리하고, 어떻게 여러 작업을 효율적으로 관리하는지를 보다 명확하게 이해할 수 있다.

## Chapter2. 아키텍처 개요

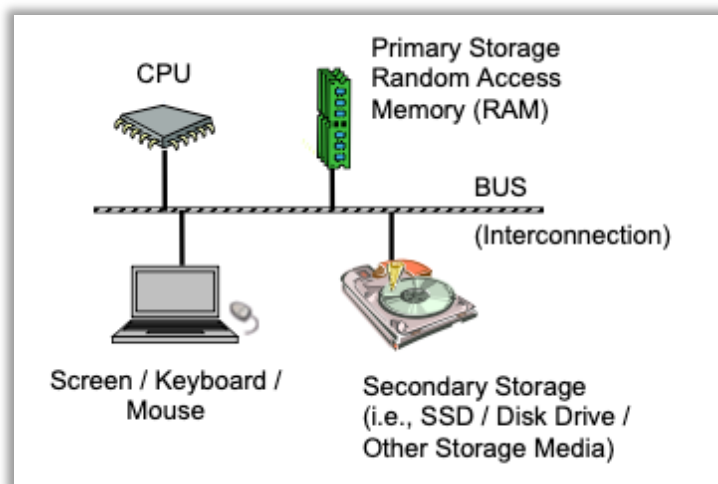
이 장은 x86-64 아키텍처의 기본 구조와 컴퓨터 시스템의 전체적인 동작 원리를 이해하는 것을 목표로 한다. 어셈블리 언어를 배우기 위해서는, 먼저 프로그램이 실제로 실행되는 하드웨어 구조를 이해해야 한다.

### [2-1] 아키텍처 개요

컴퓨터 시스템은 크게 다음과 같은 기본 구성 요소로 이루어진다.

- 중앙 처리 장치(CPU)
- 주기억장치(RAM, 랜덤 접근 메모리)
- 보조 저장 장치(Secondary Storage)
- 입출력 장치(Input/Output devices)
- 버스(BUS)

이 구성은 일반적으로 폰 노이만 아키텍처(Von Neumann Architecture) 또는 프린스턴 아키텍처(Princeton Architecture)라고 불린다. 이 구조는 1945 년 존 폰 노이만에 의해 제시되었으며, 오늘날 대부분의 컴퓨터 시스템의 기본 모델이 된다.



#### [2-1-1] CPU(Central Processing Unit)

CPU 는 컴퓨터의 “두뇌” 역할을 한다.

주기억장치(RAM)에 저장된 프로그램 명령어를 읽어와 해석하고 실행한다.

어셈블리 언어는 바로 이 CPU 가 이해하는 명령어 집합과 직접적으로 연결된다. 따라서 CPU 의 구조와 동작 방식을 이해하는 것은 매우 중요하다.

### [2-1-2] 주기억장치(RAM)

RAM은 현재 실행 중인 프로그램과 데이터를 저장하는 공간이다.  
프로그램은 반드시 RAM에 올라와야 CPU가 실행할 수 있다.

RAM의 중요한 특징은 휘발성(volatile, 발리틀)이라는 점이다. 전원이 꺼지면 저장된 데이터는 모두 사라진다.

### [2-1-3] 보조 저장 장치(Secondary Storage)

보조 저장 장치는 디스크 드라이브(HDD)나 SSD와 같은 장치이다.  
프로그램과 데이터는 일반적으로 이곳에 저장된다.

보조 저장 장치는 비휘발성(non-volatile) 메모리이기 때문에, 전원이 꺼져도 데이터가 유지된다

### [2-1-4] 프로그램 실행 과정

프로그램은 다음과 같은 과정을 통해 실행된다.

1. 프로그램이 디스크(보조 저장 장치)에 저장되어 있다.
2. 프로그램을 실행하면, 해당 코드가 RAM으로 복사된다.
3. CPU는 RAM에 있는 명령어를 하나씩 읽어 실행한다.
4. 작업 결과는 다시 디스크에 저장될 수 있다.

예를 들어, 보고서를 작성할 때 문서는 디스크에 저장되어 있다.

편집을 시작하면 그 문서는 RAM으로 복사된다.

작업 도중 전원이 꺼지면 RAM의 데이터는 사라지기 때문에, 저장하지 않은 내용은 잃게 된다.

이 경험을 휘발성 메모리(RAM)와 비휘발성 메모리(디스크)의 차이를 잘 보여준다.

### [2-1-5] 버스(BUS)

버스는 CPU, 메모리, 입출력 장치들을 연결하는 통로이다.

데이터, 주소, 제어 신호 등이 이 버스를 통해 이동한다.

즉, 컴퓨터 내부에서 각 구성 요소들이 서로 통신할 수 있게 해주는 물리적 연결 구조이다.

## [2-2] 데이터 저장 크기

x86-64 아키텍처는 2의 거듭제곱( $2^n$ )을 기반으로 한 고정된 데이터 크기를 사용한다. 이 크기 단위들은 레지스터 크기, 메모리 접근, 명령어 동작과 직접적으로 연결된다.

| C/C++ Declaration | Storage     | Size (bits) | Size (bytes) |
|-------------------|-------------|-------------|--------------|
| char              | Byte        | 8-bits      | 1 byte       |
| short             | Word        | 16-bits     | 2 bytes      |
| int               | Double-word | 32-bits     | 4 bytes      |
| unsigned int      | Double-word | 32-bits     | 4 bytes      |
| long <sup>5</sup> | Quadword    | 64-bits     | 8 bytes      |
| long long         | Quadword    | 64-bits     | 8 bytes      |
| char *            | Quadword    | 64-bits     | 8 bytes      |
| int *             | Quadword    | 64-bits     | 8 bytes      |
| float             | Double-word | 32-bits     | 4 bytes      |
| double            | Quadword    | 64-bits     | 8 bytes      |

### [2-2-1] 왜 2의 거듭제곱인가?

컴퓨터는 이진수(0 과 1)를 기반으로 동작하기 때문에, 데이터 크기는 2의 거듭제곱 단위로 구성된다.

예:

- 8 비트 =  $2^3$
- 16 비트 =  $2^4$
- 32 비트 =  $2^5$
- 64 비트 =  $2^6$
- 128 비트 =  $2^7$

이러한 구조는 CPU 설계와 메모리 정렬(alignment)에 매우 중요하다.

### [2-2-2] 배열과 리스트

배열이나 리스트 같은 연속된 메모리 공간은 위의 어떤 데이터 타입으로도 선언할 수 있다.

예를 들어:

- 1 바이트 배열 → 문자 배열
- 4 바이트 배열 → int 배열
- 8 바이트 배열 → 64 비트 정수 배열

즉, 배열은 동일한 크기의 데이터들이 연속적으로 배치된 구조이다.



### [2-2-3] 고급 언어와의 관계

이 데이터 크기들은 C/C++/Java 같은 고급 언어의 변수 타입과 직접적으로 연결된다.

| 어셈블리 크기 | C 자료형 예시         |
|---------|------------------|
| 1Byte   | char             |
| 2Bytes  | short            |
| 4Bytes  | int              |
| 8Bytes  | long(64-bit 환경)  |
| 16Bytes | __int128(GCC 확장) |

예:

```
int *ptr;
```

- ptr 은 주소를 저장하는 변수
- 64 비트 환경에서는 포인터 크기가 8 바이트(64 비트)

### [2-3] 중앙 처리 장치(Central Processing Unit, CPU)

중앙 처리 장치(CPU)는 실제 계산이 수행되는 장소이기 때문에 일반적으로 “컴퓨터의 두뇌”라고 불린다. CPU 는 하나의 칩에 집적되어 있으며, 이 칩은 프로세서(processor), 칩(chip), 또는 다이(die)라고도 불린다. 여기서 “다이(die)”는 칩 내부를 의미하는 용어이다.

CPU 칩에는 여러 기능적 구성 요소들이 포함되어 있다. 그 중 산술 논리 장치(ALU, Arithmetic Logic Unit)는 실제로 산술 연산과 논리 연산을 수행하는 핵심 구성 요소이다. ALU 가 연산을 수행하기 위해서는 데이터를 저장하고 빠르게 접근할 수 있는 공간이 필요하며, 이를 위해 프로세서 레지스터와 캐시 메모리가 다이 내부(on the die)에 포함되어 있다.

현대 프로세서의 내부 설계는 매우 복잡하지만, 본 절에서는 CPU 내부의 핵심 기능 단위들에 대해 단순화된 고수준 관점에서 설명한다.

#### [2-3-1] CPU 레지스터

CPU 레지스터(register)는 CPU 내부에 존재하는 임시 저장 공간 또는 작업 공간이다. 이는 주기억장치(RAM)와는 완전히 별개의 존재이며, 메모리보다 훨씬 빠르게 접근할 수 있다.

모든 연산은 기본적으로 레지스터를 통해 수행된다. 즉, CPU 는 메모리에서 직접 계산하지 않으며, 값을 레지스터로 가져온 뒤 연산을 수행한다.

### [1] 일반 목적 레지스터(General Purpose Registers, GPRS)

x86-64 아키텍처에는 총 16 개의 64 비트 일반 목적 레지스터(GPR)가 존재한다.

각 GPR 은 다음과 같은 특징을 가진다.

1. 전체 64 비트로 접근할 수 있다.
2. 하위 32 비트, 16 비트, 8 비트 단위로 부분 접근이 가능하다.
3. 일부 레지스터는 특정 목적을 위해 관례적으로 사용된다.

#### [1-1] 부분 레지스터 구조

하나의 GPR 은 물리적으로 하나의 64 비트 레지스터이다.

rax, eax, ax, al 은 서로 다른 저장소가 아니라, 동일한 레지스터를 서로 다른 크기로 접근하기 위한 이름들이다.



예를 들어, rax 레지스터는 다음과 같이 나뉜다.

- rax: 64 비트 전체
- eax: 하위 32 비트
- ax: 하위 16 비트
- al: 하위 8 비트
- ah: 비트 8~15

특히 rax, rbx, rcx, rdx 는 비트 8~15 영역을 각각, ah, bh, ch, dh 라는 이름으로 접근할 수 있다. 이들 중 일부는 레거시(legacy)호환성을 위해 존재하며 현대 64 비트 환경에서는 일반적으로 사용하지 않는다.

#### [1-2] 부분 레지스터 접근의 동작 원리

부분 레지스터에 값을 저장하면 해당 하위 비트만 변경되며, 상위 비트는 유지된다.

예를 들어,

초기 상태:

rax = 0000 000B A43B 7400

이 값은 50,000,000,000(오백억)에 해당한다.

이후,

```
ax = 0xC350 (50,000)
```

으로 설정하면,

```
rax = 0000 000B A43B C350
```

이 경우 하위 16 비트(ax)만 변경되며 상위 48 비트는 그대로 유지된다.

다시,

```
al = 0x32 (50)
```

로 설정하면,

```
rax = 0000 000B A43B C332
```

하위 8 비트만 변경되고 상위 56 비트는 유지된다.

이 현상은 오류가 아니라 x86 아키텍처가 오랫동안 유지해온 의도적인 설계 방식이다. 부분 레지스터는 새로운 값을 생성하기보다 기존 레지스터 값의 일부를 수정하는 개념에 가깝다.

#### [1-3] 32 비트 레지스터의 특수 규칙

32 비트 레지스터(eax)에 값을 저장하는 경우에는 다른 규칙이 적용된다.

x86-64에서는 eax에 값을 대입하면 상위 32 비트가 자동으로 0으로 초기화된다.

이 특성은 다음과 같은 상황에서 유용하다.

- unsigned 값을 64 비트로 확장할 때

그러나 signed 값의 경우에는 문제가 발생할 수 있다. 음수 값을 단순히 eax에 대입하면 상위 비트가 0으로 초기화되므로 부호 확장이 되지 않는다. 따라서 signed 값을 다룰 때는 이 동작을 정확히 이해해야 한다.

#### [1-4] 레지스터 해석의 핵심 개념

레지스터의 값은 항상 하나이다.

다만 어떤 크기의 이름을 사용하느냐에 따라 해석 방식이 달라질 뿐이다.

- rax를 사용하면 64 비트 전체로 해석
- eax를 사용하면 하위 32 비트로 해석
- ax 또는 al을 사용하면 더 작은 단위로 해석

즉, “나중에 어떻게 사용하느냐”는 저장된 값 자체가 아니라, 어떤 크기의 레지스터 이름을 선택하느냐에 의해 결정된다.

#### [1-5] 부분 레지스터 사용 시 주의점

부분 레지스터를 사용할 때는 상위 비트가 자동 초기화 되지 않는다는 점에 주의해야 한다.

특히 커널 코드나 부트 코드처럼 초기 상태가 불분명한 환경에서는 다음과 같은 방식이 권장된다.

```
xor rax, rax
```

로 전체를 0 으로 초기화한 뒤, al 또는 ax 를 사용하는 방식이다.

#### [2] 스택 포인터 레지스터(RSP)

rsp 는 현재 스택의 최상단(top)을 가리킨다.

이 레지스터는 스택 관리 전용으로 사용되며, 일반적인 데이터 저장 용도로 사용하는 것은 바람직하지 않다.

스택은 함수 호출, 지역 변수 저장, 반환 주소 관리에 사용되며, 이에 대한 자세한 내용은 프로세스 스택장에서 다룬다.

#### [3] 베이스 포인터 레지스터(RBP)

rbp 는 함수 호출 시 기준 포인터(base pointer)로 사용된다.

주로 함수의 스택 프레임을 기준으로 지역 변수와 매개변수에 접근하기 위해 사용된다.

일반 목적 레지스터이지만, 함수 구조를 유지하기 위한 목적으로 사용하는 것이 일반적이다.

#### [4] 명령어 포인터 레지스터(RIP)

rip 는 다음에 실행될 명령어의 주소를 가리킨다.

중요한 점은, rip 가 가리키는 명령어는 아직 실행되지 않은 상태라는 것이다.

디버거에서 표시되는 현재 명령어 역시 실행 전 단계의 코드이므로, 분석 시 혼동하지 않도록 주의해야 한다.

#### [5] 플래그 레지스터(rFlags)

rFlags 레지스터는 CPU 의 상태 및 제어 정보를 저장한다.

이 레지스터는 각 명령어 실행 이후 CPU 에 의해 자동으로 갱신된다. 프로그램이 직접 임의로 조작하는 대상은 아니다.

주요 상태 비트에는 다음과 같은 것들이 포함된다.

- Zero Flag(ZF)
- Sign Flag(SF)
- Carry Flag(CF)
- Overflow Flag(OF)

이 플래그들은 조건 분기 명령어(jz, jne, jc 등)의 동작에 직접적인 영향을 미친다.

## [6] XMM 레지스터

XMM 레지스터는 부동소수점 연산 및 SIMD 명령어를 지원하기 위한 전용 레지스터 집합이다.

SIMD(Single Instruction Multiple Data)는 하나의 명령어로 여러 데이터 요소를 동시에 처리하는 방식이다.

- 그래픽 처리
- 디지털 신호 처리(DSP)
- 대규모 데이터 연산

XMM 레지스터는 SSE(Streaming SIMD Extensions)를 지원하기 위해 사용된다. 일부 최신 CPU는 256 비트 확장 레지스터를 지원하지만, 본 문서 범위에서는 고려하지 않는다.

| 64-bit register | Lowest 32-bits | Lowest 16-bits | Lowest 8-bits |
|-----------------|----------------|----------------|---------------|
| <b>rax</b>      | <b>eax</b>     | <b>ax</b>      | <b>al</b>     |
| <b>rbx</b>      | <b>ebx</b>     | <b>bx</b>      | <b>bl</b>     |
| <b>rcx</b>      | <b>ecx</b>     | <b>cx</b>      | <b>cl</b>     |
| <b>rdx</b>      | <b>edx</b>     | <b>dx</b>      | <b>dl</b>     |
| <b>rsi</b>      | <b>esi</b>     | <b>si</b>      | <b>sil</b>    |
| <b>rdi</b>      | <b>edi</b>     | <b>di</b>      | <b>dil</b>    |
| <b>rbp</b>      | <b>ebp</b>     | <b>bp</b>      | <b>bpl</b>    |
| <b>rsp</b>      | <b>esp</b>     | <b>sp</b>      | <b>spl</b>    |
| <b>r8</b>       | <b>r8d</b>     | <b>r8w</b>     | <b>r8b</b>    |
| <b>r9</b>       | <b>r9d</b>     | <b>r9w</b>     | <b>r9b</b>    |
| <b>r10</b>      | <b>r10d</b>    | <b>r10w</b>    | <b>r10b</b>   |
| <b>r11</b>      | <b>r11d</b>    | <b>r11w</b>    | <b>r11b</b>   |
| <b>r12</b>      | <b>r12d</b>    | <b>r12w</b>    | <b>r12b</b>   |
| <b>r13</b>      | <b>r13d</b>    | <b>r13w</b>    | <b>r13b</b>   |
| <b>r14</b>      | <b>r14d</b>    | <b>r14w</b>    | <b>r14b</b>   |
| <b>r15</b>      | <b>r15d</b>    | <b>r15w</b>    | <b>r15b</b>   |

[7] 우리가 어셈블리에서 실제로 다루는 핵심 레지스터 정리

어셈블리 프로그래밍에서 가장 직접적으로 사용하는 레지스터는 다음과 같다.

1. 일반 목적 레지스터(데이터 연산)
2. rsp(스택 관리)
3. rbp(함수 프레임 기준)
4. rip(간접적으로 사용됨)
5. rFlags(조건 분기와 연관)

어셈블리 프로그래밍은 결국 다음 구조를 이해하는 과정이다.

- 데이터는 레지스터에 올라와야 연산 가능
- 스택은 rsp/rbp 로 관리
- 분기는 rFlags 에 의해 결정
- 실행 흐름은 rip 가 결정

이 구조를 정확히 이해하는 것은 단순한 어셈블리 문법 학습을 넘어서, 운영체제, 컴파일러, 커널, 시스템 프로그래밍 전반을 이해하는 기초가 된다.

## [2-3-2] 캐시 메모리

캐시 메모리는 주기억장치(RAM)의 일부 데이터를 CPU 내부에 복사해 둔 고속 메모리이다. 어떤 메모리 주소가 한 번 접근되면, 해당 데이터의 복사본이 캐시에 저장된다. 이후 동일한 주소에 짧은 시간 내 재접근하면 RAM 이 아닌 캐시에서 읽히므로 훨씬 빠르게 처리된다.

### [1] 캐시 히트와 캐시 미스

- 캐시 히트(cache hit): 요청한 데이터가 캐시에 존재하는 경우
- 캐시 미스(cache miss): 요청한 데이터가 캐시에 존재하지 않는 경우

캐시 히트 비율이 높을수록 전체 시스템 성능은 향상된다.

### [2] 캐시 계층 구조

현대 CPU 는 보통 다음과 같은 계층 구조를 가진다.

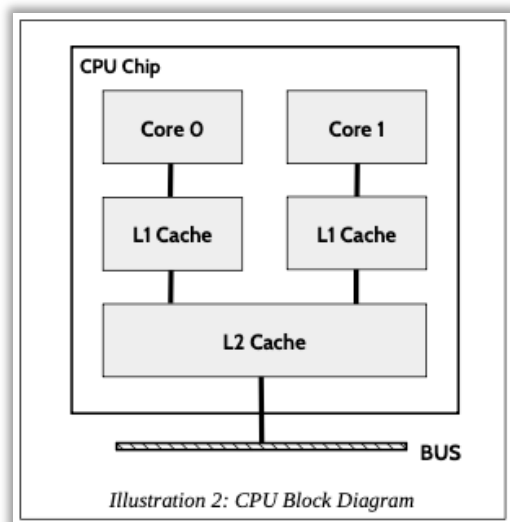
- L1 캐시(각 코어별 존재)
- L2 캐시(공유 또는 코어별)
- L3 캐시(여러 코어 공유)

모든 메모리 접근은 L1 → L2 → L3 → RAM 순으로 탐색된다.

이로 인해 동일한 데이터가 다음 위치에 동시에 존재할 수 있다.

- 레지스터
- L1 캐시
- L2 캐시
- L3 캐시
- 주기억장치(RAM)

이 일관성(coherency) 문제는 CPU 하드웨어가 자동으로 관리하며, 프로그래머가 직접 제어하지 않는다.



## [2-4] 주기억장치(Main Memory)

주기억장치(Main Memory), 즉 RAM 은 프로그램이 실행되는 동안 실제 데이터와 명령어가 저장되는 구조이다. CPU 는 보조 저장 장치(디스크, SSD 등)에 직접 접근하여 실행하지 않으며, 반드시 프로그램과 데이터를 주기억장치로 복사한 뒤 실행한다.

### [2-4-1] 메모리는 연속된 바이트의 집합

메모리는 연속된 바이트(byte)의 집합으로 볼 수 있다.

x86-64 아키텍처에서 메모리는 바이트 단위 주소 지정(byte-addressable) 방식을 사용한다. 이는 다음을 의미한다.

- 각 메모리 주소는 정확히 1 바이트(8 비트)를 저장한다.
- 메모리의 최소 접근 단위는 1 바이트이다.
- 모든 데이터는 바이트들의 연속으로 저장된다.

예를 들어,

- 1 바이트 데이터 → 1 개의 주소 필요
- 2 바이트(워드) → 2 개의 연속된 주소 필요
- 4 바이트(더블워드) → 4 개의 연속된 주소 필요
- 8 바이트(쿼드워드) → 8 개의 연속된 주소 필요

즉, 4 바이트 크기의 더블워드 데이터를 저장하려면 연속된 네 개의 메모리 주소가 필요하다.

### [2-4-2] 리틀 엔디언(Little-Endian) 저장 방식

x86-64 아키텍처는 리틀 엔디언(little-endian) 방식을 사용한다.

리틀 엔디언 방식의 핵심 원칙은 다음과 같다.

- 최하위 바이트(LSB, Least Significant Byte)가 가장 낮은 메모리 주소에 저장된다.
- 최상위 바이트(MSB, Most Significant Byte)가 가장 높은 메모리 주소에 저장된다.

즉, 숫자의 “가장 작은 자리”가 메모리상에서 먼저 배치된다.

|     |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |     |   |   |   |   |   |   |   |
|-----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|-----|---|---|---|---|---|---|---|
| 31  | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7   | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| MSB |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   | LSB |   |   |   |   |   |   |   |



### [2-4-3] 예제: 5,000,000 저장 과정

10 진수 5,000,000 은 16 진수로 다음과 같다.

5,000,000 = 0x004C4B40

이를 더블워드(4 바이트) 변수 var1 에 저장한다고 가정하자.

이 값을 바이트 단위로 나누면 다음과 같다.

00 4C 4B 40

리틀 엔디언 방식에서는 이 바이트들이 역순으로 메모리에 배치된다.

| 메모리 주소   | 저장되는 값    |
|----------|-----------|
| 가장 낮은 주소 | 40h (LSB) |
| +1       | 4Bh       |
| +2       | 4Ch       |
| 가장 높은 주소 | 00h (MSB) |

즉, 메모리에는 다음과 같이 저장된다.

40 4B 4C 00

겉보기에는 바이트 순서가 “거꾸로” 저장된 것처럼 보이지만, 이는 리틀 엔디언 아키텍처의 정상적인 동작이다.

CPU 는 내부적으로 이 바이트들을 올바르게 재조합하여 0x004C4B40 이라는 값을 정확히 해석한다.

### [2-4-4] 중요한 개념 정리

리틀 엔디언 방식의 핵심은 다음 한 문장으로 정리할 수 있다.

LSB 는 낮은 주소에, MSB 는 높은 주소에 저장된다.

이 개념은 단순한 이론이 아니라 다음과 같은 상황에서 매우 중요하다.

- 메모리 덤프 분석
- 디버거에서 메모리 확인
- 어셈블리 프로그래밍
- 네트워크 바이트 순서 이해
- 파일 포맷 분석
- 시스템 프로그래밍

특히, 디버거에서 메모리를 확인할 때 값이 “거꾸로 저장된 것처럼” 보이는 이유는 바로 리틀 엔디언 때문이다.

## [2-4-5] 왜 이 개념이 중요한가

고급 언어(C, C++, Java 등)에서는 엔디언 방식을 거의 의식하지 않아도 된다.  
그러나 어셈블리 수준에서는 다음을 직접 이해해야 한다.

- 메모리 주소 증가 방향
- 데이터 배치 방식
- 레지스터에 로드될 때의 재조합 과정

예를 들어,

```
mov eax, [var1]
```

명령이 실행되면 CPU 는 메모리에서 4 바이트를 읽어 내부적으로 다시 조합하여 하나의 32 비트 값으로 만든다.

이때 CPU 는 리틀 엔디언 규칙에 따라 바이트를 해석하므로, 프로그래머는 메모리 배치를 정확히 이해하고 있어야 한다.

## [2-4-6] 레지스터와 메모리의 관계

메모리는 바이트 단위로 저장된다.  
그러나 레지스터는 다음과 같은 크기로 동작한다.

- 8 비트
- 16 비트
- 32 비트
- 64 비트

따라서 메모리에서 레지스터로 데이터를 가져올 때는:

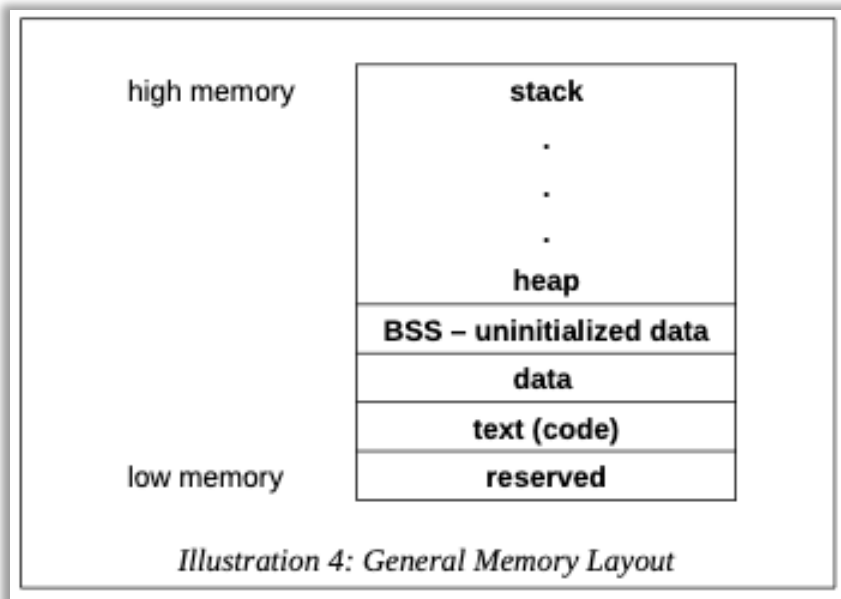
1. 지정한 크기만큼 연속된 바이트를 읽는다.
2. 리틀 엔디언 규칙에 따라 내부적으로 재조합한다.
3. 레지스터에 하나의 값으로 저장한다.

즉, 메모리는 항상 바이트의 집합이지만, CPU 는 그것을 “의미 있는 정수 값”으로 재해석하여 사용한다.

## [2-5] 메모리 배치(Memory Layout)

프로그램이 실행될 때, 운영체제는 해당 프로그램을 메모리에 특정한 구조로 배치한다. 이 구조는 일반적으로 여러 개의 영역으로 나뉘며, 각 영역은 서로 다른 목적과 특성을 가진다. 이러한 메모리 배치 구조를 이해하는 것은 어셈블리 프로그래밍, 그리고 디버깅 과정에서 매우 중요하다.

일반적인 프로그램의 메모리 배치는 다음과 같은 영역들로 구성된다.



### [2-5-1] 예약 영역(Reserved Section)

예약 영역은 사용자 프로그램이 접근할 수 없는 영역이다.

이 영역은 운영체제나 시스템 내부에서 사용되며, 프로그램이 직접 읽거나 쓸 수 없다. 잘못 접근할 경우 보호 예외(Protection Fault)가 발생할 수 있다. 따라서 이 영역은 사용자 코드와는 분리된 보호된 공간이라고 이해하면 된다.

### [2-5-2] 텍스트(Text) 영역/ 코드(Code) 영역

텍스트 영역, 또는 코드 영역은 프로그램의 실제 실행 코드가 저장되는 공간이다.

이 영역에서는 컴파일을 거쳐 생성된 기계어(machine language)가 저장된다. 즉, CPU가 직접 실행하는 명령어들이 1과 0의 비트열 형태로 이 영역에 배치된다.

특징은 다음과 같다.

- 실행 명령어가 저장도니다.
- 일반적으로 읽기 전용(read-only) 속성을 가진다.
- 프로그램 실행 동안 변경되지 않는다.

어셈블리 관점에서는 우리가 작성한 명령어들이 바로 이 영역에 위치하게 된다.

### [2-5-3] 데이터(Data) 영역

데이터 영역에는 초기값이 있는 전역 변수와 초기값이 있는 정적 변수가 저장된다.

즉, 어셈블 시점(컴파일 시점)에 이미 값이 결정되어 있는 변수들이 이 영역에 배치된다.

예를 들어,

```
int global_var = 10;  
static int count = 5;
```

이와 같이 초기값이 명시된 변수들은 데이터 영역에 저장된다.

특징은 다음과 같다.

- 프로그램 시작 시 이미 값이 설정되어 있다.
- 프로그램 실행 내내 메모리에 존재한다.
- 전역 및 정적 변수에 해당한다.

### [2-5-4] 초기화되지 않은 데이터 영역(BSS Section)

초기화되지 않은 데이터 영역은 일반적으로 BSS 섹션이라고 불린다.

이 영역에서는 선언은 되었지만 초기값이 지정되지 않은 변수들이 저장된다.

예를 들어,

```
int global_var;  
static int count;
```

이러한 변수들은 메모리에 공간만 할당되며, 어셈블 시점에 특정 값으로 초기화되지 않는다.

주의할 점은 다음과 같다.

- 값을 설정하기 전에 접근하면 의미 없는 값이 될 수 있다.
- 프로그램 시작 시 운영체제가 0으로 초기화되는 경우가 일반적이다.
- 전역 변수와 정적 변수 중 초기값이 없는 것들이 이 영역에 위치한다.

데이터 영역과 BSS 영역의 차이는 “초기값이 있느냐 없느냐”이다.

### [2-5-5] 힙(Heap)

힙은 프로그램 실행 중 동적으로 할당되는 데이터가 저장되는 영역이다.  
즉, 실행 시간(runtime)에 크기가 결정되어 할당되는 메모리 공간이다.

예를 들어,

- C 언어의 malloc()
- C++의 new
- 동적 배열 생성

이와 같은 동적 메모리 할당이 이루어질 때 힙 영역이 사용된다.

특징은 다음과 같다.

- 런타임에 크기가 결정된다.
- 프로그래머가 직접 할당 및 해제를 관리해야 한다.
- 일반적으로 낮은 주소에서 높은 주소 방향으로 성장한다.

### [2-5-6] 스택(Stack)

스택은 메모리의 높은 주소 영역에서 시작하여 아래쪽(낮은 주소 방향)으로 성장하는 영역이다.

주로 다음과 같은 정보가 저장된다.

- 함수 호출 시 의 반환주소
- 함수의 매개변수
- 지역 변수
- 레지스터 백업 값

함수가 호출될 때마다 새로운 스택 프레임(Stack Frame)이 생성되고, 함수가 종료되면 해당 프레임은 제거된다.

스택의 특징은 다음과 같다.

- 자동으로 생성 및 소멸된다.
- 함수 호출 구조와 밀접하게 연관된다.
- LIFO(Last In, First Out) 구조를 따른다.
- 높은 주소 → 낮은 주소 방향으로 성장한다.

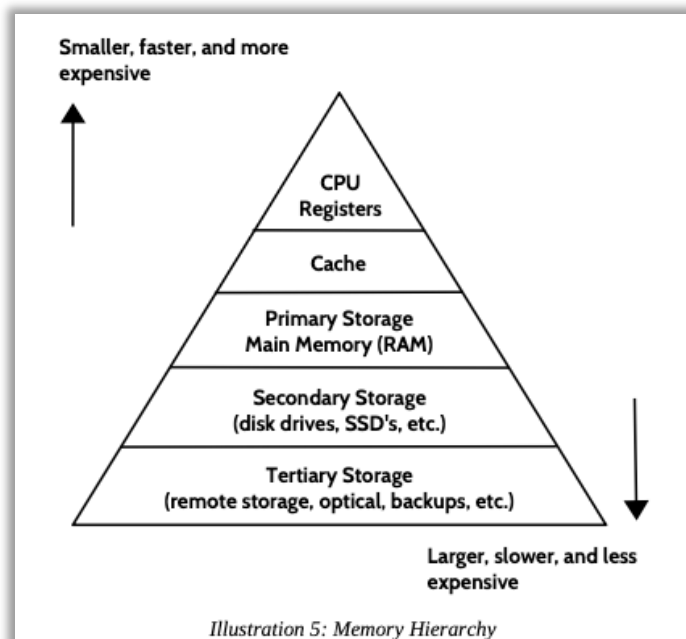
## [2-6] 메모리 계층 구조(Memory Hierachy)

서로 다른 메모리 계층과 그 사용 방식을 제대로 이해하기 위해서는 메모리 계층 구조(Memory Hierachy)를 이해하는 것이 매우 중요하다. 컴퓨터 시스템에는 다양한 종류의 메모리가 존재하며, 이들은 속도, 용량, 비용 측면에서 서로 다른 특성을 가진다.

일반적인 원칙은 다음과 같다.

- 속도가 빠를수록 가격이 비싸고 용량이 작다.
- 속도가 느릴수록 용량이 크고 가격이 저렴하다.

즉, 성능과 비용 사이에는 항상 트레이드오프(trade-off)가 존재한다.



### [2-6-1] 메모리 계층 구조의 기본 개념

예를 들어, CPU 내부의 레지스터(register)는 매우 빠르지만, 크기가 매우 작고 비용이 높다. 반면, 디스크 드라이브나 SSD 와 같은 보조 저장 장치는 용량이 매우 크지만 상대적으로 속도가 느리고 비용이 낮다.

이처럼 메모리는 여러 단계로 나뉘며, 각 단계보다 빠르지만 작고 비싸다.

메모리 계층은 구조의 궁극적인 목적은 다음과 같다.

- 작은 용량의 빠른 메모리와 큰 용량의 느린 메모리 사이에서 성능과 비용의 균형을 맞추는 것

## [2-6-2] 메모리 계층 구조의 형태

메모리 계층 구조는 일반적으로 삼각형 형태로 표현된다.

- 삼각형의 상단 → 가장 빠르고, 가장 작으며, 가장 비싼 메모리
- 아래로 내려갈수록 → 속도는 느려지고, 용량은 커지며, 비용은 낮아진다.

즉, 위쪽 = 빠름/작음/비쌈

아래쪽 = 느림/큼/저렴함

이 구조 덕분에 컴퓨터는 전체 비용을 크게 증가시키지 않으면서도 높은 성능을 유지할 수 있다.

## [2-6-3] 일반적인 메모리 계층별 특성

다음은 일반적인 메모리 계층과 그 특성이다.

| 메모리 단위              | 예시 크기          | 일반적인 접근 속도      |
|---------------------|----------------|-----------------|
| 레지스터                | 16개, 64비트 레지스터 | 약 1 나노초         |
| 캐시 메모리 (L1, L2)     | 4 ~ 8MB 이상     | 약 5 ~ 60 나노초    |
| 주기억장치 (RAM)         | 2 ~ 32GB 이상    | 약 100 ~ 150 나노초 |
| 보조 저장 장치 (SSD, 디스크) | 500GB ~ 4TB 이상 | 약 3 ~ 15 밀리초    |

이 표를 보면 계층 간의 속도 차이가 매우 크다는 것을 알 수 있다.

예를 들어,

- 주기억장치 접근 시간: 약 100 나노초
- 보조 저장 장치 접근 시간: 약 3 밀리초

이를 비교하면,

3 밀리초 = 3,000,000 나노초

즉, 주기억장치 접근은 보조 저장 장치 접근보다 약 30,000 배 빠르다.

이 차이는 단순한 “조금 느림”의 수준이 아니라, 수만 배의 차이이다.

## [2-6-4] 왜 메모리 계층 구조가 필요한가?

만약 모든 메모리를 레지스터처럼 빠르게 만들려고 한다면:

- 비용이 엄청나게 증가하고
- 기술적으로 구현 매우 어렵다.

반대로 모든 메모리를 디스크처럼 만들면:

- 비용은 낮아지지만
- 프로그램 실행 속도가 극도로 느려진다.

따라서 시스템은 다음과 같은 전략을 사용한다.

- CPU는 가장 자주 사용하는 데이터를 빠른 메모리에 두고
- 덜 사용하는 데이터는 점점 아래 계층에 둔다.

이것이 바로 캐시 메모리의 존재 이유이며, 메모리 계층 구조의 핵심 개념이다.

### [2-6-5] 상대적 속도 차이의 중요성

메모리 접근 속도는 시간이 지나면서 지속적으로 개선되고 있다. 최신 SSD는 과거의 디스크보다 훨씬 빠르다. RAM 역시 지속적으로 발전하고 있다.

그러나 중요한 점은 다음과 같다.

메모리 계층 간의 “상대적인 속도 차이”는 여전히 매우 크다

즉, 아무리 최신의 SSD를 사용하더라도:

- 레지스터 보다 빠를 수는 없고
- 캐시보다 빠를 수는 없으며
- RAM보다도 느리다.

이러한 근본적인 속도 차이 때문에 계층 구조는 여전히 필요하며, 현대 컴퓨터 시스템의 핵심 설계 원리가 된다.



## Chapter3. 데이터 표현(Data Representation)

데이터 표현(Data Representation)이란 정보가 컴퓨터 내부에서 어떻게 저장되는지를 의미한다.

컴퓨터는 모든 데이터를 결국 이진수(0 과 1) 형태로 저장한다.

그러나 데이터의 종류에 따라 저장 방식은 서로 다르다.

- 정수(integer) 저장 방식
- 부동소수점(floating-point) 저장 방식
- 문자(character) 저장 방식

은 각각 서로 다른 표현 규칙을 가진다.

이 장에서는 다음 세 가지를 다룬다.

1. 정수 표현 방식
2. 부동소수점 표현 방식
3. ASCII 문자 표현 방식

본 장에서는 독자가 이미 이진수, 십진수, 16 진수 체계에 대한 기본 지식을 가지고 있다고 가정한다. 별도로 명시되지 않은 숫자는 10 진수로 간주한다. 숫자 앞에 0x 가 붙으면 16 진수를 의미한다.

19 = 0x13

### [3-1] 정수 표현(Integer Representation)

#### [1] 정수 표현의 기본 개념

정수 표현이란 컴퓨터가 숫자를 메모리 안에 어떻게 저장하고 표현하는가에 대한 개념이다.

컴퓨터는 모든 숫자를 이진수로 저장한다.

그러나 중요한 점은 “각 변수는 사용할 수 있는 저장 공간의 크기가 제한되어 있다.”

이 저장 공간의 크기는 곧 표현 가능한 숫자의 범위를 결정한다.

#### [2] 저장 공간과 표현 가능한 값의 개수

비트 수가  $n$  이면 표현 가능한 값의 개수는:

$2^n$

이다.

예: 1 바이트(8 비트)

8 비트  $\rightarrow 2^3 = 256$  개의 서로 다른 값 표현 가능

이 256 개의 값은 해석 방식에 따라 달라진다.

### [3] 부호 없는 정수 (Unsigned Integer)

부호 없는 값은 일반적인 이진수 해석을 사용한다.

8 비트 unsigned 범위:

0 ~ 255

즉,

00000000 = 0

11111111 = 255

### [4] 부호 있는 정수 (Signed Integer)

부호 있는 정수는 2의 보수(two's complement) 방식으로 표현된다.

8 비트 signed 범위:

-128 ~ +127

즉,

10000000 = -128

01111111 = +127

여기서 중요한 점은:

양수 영역에서 일반 이진 표현과 동일하고

음수 영역에서만 2의 보수 표현이 적용된다.

### [5] 저장 단위에 따른 표현 범위

저장 단위가 커질수록 표현 가능한 범위도 커진다.

| 크기       | 비트 수  | 부호 없음 범위             | 부호 있음 범위                        |
|----------|-------|----------------------|---------------------------------|
| 바이트      | 8비트   | 0 ~ 255              | -128 ~ +127                     |
| 워드       | 16비트  | 0 ~ 65,535           | -32,768 ~ +32,767               |
| 더블 워드    | 32비트  | 0 ~ 4,294,967,295    | -2,147,483,648 ~ +2,147,483,647 |
| 쿼드 워드    | 64비트  | $0 \sim 2^{64} - 1$  | $-2^{63} \sim 2^{63} - 1$       |
| 더블 쿼드 워드 | 128비트 | $0 \sim 2^{128} - 1$ | $-2^{127} \sim 2^{127} - 1$     |

#### [6] 예시: 저장 크기의 필요성

예를 들어,

100,000

이라는 값을 저장하려고 할 경우:

- 1 바이트(최대 255) → 불가능
- 2 바이트(최대 65,535) → 불가능
- 4 바이트(최대 4,294,967,295) → 가능

따라서 이 값은 최소 32 비트 더블 워드가 필요하다.

#### [7] 고급 언어에서의 정수

C, C++, Java

int 변수

는 일반적으로 32 비트 더블워드를 사용한다.

이 경우 표현 가능한 범위는:

$-2^{31} = -2,147,483,648$   
 $+2^{31} - 1 = +2,147,483,647$

이다.

#### [8] 부호 있는 값과 부호 없는 값의 차이

어떤 값이 저장 가능한지 판단하려고 하면 다음 두 가지를 알아야 한다.

1. 저장단위의 크기(8 비트, 16 비트, 32 비트 등)
2. 값이 signed 인지, unsigned 인지

이 두 조건이 달라지면 같은 비트 패턴도 완전히 다른 값으로 해석된다.

#### [9] 비트 패턴 해석의 차이

접치는 범위(0~ +127)

예를 들어, 8 비트에서:

0x0C

- signed = 12
- unsigned = 12

완전히 동일하다.

접치지 않는 범위

0xF1

- signed = -15
- unsigned = 241

같은 8 비트 값이지만 해석 방식에 따라 완전히 다른 숫자가 된다.  
이것이 디버거로 메모리를 볼 때 혼란이 발생하는 이유다.

#### [10] 시각적 범위 비교

부호 없는 바이트:

0 ----- 255

부호 있는 바이트:

-128 ----- 0 ----- +127

구조는 동일하지만, 해석 방식이 다르다.

#### [3-1-1] 2의 보수(Two's Complement)

이 절에서는 음수 값을 표현하기 위한 2의 보수 방식을 설명한다.  
2의 보수는 음수 표현에만 적용되며, 양수에는 적용되지 않는다.

컴퓨터에서 부호 있는 정수(signed integer)는 2의 보수 방식을 사용하며 음수를 표현한다.

##### [1] 2의 보수를 구하는 절차

어떤 수의 2의 보수를 구하는 과정은 다음 두 단계로 이루어진다.

1. 1의 보수를 취한다. → 모든 비트를 반전한다(0은 1, 1은 0)
2. 1을 더한다(이진수 기준)

이 두 단계를 수행하면 해당 값의 2의 보수가 된다.

##### [2] 중요한 특징

이 동일한 과정은 다음 두 경우 모두에 사용된다.

- 10진수 값을 2의 보수 형태로 변환할 때
- 2의 보수 값을 다시 10진수로 해석할 때

즉, 변환과 역변환에 동일한 원리가 적용된다.

### [3-1-2] 바이트 예제(8 비트)

이제 바이트(8 비트) 크기에서 음수 값을 2의 보수로 표현해보자.

#### [1] -9의 2의 보수 표현(8 비트)

1 단계: +9를 8비트 이진수로 표현

$$9 = (8 + 1) = 00001001$$

2 단계: 1의 보수(비트 반전)

$$11110110$$

3 단계: 1을 더함

$$\begin{array}{r} 11110110 \\ + \quad 1 \\ \hline 11110111 \end{array}$$

따라서

$$-9 \text{의 } 2 \text{의 보수 (8 비트)} = 11110111_2 = 0xF7$$

#### [2] -12의 2의 보수 표현(8 비트)

1 단계: +12를 8비트 이진수로 표현

$$12 = (8 + 4) = 00001100$$

2 단계: 1의 보수(비트 반전)

$$11110011$$

3 단계: 1을 더함

$$\begin{array}{r} 11110011 \\ + \quad 1 \\ \hline 11110100 \end{array}$$

따라서,

$$-12 \text{의 } 2 \text{의 보수 (8 비트)} = 11110100_2 = 0xF4$$

### [3] 주의 사항(바이트 예제)

여기서 매우 중요한 점은 다음과 같다.

지정된 데이터 크기(여기서는 8 비트)에 해당하는 모든 비트를 반드시 명시해야 한다.

예를 들어, 8 비트가 아닌 16 비트로 표현하면 결과는 완전히 달라진다.

비트 수가 바뀌면 동일한 값이라도 전혀 다른 수로 해석된다.

### [3-1-2] 워드 예제(16 비트)

이번에는 워드 크기(16 비트)에서 음수를 표현해보자.

#### [1] -18 의 2 의 보수 표현(16 비트)

1 단계: + 18 을 16 비트 이진수로 표현

$18 = (16 + 2) = 0000\ 0000\ 0001\ 0010$

2 단계: 1 의 보수(비트 반전)

1111 1111 1110 1101

3 단계: 1 을 더함

```
1111 1111 1110 1101
+                   1
-----
1111 1111 1110 1110
```

따라서

-18 의 16 비트 2 의 보수 표현 = 1111 1111 1110 1110<sub>2</sub>  
= 0xFFEE

#### [2] -40 의 2 의 보수 표현(16 비트)

1 단계: + 40 을 16 비트 이진수로 표현

$40 = (32 + 8) = 0000\ 0000\ 0010\ 1000$

2 단계: 1 의 보수(비트 반전)

1111 1111 1101 0111

3 단계: 1 을 더함

1111 1111 1101 0111

+ 1

-----

1111 1111 1101 1000

따라서

-40 의 16 비트 2 의 보수 표현 = 1111 1111 1101 1000<sub>2</sub>

= 0xFFD8

### [3] 주의 사항(워드 예제)

바이트 예제와 마찬가지로, 해당 데이터 크기(여기서는 16 비트)의 모든 비트를 반드시 명시해야 한다.

예를 들어,

- -18 을 8 비트로 표현한 값

- -18 을 16 비트로 표현한 값

은 서로 다르다.

즉, 비트 수가 달라지면 같은 값이라도 전혀 다른 수가 된다.

### [3-1-3] 2 의 보수의 본질적 의미

2 의 보수 방식은 단순히 음수를 표현하기 위한 기술이 아니다.

이 방식의 핵심 목적은 다음과 같다.

- 덧셈 회로 하나로 양수 음수를 모두 처리하기 위함

예를 들어,

5 + (-5)

를 계산하면,

00000101

+

11111011 (-5 의 2 의 보수)

-----

00000000 (자리올림은 버림)

결과는 0 이 된다.

이처럼 2 의 보수는:

- 뿔셈을 덧셈으로 처리 가능하게 하고
- 하드웨어를 단순화하며
- 부호 비트를 별도로 관리하지 않아도 되도록 만든다.

### [3-2] 부호 없는 덧셈과 부호 있는 덧셈

앞서 설명했듯이, 부호 없는(unsigned) 표현과 부호 있는(signed) 표현은 동일한 비트 패턴이라도 서로 다른 값으로 해석될 수 있다.

그러나 여기서는 매우 중요한 사실은 다음과 같다.

- 덧셈과 뿔셈 연산 자체는 부호 여부와 관계없이 동일하게 수행된다.

즉, CPU는 단순히 비트 단위 연산(binary arithmetic)만 수행하며, 그 결과를 어떻게 해석할 것인지는 프로그래머가 지정한 데이터 타입에 따라 달라진다.

|       |          |      |          |
|-------|----------|------|----------|
| 241   | 11110001 | -15  | 11110001 |
| + 7   | 00000111 | + 7  | 00000111 |
| 248   | 11111000 | -8   | 11111000 |
| 248 = | F8       | -8 = | F8       |

#### [3-2-1] 동일한 연산, 다른 해석

예를 들어, 어떤 연산의 결과가 다음과 같다고 가정하자.

0xF8

이 값은 8 비트 기준으로 보면 다음과 같은 이진수이다.

11111000<sub>2</sub>

이 동일한 비트 패턴은 해석 방식에 따라 전혀 다른 의미를 가진다.

#### [1] 부호 없는(unsigned) 해석

11111000<sub>2</sub> = 248

따라서, unsigned 값으로는 248 이 된다.



[2]부호 있는(signed) 해석(2의 보수)

최상위 비트(MSB)가 1 이므로 음수이다.

2의 보수를 해석을 적용하면

1 단계: 비트 반전

00000111

2 단계: 1을 더함

00001000 = 8

따라서,

$11111000_2 = -8$

즉,

- unsigned 해석: 248
- signed 해석: -8

[3-2-2] 동일한 비트 패턴의 또 다른 해석

0xF8은 단지 정수로만 해석되는 것은 아니다.

ASCII 문자표에서 :

$0xF8 = \text{° (도 기호)}$

즉, 동일한 8비트 값이라도

- 정수로 해석할 수도 있고
- 부호 있는 정수로도 해석할 수도 있으며
- 문자로도 해석할 수도 있다.

### [3-2-3] CPU 는 “해석”하지 않는다

CPU 는 다음과 같이 동작한다.

- 비트 단위로 덧셈 수행
- 자리올림(carry) 계산
- 결과 저장
- 플래그 설정

그러나 CPU 는 이것이:

- signed 연산인지
- unsigned 연산인지
- 문자 데이터 인지

를 “알지 못한다”

CPU 는 단지 비트를 계산할 뿐이며, 해석은 프로그래머 또는 컴파일러의 책임이다.

### [3-2-4] 오버플로우와 캐리의 차이

부호 있는 연산과 부호 없는 연산의 차이는

결과 자체가 아니라 플래그 해석 방식에서 나타난다.

- unsigned 연산 → Carry Flag(CF) 중요
- signed 연산 → Overflow Flag(OF) 중요

예를 들어,

250 + 10 (unsigned 8 비트)

```
11111010 (250)
+
00001010 (10)
-----
00000100
```

결과는 4 가 되지만, 자리올림이 발생한다.

→ unsigned 에서는 오버플로우 발생

반면, signed 연산에서는 다른 조건으로 오버플로우를 판단한다.

### [3-2-5] 왜 데이터 크기와 타입 정의가 중요한가

동일한 비트 패턴이라도 다음 요소에 따라 의미가 달라진다.

- 데이터 크기(8 비트, 16 비트, 32 비트 등)
- 부호 여부(signed/unsigned)
- 자료형 해석(정수/문자/포인터 등)

따라서 연산을 수행할 때에는 반드시 다음을 명확히해야 한다.

1. 데이터의 저장 크기
2. 부호 여부
3. 해석 방식

이를 명확히 하지 않으면,

- 디버깅 시 혼란 발생
- 예상치 못한 오버플로우
- 잘못된 비교 연산
- 보안 취약점 발생

등의 문제가 발생할 수 있다.

### [3-3] 부동소수점 표현(Floating-point Representation)

정수와 달리, 실수(real number)는 유한한 비트 수로 정확하게 표현하기 어렵다.

그 이유는 실수가

- 매우 큰 값
- 매우 작은 값
- 서로 다른 정밀도 요구

를 동시에 가지기 때문이다.

예를 들어,

- 1,000,000
- 0.0000001
- 3.1415926535

값은 값들을 모두 동일한 방식으로 저장하기는 어렵다.

### [1] 고정 소수점의 한계

만약 소수점 위치를 고정하여 저장하는 고정소수점 방식을 사용한다면

- 표현 가능한 범위가 매우 제한되고
- 큰 수를 저장하면 작은 수의 정밀도가 떨어지거나
- 작은 수를 정확히 저장하면 큰 수를 표현하기 어려워진다.

이러한 문제를 해결하기 위해 컴퓨터는

과학적 표기법(scientific notation)을 기반으로 한 부동소수점(floating-point) 방식을 사용한다.

### [2] 부동소수점의 핵심 아이디어

부동소수점 표현의 핵심 아이디어는 다음과 같다.

값을 세 가지 요소로 나누어 저장한다.

1. 부호(sign)
2. 크기(scale, 지수 Exponent)
3. 정밀한 값(Detail, 가수 Mantissa 또는 Fraction)

즉,

$$\text{값} = (-1)^{\text{부호}} \times 1.\text{가수} \times 2^{(\text{지수})}$$

이 구조를 표준화한 것이 IEEE 754 부동소수점 표준이다.

### [3-3-1] IEEE 754 32 비트 표현 방식

IEEE 754 32 비트 부동소수점 수(single precision)는

총 32 개의 비트를 다음과 같이 세 부분으로 나누어 사용한다.

총 32 비트는 다음과 같이 구성된다

| 구분                  | 비트 수 | 역할                |
|---------------------|------|-------------------|
| Sign                | 1비트  | 부호 (0: 양수, 1: 음수) |
| Exponent            | 8비트  | 지수 (크기 조절)        |
| Fraction (Mantissa) | 23비트 | 정밀한 값             |

비트 구조는 다음과 같다.

[ S ][ Exponent (8bit) ][ Fraction (23bit) ]

## [1] 각 요소의 의미

### [1-1] Sign(부호)

- 0: 양수
- 1: 음수

값의 부호만을 결정하며, 크기에는 영향을 주지 않는다.

### [1-2] Exponent(지수)

값의 크기(scale)를 결정한다.

하지만 실제 지수 값을 그대로 저장하지 않는다.

IEEE 754 는 바이어스(bias)가 적용된 값을 저장한다.

32 비트 형식에서

$$\text{Bias} = 127$$

$$\text{저장된 지수} = \text{실제 지수} + 127$$

따라서 실제 지수는 다음과 같이 계산된다.

$$\text{실제 지수} = \text{저장된 지수} - 127$$

### [1-3] Fraction(가수)

값의 정밀한 소수 부분을 표현한다.

IEEE 754 는 정규화된 이진 표현을 사용하기 때문에 항상 다음 형태로 저장된다.

$$1.F \times 2^E$$

여기서 중요한 점은 앞의 1 은 저장되지 않는다.

이를 hidden bit(implicit leading 1)라고 한다.

## [2] IEEE 754 수의 해석 공식

위 세 요소를 결합하면 IEEE 754 수는 다음과 같은 수식으로 해석된다.

$$N = (-1)^S \times 1.F \times 2^{E-127}$$

### [2-1] 이진수에서의 소수 표현과 한계

부동소수점 이해의 출발점은 이진수에서 소수를 표현하는 방식이다.

이진수 소수점은 오른쪽 자릿값은 다음과 같이 가중치를 가진다.

... 8 4 2 1 . 1/2 1/4 1/8 ...

예를 들어,

$$100.101_2 = 4 + 0 + 0 + 0 + 1/2 + 0 + 1/8 = 4.625$$

그러나, 중요한 사실은 모든 10 진수 소수가 이진수로 유한하게 표현되지 않는다.  
많은 값은 이진수로 변환하면 무한 반복 소수가 된다.

컴퓨터는 유한한 비트 수만 사용할 수 있으므로 반올림과 근사가 필연적으로 발생한다.  
이것이 바로 **부동소수점 오차의 근본 원인** 이다.

## [2] 정규화된 이진 과학적 표기법

IEEE 754 는 모든 값을 정규화된 이진 과학적 표기법으로 저장한다.

정규화란,  
소수점 왼쪽에 항상 1 하나만 남기도록 만드는 것

즉, 모든 값은 다음 형태가 된다.

$$1.xxxxx \times 2^n$$

예시

$$\begin{aligned} - 8.125^{10} &\rightarrow 1000.001^2 \rightarrow 1.000001 \times 2^3 \\ - 0.125^{10} &\rightarrow 0.001^2 \rightarrow 1.0 \times 2^{-3} \end{aligned}$$

정규화된 이진수에서는 항상 맨 앞자리가 1 이므로, IEEE 754 는 이 1 을 저장하지 않는다.  
이것이 바로 hidden bit 개념이다.

이 설계 덕분에 같은 23 비트로 더 높은 정밀도를 얻을 수 있다.

### [3] 바이어스 지수의 존재 이유

지수는 양수와 음수를 모두 표현해야 한다.

그러나 지수부는 부호 비트 없이 저장된다.

이를 가능하게 하기 위해 bias 방식을 사용한다.

32 비트에서

Bias = 127

저장값 = 실제 지수 + 127

이 방식의 장점:

- 지수를 unsigned 정수처럼 비교 가능
- 하드웨어 구현이 단순해짐
- 연산 속도 향상

### [4] 부동소수점 값을 해석하는 과정

IEEE 754 비트 패턴을 실제 수로 해석하는 절차는 다음과 같다.

1. 부호 비트를 확인한다.
2. 지수부에서 바이어스를 빼서 실제 지수를 구한다.
3. 가수부 앞에 숨겨진 1.을 복원한다.
4. 다음 형태로 계산한다.

$1.F \times 2^E$

이때 중요한 점은,

- 저장된 값은 “정확한 값”이 아니라
- “가장 가까운 근사값”일 수 있다는 것이다.

### [5] 왜 0.1 은 정확히 표현되지 않는가

10 진수 0.1 을 이진수로 변환하면 무한 반복 소수가 된다.

따라서 23 비트 가수부로는 정확히 표현할 수 없다.

메모리에 저장되는 값은 0.1 이 아니라, 0.1 에 가장 가까운 이진 근사값이다.

이로 인해 다음과 같은 현상이 발생한다.

$0.1 + 0.2 \neq 0.3$

이는 구현 오류가 아니라,

부동소수점 표현 방식의 구조적 특성 때문이다.

#### [5-1] 10 진수 “소수”를 2 진수로 바꾸는 법

정수는 2로 나누면서 변환하지만 소수는 반대로 2를 계속 곱한다.

|                                    |
|------------------------------------|
| $0.1 \times 2 = 0.2 \rightarrow 0$ |
| $0.2 \times 2 = 0.4 \rightarrow 0$ |
| $0.4 \times 2 = 0.8 \rightarrow 0$ |
| $0.8 \times 2 = 1.6 \rightarrow 1$ |
| $0.6 \times 2 = 1.2 \rightarrow 1$ |
| $0.2 \times 2 = 0.4 \rightarrow 0$ |
| $0.4 \times 2 = 0.8 \rightarrow 0$ |
| $0.8 \times 2 = 1.6 \rightarrow 1$ |
| ...                                |

0.1이 유한한 이진수로 표현되려면 어느 시점에서 소수 부분이 0이 되어야 한다.  
하지만 0이 되지 않고 반복된다.

즉,

|                                                  |
|--------------------------------------------------|
| $0.1_{10} =$<br>$0.0001100110011001100110011...$ |
|--------------------------------------------------|

#### [5-2] IEEE 754는 23 비트만 저장 가능

32 비트 float 구조:

|                |
|----------------|
| 1 비트 sign      |
| 8 비트 exponent  |
| 23 비트 fraction |

fraction은 23비트까지만 저장 가능  
하지만 0.1은

|                                  |
|----------------------------------|
| $0.0001100110011001100110011...$ |
|----------------------------------|

무한히 반복되는 것을 알 수 있다.

결국 23비트에서 잘라야 한다. 실제 저장된 값은 다음과 같다.

|                                    |
|------------------------------------|
| 0 01111011 10011001100110011001101 |
|------------------------------------|

16진수로는:

|            |
|------------|
| 0x3DCCCCCD |
|------------|

이 값이 실제로 의미하는 10진수는:

|                               |
|-------------------------------|
| 0.100000001490116119384765625 |
|-------------------------------|



[6] 예제: +5.75

1 단계: 이진수로 변환

$$5.75 = 101.11_2$$

2 단계: 정규화

$$1.0111 \times 2^2$$

3 단계: 각 부분 채우기

sign

0 (양수)

exponent

실제 지수 = 2

저장 값 = 2 + 127 = 129

$$129 = 10000001_2$$

fraction

0111 뒤에 0 으로 채워서 23 비트

4 단계: 최종 표현

0 10000001 01110000000000000000000

[3-3-3] IEEE 64 비트 표현 방식(Double Precision)

64 비트 부동소수점은 다음과 같이 구성된다.

63 | 62 ..... 52 | 51 ..... 0 |  
s | biased exponent | fraction |

구조는 32 비트와 동일하지만,

비트 수가 확장되었다.

| 항목       | 32비트 | 64비트 |
|----------|------|------|
| Sign     | 1비트  | 1비트  |
| Exponent | 8비트  | 11비트 |
| Fraction | 23비트 | 52비트 |

64 비트의 bias 값은:

1023

지수 범위가 훨씬 넓어지고, 가수부가 52 비트이므로 정밀도도 크게 향상된다.

### [3-3-4] 숫자가 아님(Not a Number, NaN)

IEEE 754 형식을 만족하지 못하거나,  
정상적인 수로 표현할 수 없는 경우

해당 값은 NaN(Not a Number)로 처리된다.

발생 예:

- 정수 값을 부동소수점으로 잘못 해석한 경우
- 0 으로 나누기(0/0)
- 표현 범위를 초과한 연산 결과

NaN 은 “숫자가 아님”을 의미하며, 정상적인 실수 값으로 사용할 수 없다.

### [3-4] 문자와 문자열(Characters and Strings)

프로그램은 수치 데이터뿐만 아니라,  
기호적(symbolic) 데이터, 즉 비수치 데이터도 다룬다.

예를 들어,

“Hello World”

와 같은 메시지는 인간에게는 의미 있는 문장이지만,  
컴퓨터 메모리는 숫자만 저장하도록 설계되어 있다.

따라서 컴퓨터는 각 문자에 특정한 숫자 값을 대응시켜 저장한다.  
즉, 문자를 저장한다는 것은 실제로는 “문자에 대응되는 숫자”를 저장하는 것이다.

#### [3-4-1] 문자 표현

컴퓨터에서 문자(character)란  
하나의 기호에 대응되는 정보 단위를 의미한다.  
문자의 예:

- 알파벳 문자
- 숫자 문자
- 구두점
- 공백
- 제어 문자

### [1] 제어 문자(Control Character)

제어 문자는 화면에 출력되는 기호는 아니지만,  
텍스트 처리를 위해 필요한 특수 기능을 수행한다.

예:

- 캐리지 리턴(CR)
- 줄 바꿈(LF)
- 탭(Tab)

### [2] 정보 교환을 위한 미국 표준 코드(ASCII, American Standard Code for Information Interchange)

문자를 숫자로 표현하기 위한 가장 기본적인 표준이 ASCII 이다.

ASCII 는 각 문자와 제어 문자에 고유한 숫자 값을 할당한다.

예시:

| 문자  | 10진수 | 16진수 |
|-----|------|------|
| 'A' | 65   | 0x41 |
| '9' | 57   | 0x39 |

동작 방식

- 메모리에는 0x41 이 저장됨
- 콘솔에 출력하면 → “A”가 표시됨

### [2-1] 중요한 개념: 문자 vs 정수

예를 들어,

- 문자 '9' → 0x39(10 진수 57)
- 정수 9 → 0x09

이 둘은 전혀 다르다.

만약 정수 0x09 를 ASCII 문자로 출력하면 이는 탭(Tab) 문자로 해석된다.

### [2-2] 차이 정리

| 구분    | 문자 '2' | 정수 2     |
|-------|--------|----------|
| 메모리 값 | 0x32   | 0x02     |
| 출력 가능 | O      | 직접 출력 불가 |
| 계산 가능 | X      | O        |

- 문자는 계산용이 아니다.
- 정수는 표현 변환 없이 문자처럼 출력할 수 없다.

## [2-3] 문자 저장 크기

일반적으로:

문자 1 개 = 1 바이트(8 비트)

이는 대부분의 시스템이

바이트 단위로 주소를 지정(byte-addressable)하기 때문이다.

## [3] 유니코드(Unicode)

ASCII 는 영어권 중심의 문자 체계이다.

그러나 전 세계의 다양한 언어를 표현하기에는 부족하다.

이를 해결하기 위해 등장한 것이 유니코드(Unicode)이다.

### [3-1] 유니코드의 목표

- 모든 문자에 대해 고유한 숫자 값 제공
- 플랫폼/장치/언어와 무관하게 동일한 표현 유지

### [3-2] 주요 인코딩 방식

- UTF-8
- UTF-16
- UTF-32

가장 널리 사용되는 방식은 UTF-8 이다.

UTF-8 의 특징

- ASCII 문자와 완전히 호환
- 영어는 1 바이트 그대로 사용
- 다른 언어 문자에 대해서만 추가 바이트 사용

즉, 영어 텍스트는 기존 ASCII 와 동일하게 저장된다.

### [3-4-2] 문자열 표현

문자열(string)은

일련의 문자들이 연속적으로 저장된 구조이다.

일반적으로 NULL 문자(0x00)로 종료된다.

NULL 문자는 출력되지 않는 제어 문자이며, 문자열의 끝을 표시하는 역할을 한다.

[1] 예시: “Hello”

메모리에는 다음과 같이 저장된다.

| Character             | “H”  | “e”  | “l”  | “l”  | “o”  | NULL |
|-----------------------|------|------|------|------|------|------|
| ASCII Value (decimal) | 72   | 101  | 108  | 108  | 111  | 0    |
| ASCII Value (hex)     | 0x48 | 0x65 | 0x6C | 0x6C | 0x6F | 0x0  |

각 문자는 1 바이트이며, 마지막에 NULL(0x00)이 추가된다.

[2] 숫자로 구성된 문자열

| Character             | “1”  | “9”  | “6”  | “5”  | “3”  | NULL |
|-----------------------|------|------|------|------|------|------|
| ASCII Value (decimal) | 49   | 57   | 54   | 53   | 51   | 0    |
| ASCII Value (hex)     | 0x31 | 0x39 | 0x36 | 0x35 | 0x33 | 0x0  |

총 6 바이트 사용:

- 문자 5 개 (각 1 바이트)
- NULL 1 바이트

[2-1] 중요한 부분

문자열 “19653”과 정수 19,653 은 완전히 다르다.

| 항목     | 문자열 "19653" | 정수 19653         |
|--------|-------------|------------------|
| 저장 방식  | 문자 배열       | 이진 정수            |
| 메모리 사용 | 6바이트        | 2바이트(예: 16비트 정수) |
| 계산 가능  | X (변환 필요)   | O                |
| 출력 가능  | O           | 변환 필요            |

- 문자열은 “문자의 나열”이다.
- 정수는 “이진 수치 값”이다.

## Chapter4. 프로그램 형식

본 장에서는 어셈블리 언어 프로그램의 기본 형식 규칙을 요약한다.

설명은 yasm 어셈블러를 기준으로 하며, 다른 어셈블러에서는 일부 문법 차이가 있을 수 있다.

또한, 올바른 프로그램 구조를 이해할 수 있도록 완전한 어셈블리 프로그램 예제가 함께 제시된다.

### [1] 어셈블리 소스 파일의 기본 구성

올바르게 작성된 어셈블리 소스 파일은 일반적으로 다음과 같은 세 개의 주요 영역으로 구성된다.

#### [1-1] Data 섹션

- 초기화된 데이터가 선언 및 정의되는 영역
- 어셈블 시점에 값이 지정된 변수들이 저장됨

#### [1-2] BSS 섹션

- 초기화되지 않은 데이터가 선언되는 영역
- 저장 공간만 예약되며, 초기값은 지정되지 않음

#### [1-3] Text 섹션

- 실제 실행 코드가 위치하는 영역
- CPU가 실행할 명령어들이 포함됨

이 장에서는 기본적인 형식 규칙과 문법만 다루며, 추가적인 세부 사항은 yasm 레퍼런스 매뉴얼을 참고한다.

### [4-1] 주석(Comments)

어셈블리 언어에서

세미콜론(;)은 주석을 나타내는 기호이다.

- 주석은 소스 코드의 어느 위치에든 작성할 수 있다.
- 명령어 뒤에도 작성할 수 있다.
- 세미콜론 이후의 모든 내용은 어셈블러에 의해 무시된다.

예시:

```
mov rax, 5      ; rax 에 5 를 저장  
; mov rbx, 10   ; 이 명령어는 현재 비활성화됨
```

- 첫 줄의 주석은 설명용
- 두 번째 줄은 완전히 무시됨

## [4-2] 숫자 값(Numeric Values)

어셈블리 언어에서 숫자 값은 여러 진법으로 표현할 수 있다.

지원되는 진법은 다음과 같다.

- 10 진수(Decimal)
- 16 진수(Hexadecimal)
- 8 진수(Octal)

### [4-2-1] 10 진수(Decimal)

기본 진법(radix)은 10 진수이다.

따라서 10 진수 값은 별도의 표기 없이 그대로 작성하면 된다.

```
127  
1000  
42
```

### [4-2-2] 16 진수(Hexadecimal)

16 진수는 접두사 0x 를 사용하여 표현한다.

예를 들어, 10 진수 127 은 16 진수로 다음과 같이 표현된다.

```
0x7f
```

### [4-2-3] 8 진수(Octal)

8 진수는 숫자 뒤에, q 를 붙여 표현한다.

예를 들어, 10 진수 511 은 8 진수로 다음과 같이 표현된다.

```
777q
```

## [4-3] 상수 정의

어셈블리 언어에서는 equ 지시자를 사용하여 상수를 정의한다.

기본 형식:

```
<이름> equ <값>
```

예:

```
SIZE equ 10000
```

#### [4-3-1] 상수의 특징

1. 값은 실행 중 변경될 수 없다.
2. 상수는 어셈블리 과정에서 단순 치환(substitution)된다. 즉, 컴파일 전에 값으로 바뀐다.
3. 메모리 공간을 할당받지 않는다.
4. 특정 자료형이나 크기(byte, word, double-word 등)에 묶이지 않는다.

#### [4-3-2] 크기와 관계

상수는 자체적으로 크기를 가지지 않지만,  
사용할 때는 해당 문맥에서 요구하는 크기를 만족해야 한다.

예:

```
SIZE equ 10000
```

- 워드나 더블워드에서는 사용 가능
- 바이트 범위(0~255 또는 -128 ~ +127)에는 맞지 않으므로 사용 불가

#### [4-4] 데이터 섹션(Data Section)(\*)

초기화된 데이터는 반드시 다음 영역에 선언되어야 한다.

```
section .data
```

※ section 과 .data 사이에는 반드시 공백이 필요하다

#### [4-4-1] 데이터 섹션의 역할

.data 섹션에는 다음이 위치한다.

- 초기값이 있는 변수
- 초기화된 상수
- 초기화된 배열
- 초기화된 문자열
- 초기화된 부동소수점 값

즉, 프로그램 시작 시 이미 값이 정해져 있는 데이터가 저장되는 영역이다.

#### [4-4-2] 변수 이름 규칙

변수 이름은 다음 규칙을 따른다.

- 반드시 문자로 시작
- 이후에는
  - 문자
  - 숫자
  - 일부 특수 문자(예: `_`) 사용 가능



예:

```
myVar  
count1  
data_value
```

#### [4-4-3] 변수 정의 형식

변수를 정의할 때는 반드시 다음 세 요소가 필요하다.

- 변수 이름
- 데이터 타입
- 초기값

기본 형식

〈변수이름〉 〈res 타입〉 〈개수〉

#### [4-4-4] 지원되는 데이터 타입

| 선언  | 설명               |
|-----|------------------|
| db  | 8비트 변수           |
| dw  | 16비트 변수          |
| dd  | 32비트 변수          |
| dq  | 64비트 변수          |
| ddq | 128비트 변수 (정수)    |
| dt  | 128비트 변수 (부동소수점) |

※ d 는 define(정의하다)의 의미

이 지시자들은 초기화된 데이터 선언을 위한 기본 어셈블리 지시자이다.

#### [4-4-5] 초기화된 배열

덱표(,)로 구분된 여러 값을 나열하면 배열이 된다.

예시:

```
bVar db 10 ; 바이트 변수  
cVar db "H" ; 단일 문자  
strng db "Hello World" ; 문자열  
wVar dw 5000 ; 16 비트 변수  
dVar dd 50000 ; 32 비트 변수  
arr dd 100, 200, 300 ; 3 요소 배열  
flt1 dd 3.14159 ; 32 비트 부동소수점  
qVar dq 1000000000 ; 64 비트 변수
```

#### [4-4-6] 문자열 선언

문자열도 db 로 선언된다.

```
strng db "Hello World"
```

- 각 문자는 1 바이트
- ASCII 값으로 저장됨
- NULL 문자는 자동으로 붙이지 않음(필요하면 직접 추가해야 함)

#### [4-4-7] 중요한 제약 조건

초기값은 반드시 해당 데이터 타입의 표현 범위 내에 있어야 한다.

예:

- db(8 비트) → 0~255 또는 -128 ~ 127 범위
- dw(16 비트) → 0~65535 또는 -32768 ~ 32767

잘못된 예:

```
bVar db 500 ; 오류 발생 (8 비트 범위 초과)
```

- 어셈블리 에러 발생

#### [4-5] BSS 섹션

초기화되지 않은 데이터는 반드시 다음 영역에 선언된다.

```
section .bss
```

※ section 과 .bss 사이에는 반드시 공백이 필요하다

##### [4-5-1] BSS 섹션의 역할

.bss 섹션에는 다음이 선언된다.

- 초기값이 없는 변수
- 초기화되지 않은 배열
- 실행 중 사용할 공간만 예약되는 데이터

즉, 값은 정해져 있지 않고 공간만 확보하는 영역이다.

#### [4-5-2] 변수 이름 규칙

.data 섹션과 동일하다.

- 반드시 문자로 시작
- 이후에는
  - 문자
  - 숫자
  - 일부 특수 문자(예: `_`) 사용 가능

예:

```
buffer  
count1  
temp_value
```

#### [4-5-3] 변수 정의 형식

.bss 에서는 값을 주지 않는다.

대신 공간의 개수(count)를 지정한다.

기본 형식

〈변수이름〉 〈res 타입〉 〈개수〉

※ 여기서 res 는 reserve(예약하다)의 의미이다.

#### [4-5-4] 지원되는 데이터 타입

| 선언    | 크기    | 설명       |
|-------|-------|----------|
| resb  | 8비트   | 바이트 예약   |
| resw  | 16비트  | 워드 예약    |
| resd  | 32비트  | 더블워드 예약  |
| resq  | 64비트  | 쿼드워드 예약  |
| resdq | 128비트 | 128비트 예약 |

이 지시자들은 비초기화 데이터 선언을 위한 기본 어셈블러 지시자이다.

#### [4-5-5] 예제

```
bArr resb 10    ; 10 개의 바이트 배열
wArr resw 50    ; 50 개의 워드 배열
dArr resd 100   ; 100 개의 더블워드 배열
qArr resq 200   ; 200 개의 쿼드워드 배열
```

의미를 한 줄만 해석하면

- resb 10 : 1 바이트 × 10 개 공간 확보

#### [4-5-6] 중요한 특징

- .bss 는 공간만 예약한다.
- 어떤 값으로도 초기화되지 않는다.
- 초기 내용은 정의되지 않음(garbage 값일 수도 있음)

즉,

```
bArr resb 10
```

은 메모리에 10 바이트 공간을 확보할 뿐이며, 그 안의 값은 프로그램이 직접 설명하기 전까지는 알 수 없다.

#### [4-5-7] .data 와 .bss 의 차이 비교

| 구분    | .data         | .bss              |
|-------|---------------|-------------------|
| 초기값   | 있음            | 없음                |
| 문법    | <이름> <타입> <값> | <이름> <res타입> <개수> |
| 메모리   | 값 포함          | 공간만 예약            |
| 사용 목적 | 상수, 초기화 변수    | 버퍼, 입력공간, 임시배열    |

#### [4-6] 텍스트(Text) 섹션

프로그램의 실제 실행 코드는 다음 영역에 작성된다.

```
section .text
```

※ section 과 .text 사이에는 반드시 공백이 필요하다

#### [4-6-1] 텍스트 섹션의 역할

.text 섹션은 다음을 포함한다.

- 실행 가능한 명령어
- 프로그램의 시작 지점
- 레이블(Label)
- 시스템 호출 코드

즉, CPU 가 실제로 실행하는 코드 영역이다.

#### [4-6-2] 명령어 작성 규칙

- 명령어는 한 줄에 하나씩 작성
- 각 명령어는
  - 하나의 유효한 명령어(opcode)
  - 적절한 피연산자(operand)

형식 예:

```
mov rax, 1
add rbx, rax
```

각 줄은 하나의 완전한 명령이어야 한다.

#### [4-6-3] 프로그램의 시작점 정의

텍스트 섹션에는 프로그램의 초기 진입점(entry point)을 정의해야 한다.

표준 시스템 링커를 사용하는 경우 반드시 다음 선언이 포함된다.

```
global _start
_start:
```

- global \_start
  - \_start 레이블을 외부에서 참조 가능하도록 설정
  - 링커가 프로그램의 시작 위치로 인식
- \_start:
  - 실제 실행이 시작되는 위치를 정의하는 레이블

즉, 운영체제가 프로그램을 실행하면 \_start 부터 실행이 시작된다.

#### [4-6-4] 프로그램 종료 처리

특별한 종료 레이블은 요구되지 않는다.

하지만 프로그램이 끝날 때는 반드시

- 운영체제에 종료를 알리고
- 사용한 메모리 및 자원을 반환해야 한다.

이를 위해 시스템 서비스(시스템 호출)를 사용한다.

종료 예시는 일반적으로 다음과 같은 형태를 가진다.

```
mov rax, 60      ; exit 시스템 호출 번호 (예: Linux x86-64)
mov rdi, 0       ; 종료 코드
syscall
```

(구체적인 시스템 호출 방식은 다음 절의 예제에서 다뤄진다.)

#### [4-6-5] 전체 구조 예시

어셈블리 프로그램의 기본 구조는 다음과 같다.

```
section .data
    ; 초기화된 데이터

section .bss
    ; 초기화되지 않은 데이터

section .text
    global _start

_start:
    ; 실행 코드
```

#### [4-7] 예제 프로그램

이 예제는 어셈블리 언어 프로그램의 기본 형식(layout)과 구성 구조를 보여주기 위한 매우 단순한 프로그램이다.

```
; *****
; 기본적인 프로그램 형식과 구조를 보여주는 간단한 예제
; Ed Jorgensen
; 2014 년 7 월 18 일;
; *****

; =====
; Data Section
; =====

section .data

; -----
; Constants
EXIT_SUCCESS equ 0      ; successful operation
SYS_exit      equ 60     ; system call number for exit

; -----
; Byte (8-bit) variables
bVar1    db 17
bVar2    db 9
bResult  db 0
```

```

; -----
; Word (16-bit) variables
wVar1    dw 17000
wVar2    dw 9000
wResult  dw 0

; -----
; Double-word (32-bit) variables
dVar1    dd 17000000
dVar2    dd 9000000
dResult  dd 0

; -----
; Quad-word (64-bit) variables
qVar1    dq 170000000
qVar2    dq 90000000
qResult  dq 0

; =====
; Code Section
; =====
section .text
global _start

_start:

; -----
; Byte example
; bResult = bVar1 + bVar2
mov al, byte [bVar1]
add al, byte [bVar2]
mov byte [bResult], al

; -----
; Word example
; wResult = wVar1 + wVar2
mov ax, word [wVar1]
add ax, word [wVar2]
mov word [wResult], ax

```

```

; -----
; Double-word example
; dResult = dVar1 + dVar2
mov eax, dword [dVar1]
add eax, dword [dVar2]
mov dword [dResult], eax

; -----
; Quad-word example
; qResult = qVar1 + qVar2
mov rax, qword [qVar1]
add rax, qword [qVar2]
mov qword [qResult], rax

; -----
; Program termination
mov rax, SYS_exit
mov rdi, EXIT_SUCCESS
syscall

```

#### [4-7-1] 전체 구조 개요

프로그램은 크게 두 영역으로 구성된다.

```

section .data    ; 데이터 영역
section .text    ; 코드 영역

```

#### [4-7-2] Data Section(section .data)

이 영역에서는

- 상수
- 초기화된 변수
- 계산 결과 저장 변수

가 선언된다.



### [1] 상수 정의

```
EXIT_SUCCESS equ 0
SYS_exit      equ 60
```

의미

| 상수           | 의미                          |
|--------------|-----------------------------|
| EXIT_SUCCESS | 정상 종료 코드                    |
| SYS_exit     | Linux x86-64 exit 시스템 호출 번호 |

- equ 는 값 치환용 상수
- 메모리 공간을 차지하지 않음

### [2] 8 비트(Byte) 변수

```
bVar1    db 17
bVar2    db 9
bResult  db 0
```

- db: 8 비트 변수
- 결과 저장용 변수도 미리 초기화

### [3] 16 비트(Word) 변수

```
wVar1    dw 17000
wVar2    dw 9000
wResult  dw 0
```

- dw: 16 비트 변수

### [4] 32 비트(Double-word) 변수

```
dVar1    dd 17000000
dVar2    dd 90000000
dResult  dd 0
```

- dd: 32 비트 변수

### [5] 64 비트(Quad-word) 변수

```
qVar1    dq 1700000000
qVar2    dq 9000000000
qResult  dq 0
```

- dq: 64 비트 변수

#### [4-7-3] Code Section(section .text)

```
section .text
global _start
```

- \_start 가 프로그램의 진입점
- 실제 실행 코드 작성

#### [4-7-4] 연산 예제 분석

이 프로그램은 각 데이터 크기별 덧셈 연산을 수행한다. 하나의 연산만 설명한다.

##### [1] Byte 예제(8 비트)

```
mov al, byte [bVar1]
add al, byte [bVar2]
mov byte [bResult], al
```

실행 과정

1. bVar1 → AL 에 로드
2. bVar2 → AL 에 더함
3. 결과 → bResult 에 저장

사용 레지스터: AL(8 비트)

#### [4-7-5] 프로그램 종료

```
mov rax, SYS_exit
mov rdi, EXIT_SUCCESS
syscall
```

동작과정

1. rax ← 시스템 호출 번호(60)
2. rdi ← 종료 코드(0)
3. syscall 실행 → 운영체제에 종료 요청

#### [4-7-6] 대괄호([])의 의미

어셈블리에서 []는 메모리 접근을 의미한다.

##### [1] 심볼 단독 사용

```
bVar1 ; 변수의 주소(address)
```

##### [2] 대괄호 사용

```
[bVar1] ; 해당 주소에 저장된 실제 값(value)
```

### [3] 레지스터 예

| 표현    | 의미                  |
|-------|---------------------|
| rax   | 레지스터 자체             |
| [rax] | rax가 가리키는 주소의 메모리 값 |

이것은 C 언어의 포인터 간접 참조와 동일한 개념이다.

### [4-7-7] 왜 byte, word 같은 크기 지정이 필요한가?

메모리는 타입 정보를 갖지 않는다.

따라서 어셈블리는 다음을 알아야 한다.

- 몇 바이트를 읽을 것인가?
- 몇 바이트를 저장할 것인가?

그래서 다음과 같은 크기 지정자를 사용한다.

| 지정자   | 크기   |
|-------|------|
| byte  | 8비트  |
| word  | 16비트 |
| dword | 32비트 |
| qword | 64비트 |

예:

```
mov al, byte [bVar1]
```

- 1 바이트만 읽어라

## Chapter5. 툴 체인(Tool Chain)

툴 체인이란

프로그램을 실행 가능한 형태로 만들기 위해 사용하는  
도구들의 집합을 툴 체인(tool chain)이라고 한다.

본 교재에서 사용하는 툴 체인은 다음 4 가지 도구로 구성된다.

| 도구        | 역할                        |
|-----------|---------------------------|
| Assembler | 소스 코드를 기계어로 변환            |
| Linker    | 여러 오브젝트 파일을 결합하여 실행 파일 생성 |
| Loader    | 실행 파일을 메모리에 적재            |
| Debugger  | 프로그램 실행 과정 분석 및 오류 추적     |

이 교재에서는

- x86-64 완전 지원
- 서로 호환되는 오픈소스 도구

### [5-1] Assemble → Link → Load 개요

어셈블 → 링크 → 로드 과정은

“사람이 작성한 코드가 실제 실행 가능한 프로그램이 되는 전체 과정”이다.

#### [5-1-1] 전체 흐름 요약

소스 코드

↓ (Assembler)

오브젝트 파일

↓ (Linker)

실행 파일

↓ (Loader)

메모리 적재 후 실행

## [1] Assemble 단계

입력:

- 어셈블리 언어 소스 파일(.asm)

출력:

- 오브젝트 파일(.o)
- 리스팅 파일(선택적)

수행 작업: 어셈블러는 다음을 수행한다.

- 어셈블리 명령어 → 기계어(machine code)변환
- 심볼 테이블 생성
- 오브젝트 파일 생성

중요한 점:

- 이 단계의 결과물은 아직 실행할 수 없다.

왜냐하면:

- 외부 심볼이 해결되지 않았고 주소가 확정되지 않았기 때문

## [2] Link 단계

링커는 여러 오브젝트 파일을 결합한다.

입력:

- 하나 이상의 오브젝트 파일
- 라이브러리(필요 시)
- 공유 오브젝트 파일(.so 등)

수행 작업:

(1) 심볼 해석(Symbol Resolution)

- 함수
- 전역 변수
- 외부 참조

같은 이름의 심볼을 실제 주소에 연결

(2) 주소 재배치(Relocation)

- 각 코드와 데이터의 실제 메모리 위치 결정
- 상대 주소 → 절대 주소 확정

(3) 실행 파일 생성

- 최종적으로 실행 가능한 파일 생성
- 이 단계가 끝나야 실제 실행 가능

### [3] Load 단계

로드 단계는 운영체제가 수행한다.

수행 과정:

1. 실행 파일을 RAM 에 적재
2. 코드와 데이터 섹션을 메모리에 배치
3. 스택, 힙 등 런타임 환경 설정
4. 프로그램의 entry point 로 제어 이동

```
global _start
_start:
```

이 지점으로 CPU 가 점프하면서 실행 시작

### [5-1-2] 단계별 책임 비교

| 단계       | 담당   | 결과             |
|----------|------|----------------|
| Assemble | 어셈블러 | 오브젝트 파일        |
| Link     | 링커   | 실행 파일          |
| Load     | 운영체제 | 메모리 적재 및 실행 시작 |

### [5-2] Assembler

어셈블러는 어셈블리 언어 소스 파일을 CPU 가 이해하는 기계어(binary)로 변환하는 프로그램이다.

입력

- 사람이 읽을 수 있는 어셈블리 소스 파일(.asm)

출력

- 오브젝트 파일(.o)

어셈블러의 역할

어셈블러는 단순한 번역기가 아니다. 다음 작업을 수행한다.

- 주석 제거
- 변수 이름 처리
- 레이블(label)처리
- 기호(symbol)를 실제 주소로 변환
- 기계어 코드 생성

즉, 기호적 표현을 수치적 표현으로 변환하는 역할을 한다.

## [5-2-1] Assemble 명령어

### 기본 명령어

```
yasm -g dwarf2 -f elf64 example.asm -l example.lst
```

### 옵션 설명

| 옵션             | 의미                    |
|----------------|-----------------------|
| -g dwarf2      | 디버깅 정보 포함             |
| -f elf64       | 출력 형식: 64비트 Linux ELF |
| example.asm    | 입력 파일                 |
| -l example.lst | 리스트 파일 생성             |

※ -l 은 숫자 1 이 아닌 소문자 L

### 오류 발생 시

- 오류가 있으면 오브젝트 파일이 생성되지 않음
- 반드시 오류 수정 후 링크 단계 진행

## [5-2-2] 리스트 파일(List File)

리스트 파일은 선택적으로 생성된다.

### 포함 정보

한 줄에 다음이 표시된다.

- 소스 코드의 라인 번호
- 상대 주소
- 기계어 코드(16 진수)
- 원본 어셈블리 코드

디버깅 시 매우 유용하다.

### [1] 데이터 섹션 예시 설명

다음 데이터 섹션에 대한 리스트 파일의 일부이다.

```
36 00000009 40660301 dVar1 dd 17000000
37 0000000D 40548900 dVar2 dd 90000000
38 00000011 00000000 dResult dd 0
```

### [1-1] 구성 요소 분석

| 항목                | 의미             |
|-------------------|----------------|
| 36                | 소스 코드 라인 번호    |
| 00000009          | 데이터 영역 내 상대 주소 |
| 40660301          | 메모리에 저장된 실제 값  |
| dVar1 dd 17000000 | 원본 소스          |

### [1-2] 주소 증가 이유

dVar1 은 doubl-word(4 바이트)

→ 0x00000009 ~ 0x0000000C 사용

→ 다음 변수 시작 주소 = 0x0000000D

### [1-3] 리틀 엔디언(Little Endian)

17000000(10 진수) → 16 진수: 0x01036640

메모리 저장 순서:

40 66 03 01

가장 낮은 바이트가 가장 낮은 주소에 저장된다.

그래서 리스트 파일에서 값이 “거꾸로”보인다.

### [2] 텍스트 섹션 예시 설명

다음은 코드(text) 섹션에 대한 리스트 파일 일부이다.

95 last:

96 0000005A 48C7C03C000000 mov rax, SYS\_exit

97 00000061 48C7C300000000 mov rdi, EXIT\_SUCCESS

98 00000068 0F05 syscall

### [2-1] 분석

| 항목                | 의미        |
|-------------------|-----------|
| 95                | 소스 라인 번호  |
| 0000005A          | 명령어 상대 주소 |
| 48C7C03C000000    | 기계어 코드    |
| mov rax, SYS_exit | 원본 코드     |

### [2-2] 레이블의 특징

last:

- 주소를 가리키는 이름
- 실행 명령어 아님
- 기계어 코드 없음

즉, last:는 “0000005A 라는 주소에 붙인 이름”이며, 기계어를 생성하지 않는 주소 표시자이다.

| variable name | value | address    |
|---------------|-------|------------|
|               | 00    | 0x00000010 |
|               | 89    | 0x0000000F |
|               | 54    | 0x0000000E |
| dVar2 →       | 40    | 0x0000000D |
|               | 01    | 0x0000000C |
|               | 03    | 0x0000000B |
|               | 66    | 0x0000000A |
| dVar1 →       | 40    | 0x00000009 |

Illustration 7: Little-Endian, Multiple Variable Data Layout



### [5-2-3] Two-Pass Assembler(2 패스 어셈블러)

어셈블러는 프로그래머가 작성한 어셈블리 언어 소스 파일을 읽어, CPU가 이해할 수 있는 기계어(0과 1의 이진 코드)로 변환하는 프로그램이다.

어셈블리 명령어와 기계어 사이에는 거의 일대일 대응 관계가 존재하므로, 생성된 기계어는 다시 어셈블리 해자로 역변환(disassemble)할 수 있다.

그러나 이 과정에서는 주석, 변수 이름, 레이블 이름과 같은 기호적 정보가 제거되기 때문에, 역변환된 코드는 사람이 이해하기 어렵다.

어셈블러는 소스 코드를 위에서 아래로 읽으면서 각 명령어를 대응되는 기계어로 변환한다. 점프나 분기와 같은 제어 흐름 변경 명령어가 없다면 이러한 단순한 처리 방식으로든 문제가 발생하지 않는다.

그러나 실제 프로그램에는 조건 분기나 무조건 점프와 같은 명령어가 포함되며, 이 경우 주소 계산과 관련된 문제가 발생한다.

예를 들어, 다음과 같은 코드가 있다고 가정하자.

```
mov rax, 0
jmp skipRest
...
...
skipRest:
```

여기서 `jmp skipRest`는 아직 정의되지 않은 레이블을 참조한다.

이처럼 나중에 레이블을 먼저 참조하는 경우를 전방 참조(forward reference)라고 한다.

어셈블러가 소스를 한 번만 읽는다면, 해당 레이블이 실제로 어디에 위치하는지 알 수 없으므로 정확히 기계어 코드를 생성할 수 없다.

점프 명령어는 실제 주소 또는 상대 오프셋 값을 필요로 하기 때문에, 레이블의 위치가 한정되지 않은 상태에서는 완전한 기계어를 만들 수 없다.

이 문제를 해결하기 위해 대부분의 어셈블러는 Two-Pass Assembler(2 패스 어셈블러) 구조를 사용한다.

즉, 소스 파일을 두 번 읽는 방식이다.

#### [1] First Pass(첫 번째 패스)

첫 번째 패스의 목적은 전체 소스 코드의 구조를 파악하고, 필요한 주소 정보를 수집하는 것이다.

이 단계에서 일반적으로 다음과 같은 작업이 수행된다.

- 심볼 테이블(symbol table) 생성
- 각 명령어 및 데이터에 대한 주소 할당
- 매크로(macro) 확장
- 상수 표현식(constant expression) 계산

심볼 테이블은 첫 번째 패스의 가장 중요한 결과물이다. 심볼 테이블에는 변수 이름과 레이블 이름, 그리고 이들에 대응되는 메모리 주소와 기록된다. 이를 통해 이후 점프 명령어나 변수 참조가 정확한 주소로 연결될 수 있다.

어셈블러는 내부적으로 위치 카운터(location counter)를 사용하여 각 명령어나 데이터가 차지하는 바이트 수를 계산하고, 이를 기반으로 다음 주소를 결정한다. 이 과정을 통해 소스 코드의 모든 구성 요소에 대한 주소가 확장된다.

또한, 상수 표현식은 실행 시간이 아니라 어셈블 시점에 계산 된다. 예를 들어,

```
mov rax, BUFF + 5
```

와 같은 경우 `BUFF + 5`는 첫 번째 패스에서 계산되어 하나의 수치 값으로 치환된다. 첫 번째 패스가 끝나면, 프로그램의 전체의 구조와 각 심볼의 주소가 모두 결정된다.

### [2] Second Pass(두 번째 패스)

두 번째 패스에서는 첫 번째 패스에서 생성된 심볼 테이블을 참조하여 최종 기계어 코드를 생성한다. 이 단계에서는 다음 작업이 수행된다.

- 최종 기계어 코드 생성
- 리스트 파일(list file) 생성 (선택 사항)
- 오브젝트 파일(object file) 생성

이제 모든 레이블과 변수의 주소가 확정되어 있으므로, 전방 참조 문제 없이 정확한 기계어를 생성할 수 있다.

예를 들어, `jmp skipRest`와 같은 명령어는 심볼 테이블에 기록된 주소를 참조하여 올바른 점프 오프셋으로 변환된다.

어셈블러의 구현 방식에 따라 일부 코드 생성이 첫 번째 패스에서 이루어질 수 있으나, 최종적인 기계어 생성은 반드시 두 번째 패스에서 완료된다. 오류가 없다면 이 단계에서 오브젝트 파일이 생성된다.

### [3] Two-Pass 구조의 의의

2 패스 구조는 단순히 전방 참조 문제를 해결하기 위한 기술적 장치가 아니다.

이는 프로그램의 전체의 구조를 먼저 분석하고, 그 결과를 정확한 기계어를 생성하기 위한 체계적인 설계 방식이다.

정리하면 다음과 같다.

- 첫 번째 패스: 구조 분석 및 주소 확장
- 두 번째 패스: 기계어 생성

이러한 구조를 통해 어셈블러는 기호 기반의 사람이 읽기 쉬운 코드를, 하드웨어가 실행할 수 있는 정확한 수치 기반 기계어로 안정적으로 변환할 수 있다.

## [5-2-4] 어셈블러 지시자(Assembler Directives)

어셈블러 지시자(assembly directive)는 CPU가 실행하는 명령어가 아니라, 어셈블러에게 특정 작업을 지시하기 위한 명령이다.

즉, 프로그램 실행 단계에서 사용되는 명령이 아니라, 어셈블 과정에서만 의미를 가지는 문법 요소이다.

어셈블러 지시자는 데이터의 배치, 섹션 정의, 상수 선언, 심볼 공개 등과 같은 작업을 수행한다. 이러한 지시자들은 기계어로 변환되지 않으며, 실행 파일에 명령어 형태로 포함되지 않는다.

### [1] 어셈블러 지시자와 명령어의 차이

어셈블러 코드에는 크게 두 가지 요소가 존재한다.

1. CPU 명령어(instruction)
2. 어셈블러 지시자(directive)

두 요소의 차이는 다음과 같다.

| 구분     | CPU 명령어   | 어셈블러 지시자   |
|--------|-----------|------------|
| 실행 주체  | CPU       | 어셈블러       |
| 실행 시점  | 프로그램 실행 시 | 어셈블 시      |
| 기계어 생성 | 생성됨       | 생성되지 않음    |
| 목적     | 연산 수행     | 구조 및 배치 지정 |

예를 들어,

```
mov rax, 1
```

은 CPU가 실제로 실행하는 명령어이며 기계어 코드가 생성된다.

반면,

```
section .data
```

는 어셈블러에게 데이터 영역을 정의하라고 지시하는 문장으로, 기계어로 변환되지 않는다.

### [2] 주요 어셈블러 지시자의 종류

#### [2-1] 섹션 정의 지시자

```
section .data
section .text
section .bss
```

- .data: 초기화된 데이터 영역
- .text: 실행 코드 영역
- .bss: 초기화되지 않은 데이터 영역

이 지시자는 이후에 선언되는 코드나 데이터가 어느 영역에 배치될지를 지정한다.

## [2-2] 데이터 선언 지시자

```
db    ; 1 바이트  
dw    ; 2 바이트  
dd    ; 4 바이트  
dq    ; 8 바이트
```

예:

```
value dd 10
```

이 지시자는 메모리 공간을 할당하고 초기값을 저장한다.  
CPU 명령이 아니라, 어셈블러가 데이터 영역을 구성하기 위한 지시이다.

## [2-3] 상수 정의 지시자

```
equ
```

예:

```
SYS_exit equ 60
```

equ 는 메모리를 할당하지 않으며, 단순히 기호를 특정 값으로 치환한다.  
어셈블 과정에서 값이 직접 대체된다.

## [2-4] 심볼 공개 지시자

```
global _start
```

이 지시자는 특정 심볼을 외부에서 참조할 수 있도록 설정한다.  
링커가 해당 심볼을 인식할 수 있게 만드는 역할을 한다.

## [3] 어셈블러 지시자의 처리 시점

어셈블러 지시자는 프로그램 실행 시점이 아니라, 어셈블 시점에 처리된다.  
처리 과정은 다음과 같다.

- 일부 지시자는 First Pass 에서 처리됨(예: 심볼 정의)
- 일부 지시자는 메모리 배치 과정에서 처리됨(예: 데이터 선언)
- 기계어로 변환되지는 않음

따라서 실행 파일에는 지시자 자체가 존재하지 않는다.

## [4] 어셈블러 지시자의 역할

어셈블러 지시자는 프로그램의 구조와 배치 방식을 정의하는 메타 정보이다.  
정리하면 다음과 같다.

- 코드와 데이터 영역을 구분한다.
- 메모리 공간을 할당한다.
- 상수를 정의한다.
- 링커와의 연결을 준비한다.

즉, 어셈블러 지시자는 프로그램의 실행 동작을 정의하는 것이 아니라, 프로그램이 메모리에 어떻게 구성될지를 정의한다.

#### [5] 정리

어셈블러 지시자는 CPU 명령어와 달리 실행되지 않는다.

이들은 어셈블러가 소스 코드를 올바르게 해석하고, 메모리를 배치하며, 오브젝트 파일을 생성하도록 돕는 구조적 지시문이다.

따라서 어셈블리 프로그램은

- |                                                                                                     |
|-----------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>- 실행 명령어(instruction)</li><li>- 어셈블러 지시자(directive)</li></ul> |
|-----------------------------------------------------------------------------------------------------|

의 조합으로 구성되며, 이 둘의 역할을 명확히 구분하는 것이 중요하다.

### [5-3] Linker

링커(linker)는 하나 이상의 오브젝트 파일(object file)을 결합하여 하나의 실행 파일(executable file)을 생성하는 프로그래이다. 링커는 linkage editor 라고도 불린다.

어셈블러가 생성한 오브젝트 파일은 아직 완전한 실행 파일이 아니다. 오브젝트 파일에는 기계어 코드와 데이터가 포함되어 있지만, 외부 심볼에 대한 참조가 해결되지 않았으며, 최종 메모리에 배치도 확정되지 않은 상태이다.

링커는 이러한 미완성 상태의 파일들을 결합하여, 실제로 실행 가능한 형태로 완성한다.

필요한 경우 링커는 사용자 라이브러리 또는 시스템 라이브러리에 포함된 코드도 함께 포함시킨다.

이 교재에서는 GNU 계열의 링커인 GNU ld(gold linker)를 사용한다.

기본 링크 명령은 다음과 같다.

```
ld -g -o example example.o
```

옵션의 의미는 다음과 같다.

- o: 출력 파일 이름 지정(소문자 o, 숫자 0 과 구분)
- g: 디버깅 정보 포함
- example.o: 입력 오브젝트 파일

-o 옵션을 생략하면 기본적으로 a.out 이라는 이름의 실행 파일이 생성된다.

#### [5-3-1] 여러 파일을 링크하는 경우

실제 프로그램 개발에서는 하나의 거대한 파일로 모든 기능을 구현하기보다는, 기능을 여러 개의 작은 단위로 분할하여 각각을 독립적으로 작성한다.

이러한 단위는 서로 다른 소스 파일로 구성될 수 있으며, 각 파일은 개별적으로 어셈블되어 오브젝트 파일로 생성된다.

예를 들어, 다음과 같은 오브젝트 파일이 있다고 가정하자.

- main.o
- funcs.o

```
ld -g -o example main.o funcs.o
```

링커는 두 오브젝트 파일의 코드와 데이터를 하나로 병합하여 단일 실행 파일을 생성한다. 다른 파일에 정의된 함수를 사용하는 경우, 해당 심볼은 반드시 extern 으로 선언해야 한다. 이는 어셈블러와 링커에게 “이 심볼은 현재 파일이 아니라 외부 파일에 정의되어 있다”는 사실을 알리기 위한 역할을 한다.

변수 역시 extern 으로 접근할 수 있으나, 실제 프로그램 설계에서는 함수 인자를 통해 데이터를 전달하는 방식이 더 일반적이다.

### [5-3-2] 링킹 과정의 핵심 개념

링킹(linking)이란 여러 개의 독립적으로 생성된 프로그램 조각을 하나의 완전한 실행 단위로 결합하는 과정이다. 이 과정에서 링커는 다음과 같은 작업을 수행한다.

1. 여러 오브젝트 파일의 기계어 코드 및 데이터 병합
2. 외부 심볼(external symbol) 해결
3. 재배치(relocation) 수행

#### [1] 코드 및 데이터 병합

각 오브젝트 파일은 .text, .data, .bss 와 같은 섹션을 가진다. 링커는 동일한 종류의 섹션을 모아 하나의 큰 섹션으로 병합한다.

예를 들어,

- main.o 의 .text
- funcs.o 의 .text

는 최종 실행 파일의 하나의 .text 영역으로 결합된다.

#### [2] 외부 심볼 해결(Symbol Resolution)

각 오브젝트 파일은 독립적으로 어셈블되므로, 다른 파일에 정의된 함수나 변수의 실제 주소를 알 수 없다.

예를 들어, main.o 가 funcs.o 에 정의된 func1 을 호출한다고 가정하자. main.o 를 어셈블할 당시에는 func1 의 실제 위치를 알 수 없으므로, 이는 외부 참조(external reference)로 기록된다.

링커는 다음 작업을 수행한다.

- funcs.o 에서 func1 의 실제 정의 위치 확인
- 해당 심볼의 최종 주소 계산
- main.o 의 호출 지점을 그 주소로 연결

이 과정을 통해 외부 심볼이 해결된다.

#### [3] 재배치(Relocation)

각 오브젝트 파일은 자신의 최종적으로 메모리의 어디에 배치될지 알지 못한다. 따라서 내부 주소는 임시 기준(보통 섹션 시작 주소 0x0)을 기준으로 계산된다.

예를 들어, funcs.o 의 .text 섹션이 내부적으로 0x0 부터 시작한다고 가정한다.

그러나 최종 실행 파일에서 해당 섹션이 0x400 에 배치된다면, 모든 주소는 0x400 만큼 조정되어야 한다.

이 주소 수정 작업을 재배치라고 한다.

링커는 다음 작업을 수행한다.

- |                                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>- 각 심볼의 최종 절대 주소 계산</li><li>- 코드 및 데이터 내 참조 주소 수정</li><li>- 모든 재배치 항목 처리</li></ul> |
|--------------------------------------------------------------------------------------------------------------------------|

이 과정을 통해 함수 호출과 데이터 참조가 정확하게 동작하도록 보장된다.

### [5-3-3] 동적 링크(Dynamic Linking)

리눅스 운영체제는 동적 링크(dynamic linking)를 지원한다. 동적 링크는 일부 심볼의 실제 주소 결정을 프로그램 실행 시점(run-time)까지 미루는 방식이다.

이 방식에서는 라이브러리 코드가 실행 파일 내부에 직접 포함되지 않는다. 대신 실행 시점에 로더(loader)에 의해 필요한 라이브러리가 메모리에 적재되고, 그때 심볼이 해석된다.

리눅스/유닉스 계열에서 동적 라이브러리는 일반적으로 .so(shared object) 확장자를 가진다. 윈도우 환경에서는 .dll(dynamic linked library) 확장자를 사용한다.

#### [1] 동적 링크의 장점

##### 1. 라이브러리 코드 중복 제거

여러 프로그램이 하나의 라이브러리를 공유할 수 있다.

디스크 공간과 메모리 사용량을 절약할 수 있다.

##### 2. 라이브러리 업데이트 반영

라이브러리 버그가 수정되면, 재링크 없이 다음 실행부터 수정된 코드가 사용된다.

#### [2] 동적 링크의 단점

##### 1. 호환성 문제

라이브러리 인터페이스가 변경되면 기본 프로그램이 정상적으로 동작하지 않을 수 있다.

##### 2. 구성 고정의 어려움

특정 버전의 라이브러리와 함께 검증된 프로그램을 그대로 유지해야 하는 경우, 실행 시점에 라이브러리가 변경될 수 있어 문제가 될 수 있다.

#### [3] 정리

링커는 오브젝트 파일을 결합하여 실행 파일을 생성하는 프로그램이다. 이 과정에서 코드 병합, 외부 심볼 해결, 재배치 작업을 수행한다.



정적 링크는 모든 코드가 실행 파일 내부에 포함되는 방식이며, 동적 링크는 실행 시점에 라이브러리를 연결하는 방식이다. 각각은 성능, 공간 효율성, 유지보수 측면에서 장단점을 가진다.

#### [5-4] 어셈블 / 링크 스크립트

어셈블리 프로그램을 작성하다 보면, 어셈블러(yasm 또는 nasm)와 링커(ld)를 반복해서 실행해야 하는 경우가 매우 많다.

프로그램 하나를 수정할 때마다 다음과 같은 명령을 다시 입력해야 한다 .

```
yasm ...  
ld ...
```

이 과정을 매번 수동으로 입력하는 것은 번거롭고, 옵션을 잘못 입력하는 실수도 발생할 수 있다.

이러한 반복 작업을 줄이기 위해 여러 명령을 하나의 파일에 작성해 두고 한 번에 실행할 수 있다. 이러한 파일을 스크립트 파일(script file)이라고 한다.

스크립트 파일을 실행하면, 파일 내부에 작성된 명령들이 위에서 아래로 순서대로 자동 실행된다.

스크립트 사용은 필수는 아니지만, 개발 효율을 높이고 작업을 단순화하는 데 매우 유용하다.

#### [5-4-1] Bash 기반 어셈블 / 링크 스크립트 예제

다음은 간단한 Bash 기반 어셈블/링크 스크립트 예제이다.

##### [1] Bash 기반 어셈블 / 링크 스크립트 예제

다음은 간단한 bash 기반 어셈블/링크 스크립트 예제이다.

```
#!/bin/bash  
# Simple assemble/link script.  
  
if [ -z $1 ]; then  
    echo "Usage: ./asm64 <asmMainFile> (no extension)"  
    exit  
fi  
  
# Verify no extensions were entered  
if [ ! -e "$1.asm" ]; then  
    echo "Error, $1.asm not found."  
    echo "Note, do not enter file extensions."
```

```

        exit
fi

# Compile, assemble, and link.
yasm -Worphan-labels -g dwarf2 -f elf64 $1.asm -l $1.lst
ld -g -o $1 $1.o

```

## [5-4-2] 스크립트 구조 분석

### [1] 첫 줄(Shebang)

```
#!/bin/bash
```

이 줄은 해당 파일이 Bash 셸에 의해 실행되어야 함을 의미한다.

### [2] 명령줄 인자 검사

```
if [ -z $1 ]; then
```

- \$1: 첫 번째 명령줄 인자
- -z: 문자열이 비어 있는지 검사

파일 이름이 전달되지 않은 경우, 사용법을 출력하고 종료한다.

```
echo "Usage: ./asm64 <asmMainFile> (no extension)"
exit
```

### [3] 파일 존재 여부 확인

```
if [ ! -e "$1.asm" ]; then
```

- \$1.asm 파일이 존재하는지 검사
- ! -e: 파일이 존재하지 않으면 참

이 스크립트는 사용자가 확장자를 입력하지 않은 것을 전제로 한다.

예를 들어, example.asm 파일이 있으면 다음과 같이 입력해야 한다.

```
./asm64 examle
```

### [4] 어셈블 및 링크 수행

```
yasm -Worphan-labels -g dwarf2 -f elf64 $1.asm -l $1.lst
ld -g -o $1 $1.o
```

이 부분이 스크립트의 핵심이다.

#### [4-1] 어셈블 단계

```
yasm -Worphan-labels -g dwarf2 -f elf64 $1.asm -l $1.lst
```

옵션 설명:

- -Worphan-labels: 고립된 레이블 경고
- -g dwarf2: DWARF2 형식 디버깅 정보 생성
- -f elf64: 64 비트 ELF 형식으로 출력
- \$1.asm: 입력 파일
- -l \$1.lst: 리스트 파일 생성

이 명령은 어셈블리 소스를 오브젝트 파일(\$1.o)로 변환한다.

#### [4-2] 링크 단계

```
ld -g -o $1 $1.o
```

- -g: 디버깅 정보 포함
- -o \$1: 실행 파일 이름 지정
- %1.o: 입력 오브젝트 파일

이 명령은 오브젝트 파일을 실행 파일로 변환한다.

링커로는 GNU 계열의 GNU ld 를 사용한다.

#### [5-4-3] 스크립트 사용 예

예를 들어, 소스 파일이 example.asm 이라면 다음과 같이 실행한다.

```
./asm64 example
```

.asm 확장자는 입력하지 않는다.

스크립트 내부에서 자동으로 확장자를 붙이기 때문이다.

실행 결과 다음 파일이 자동 생성된다.

- example.lst (리스트 파일)
- example.o (오브젝트 파일)
- example (실행 파일)

즉, 어셈블과 링크 과정을 한 번의 명령으로 처리한다.

#### [5-4-4] 빌드 자동화의 확장

본 절의 스크립트는 도구 체인(Assembler → Linker)의 흐름을 이해하기 위한 최소한의 자동화 방법이다.

그러나 실제 프로젝트에서는 다음과 같은 요구가 발생한다.

- 여러 소스 파일 관리
- 파일 간 의존성 추적
- 변경된 파일만 다시 빌드
- 라이브러리 자동 연결

이러한 작업을 효율적으로 수행하기 위해 일반적으로 make 와 같은 빌드 시스템을 사용한다. 본 프로젝트에서는 NASM 어셈블러와 GNU ld 링커를 사용하며, 빌드 자동화 도구로 make 를 활용한다.

#### [5-5] Loader(로더)

로더(Loader)는 운영체제의 구성 요소로, 보조 저장장치(디스크)에 있는 실행 파일을 주 기억장치(Main Memory)에 적재하는 역할을 한다.

즉, 로더는 디스크에 존재하는 실행 파일을 실행 중인 프로세스로 변환하는 역할을 수행한다.

##### [5-5-1] 로더의 주요 기능

로더는 개략적으로 다음 작업을 수행한다.

1. 올바른 형식의 실행 파일(executable file) 확인
2. 해당 파일을 디스크에서 읽기
3. 새로운 프로세스(process) 생성
4. 프로그램의 코드(code)와 데이터(data)를 메모리에 적재
5. 프로그램을 실행 가능한 형태(ready state)로 전환

이 과정을 통해 단순한 “파일”이 운영체제에서 관리되는 “프로세스”가 된다.

##### [5-5-2] 스케줄러와의 관계

프로그램이 메모리에 로드되었다고 해서 즉시 실행되는 것은 아니다.

어떤 프로세스를 언제 실행할지는 운영체제의 스케줄러(Scheduler)가 결정한다.

역할은 다음과 같이 구분된다.

- 로더 → 프로세스를 준비 상태(Ready)로 만든다.
- 스케줄러 → CPU 를 언제 할당할지 결정한다.

즉, 로더와 스케줄러는 서로 다른 책임을 가진다.

로더는 “적재(load)”를 담당하고, 스케줄러는 “실행 지점 결정”을 담당한다.

### [5-5-3] 로더는 언제 호출되는가?

사용자는 로더를 직접 호출하지 않는다.

예를 들어, 다음과 같은 명령을 입력하면:

```
./example
```

운영체제가 내부적으로 다음을 수행한다.

1. 현재 디렉터리에서 example 실행 파일을 검색
2. 해당 파일을 메모리에 로드
3. 새로운 프로세스 생성
4. 실행 준비 상태로 전환

여기서 example 파일을 앞선 단계(assembly → link)를 통해 생성된 실행 파일이다.  
리눅스 환경에서는 일반적으로 ELF 형식의 실행 파일이 사용된다.

### [5-5-4] 출력이 없는 프로그램의 경우

이전 예제 프로그램은 내부적으로 계산만 수행하고 화면에 출력하지 않는다.

따라서 실행해도 콘솔에는 아무것도 표시되지 않는다.

이런 경우 다음과 같은 방법으로 결과를 확인해야 한다.

- 레지스터 값 확인
- 메모리 변수 값 확인
- 실행 흐름 추적

이를 위해 사용하는 도구가 바로 디버거(Debugger)이다.

### [5-6] Debugger(디버거)

디버거(debugger)는 프로그램의 실행을 관찰하고 제어하기 위한 도구이다.

디버거를 사용하면 다음 작업이 가능하다.

- 한 줄씩 실행(step-by-step)
- 특정 위치에서 실행 중단(breakpoint)
- 레지스터 값 확인
- 메모리 내용 확인
- 실행 흐름 추적

즉, 디버거는 “프로그램이 실제로 어떻게 실행되는지 관찰하고 통제하는 도구”이다.

### [5-6-1] 왜 디버거가 필요한가?

출력이 없는 프로그램의 경우, 계산 결과를 눈으로 확인할 방법이 없다.

예를 들어, 다음과 같은 값들을 확인해야 할 수 있다.

- bResult
- wResult
- dResult
- rax, eax, ax, al 레지스터 값

이때 디버거를 사용하면 실행 중인 프로그램의 내부 상태를 직접 확인할 수 있다.

### [5-6-2] 디버깅 정보를 포함해야 하는 이유

디버거를 제대로 사용하려면 실행 파일에 디버깅 정보(debug information)가 포함되어 있어야 한다.

이를 위해 다음 옵션을 사용한다.

#### [1] 어셈블 시

- g dwarf2
- -g dwarf2: DWARF2 형식의 디버깅 정보 생성

#### [2] 링크 시

- g

이 옵션이 없으면 다음 정보가 사라진다.

- 레이블(label)
- 소스 코드 위치 정보
- 심볼(symbol) 정보

그 결과, 디버깅이 매우 어려워진다.

리눅스 환경에서는 DWARF 디버깅 포맷과 함께 GNU ld 링커가 사용된다.

### [5-6-3] 사용되는 디버거

이 교재에서는 다음 도구를 사용한다.

#### 1. GNU Debugger(gdb)

- GNU 계열의 명령줄 기반 디버거
- 강력하고 표준적인 디버깅 도구
- 브레이크 포인트, 스텝 실행, 레지스터/메모리 확인 가능

## 2. DDD

- gdb 를 위한 그래픽 프론트엔드
- 시각적인 인터페이스 제공
- 내부적으로는 gdb 를 사용

DDD 는 UI 를 제공하지만, 실제 디버깅 기능의 핵심은 gdb 가 담당한다.

### [5-7] ELF(Executable and Linkable Format)

리눅스에서 실행 파일은 대부분 ELF(Executable and Linkable Format) 형식을 사용한다.  
ELF 다음과 같은 파일에 공통적으로 사용된다.

- 실행 파일(executable)
- 목적 파일(.o)
- 공유 라이브러리(.so)

로더는 ELF 구조를 해석하여 코드와 데이터를 메모리에 적재하고, 프로그램을 실행 가능한 상태로 만든다.

#### [5-7-1] ELF 파일의 전체 구조

ELF 파일은 크게 다음과 같은 구조로 이루어진다.

```
+-----+
| ELF Header          |
+-----+
| Program Header Table |
+-----+
| Section Header Table |
+-----+
| Sections (.text 등)  |
+-----+
```

각 영역의 역할은 다음과 같다.

#### [5-7-2] ELF Header

ELF 파일의 맨 앞에 위치한다.  
파일의 전체 구조에 대한 정보를 담고 있다.

주요 정보는 다음과 같다.

- 이 파일이 ELF 인지 여부(매직 넘버 0x7F 'ELF')
- 32 비트인지 64 비트인지
- 엔디안 방식(Little / Big)
- 실행 파일인지, 목적 파일인지
- Program Header 위치

- Section Header 위치

즉, ELF Header 는 “이 파일을 어떻게 해석해야 하는지 설명하는 안내서” 역할을 한다.

### [5-7-3] Program Header Table

로더가 실제로 사용하는 영역이다.

Program Header 는 “이 파일을 메모리에 어떻게 배치할 것인가”에 대한 정보가 들어 있다.  
각 엔트리는 다음과 같은 정보를 포함한다.

- 세그먼트 종류(Load 등)
- 파일 내 오프셋
- 메모리에 배치될 가상 주소
- 읽기/쓰기/실행 권한
- 메모리 크기

즉,

- 파일 관점 → Section 중심
- 실행 관점 → Segment 중심

로더는 Section 이 아니라 Segment 단위로 적재한다.

### [5-7-4] Section Header Table

컴파일러와 링커가 사용하는 영역이다.

로더는 보통 이 정보를 직접 사용하지 않는다.

Section 의 예시는 다음과 같다.

- .text → 실행 코드
- .data → 초기화된 전역 변수
- .bss → 초기화되지 않은 전역 변수
- .rodata → 읽기 전용 데이터
- .symtab → 심볼 테이블
- .debug\_\* → 디버깅 정보

Section 은 논리적 분류이고, Segment 는 실제 메모리 배치 단위이다.



### [5-7-5] 실행 시 메모리 배치 과정

사용자가 다음 명령을 입력하면

```
./example
```

운영체제는 다음을 수행한다.

1. ELF Header 확인
2. Program Header 분석
3. LOAD 세그먼트를 메모리에 복사
4. .bss 영역은 0 으로 초기화
5. 스택, 힙 영역 생성
6. 엔트리 포인트로 점프

이때 제어는 `_start` 로 이동하며, 그 후 `main()`이 호출된다.

### [5-7-6] Section vs Segment 정리

| 구분      | Section      | Segment       |
|---------|--------------|---------------|
| 사용 주체   | 컴파일러, 링커     | 로더            |
| 목적      | 논리적 구분       | 실제 메모리 적재     |
| 예시      | .text, .data | LOAD, DYNAMIC |
| 실행 시 사용 | 거의 사용 안 함    | 사용            |

### [5-7-7] 디버깅과 ELF 의 관계

-g 옵션을 사용하면

- .debug\_info
- .debug\_line
- .debug\_str

같은 디버그 섹션이 ELF 에 포함된다.

이 정보가 있어야 디버거(gdb)가

- 소스 코드 라인 매핑
- 변수 이름 인식
- 심볼 추적

을 할 수 있다.

옵션이 없으면 실행은 가능하지만, 디버깅은 매우 어렵다.

## Chapter 6. DDD 디버거(DDD Debugger)

디버거는 사용자가 프로그램의 실행을 제어하고, 변수와 기타 메모리(예: 스택 공간)를 검사하며, 프로그램 출력이 있다면 이를 표시할 수 있게 해준다.

오픈 소스인 GNU Data Display Debugger(DDD)는 GNU Debugger(GDB)의 시각적 프론트엔드이며, 널리 사용 가능하다. 원한다면 다른 디버거들도 쉽게 사용할 수 있다.

이 장에서는 기본적인 디버거 명령어만 다룬다.

DDD 디버거에는 여기서 다루지 않는 훨씬 더 많은 기능과 옵션들이 존재한다. 경험이 쌓인다.

DDD의 기능은 다양한 플러그인을 사용해 확장할 수 있다. 이 플러그인들은 필수는 아니며, 이 장에서는 다루지 않는다.

이 장은 도구로서의 GNU DDD 디버거 사용법을 다룬다.

프로그램을 어떻게 논리적으로 디버깅해야 하는지에 대한 과정 자체는 이 장에서 다루지 않는다.

(그러면 일단 여기는 생략)

## Chapter 7. 명령어 집합 개요(Instruction Set Overview)

이 장은 x86-64 명령어 집합 중 정수 연산에 필요한 핵심 부분집합을 다룬다.  
전체 명령어 체계를 모두 다루지는 않으며, 본 교재의 범위 내에서 실제 프로그램 작성에  
필요한 명령어들에 초점을 맞춘다.

제외되는 내용은 다음과 같다.

- SIMD 고급 벡터 명령어
- 시스템 전용 명령어
- 보호 모드/특권 모드 전용 명령어
- 고급 최적화 명령어

본 장의 명령어 분류는 다음 순서를 따른다.

1. 데이터 이동
2. 변환 명령어
3. 산술 명령어
4. 논리 명령어
5. 제어 명령어

함수 호출과 관련된 호출 규약(call convention), call, ret 등 Chapter12에서 별도로 다룬다.

### [7-1] 표기 규칙(Notational Conventions)

어셈블리 명령어는 다음 두 요소로 구성된다.

|                                  |
|----------------------------------|
| 연산자 (operation) + 피연산자 (operand) |
|----------------------------------|

예:

|              |
|--------------|
| add rax, rbx |
|--------------|

- add: 연산자
- rax, rbx: 피연산자

명령어는 CPU가 수행할 연산을 정의하며,  
피연산자는 연산에 사용될 데이터의 위치 및 결과 저장 위치를 지정한다.

### [7-1-1] 피연산자 표기법(Operand Notation)

이 교재에서는 명령어 설명 시 추상 기호를 사용한다.  
각 기호의 의미는 다음과 같다.

#### [1] <reg> - 레지스터 피연산자

레지스터만 허용됨

```
add rax, rbx
```

- rax → <reg>
- rbx → <reg>

#### [2] <reg8, reg16, reg32, reg64> - 크기 지정 레지스터

레지스터 이름은 곧 데이터 크기를 의미한다.

| 크기   | 예   |
|------|-----|
| 8비트  | al  |
| 16비트 | ax  |
| 32비트 | eax |
| 64비트 | rax |

예:

```
mov al, 1      ; reg8
mov ax, 2      ; reg16
mov eax, 3     ; reg32
mov rax, 4     ; reg64
```

중요한 특징:

- 32 비트 레지스터에 값을 쓰면 상위 32 비트는 자동으로 0 으로 확장
- 8 비트/16 비트 쓰기는 상위 비트를 변경하지 않는다.

#### [3] <dest> - 목적지 피연산자(결과가 저장됨)

연산 결과가 저장되는 위치

```
mov rax, rbx
```

- rax → <dest>
- rbx → <src>

메모리도 목적지가 될 수 있다.

```
mov [result], rax
```

- [result] → <dest>

[4] <src> - 소스 피연산자

연산에 사용되지만 값이 변하지 않는 피연산자

```
sub rax, rcx
```

- rcx → <src>

[5] <imm> - 즉시값(상수)

명령어 내부에 포함된 상수 값

```
mov rax, 10
```

```
mov, rbx, 0xFF
```

- 10 → <imm>

- 0xFF → <imm>

기본적으로 숫자는 10 진수로 해석된다.

16 진수는 반드시 0x 접두어를 사용한다.

[6] <mem> - 메모리 피연산자

대괄호 []로 표현된다.

```
mov rax, [value]
```

```
mov eax, [rbp - 4]
```

- [value] → <mem>

- [rbp - 4] → <mem>

메모리는 직접 값이 아니라 주소를 통한 간접 참조(indirection)이다.

[7] <op> 또는 <operand>

레지스터 또는 메모리 중 하나가 올 수 있다.

```
inc rax
```

```
inc [counter]
```

- rax → <op>

- [counter] → <op>

[8] <op8>, <op16>, <op32>, <op64>

크기를 명시해야 하는 경우 사용된다.

```
inc byte [flag] ; op8
```

```
inc word [data] ; op16
```

```
inc dword [count] ; op32
```

```
inc qword [total] ; op64
```

메모리 연산 시 크기 지정은 필수이다.

### [9] <label> - 프로그램 레이블

코드 위치를 식별하는 이름

```
loop_start:
    dec rcx
    jnz loop_start
```

- loop\_start → <label>

레이블은 주소를 의미하며, 실행 명령어가 아니다.

### [7-1-2] 두 피연산자 명령어의 의미

대부분의 x86-64 정수 명령어는 다음 형태를 가진다.

```
operation dest, src
```

의미는 다음과 같다.

```
dset ← dest(operation) src
```

예:

```
add rax, rbx
```

의미:

```
rax ← rax + rbx
```

결과는 항상 첫 번째 피연산자에 저장된다.

### [7-1-3] 메모리 연산 제약

X86-64의 일반 규칙:

- 레지스터 → 레지스터 가능
- 레지스터 → 메모리 가능
- 메모리 → 레지스터 가능
- 메모리 → 메모리 직접 연산은 불가능

즉, 다음은 허용되지 않는다.

```
mov [a], [b] ;불가능
```

반드시 중간 레지스터를 사용해야 한다.

#### [7-1-4] 수 체계 표기

| 표기     | 의미                |
|--------|-------------------|
| 15     | 10진수              |
| 0x0F   | 16진수              |
| 0b1111 | (어셈블러에 따라 지원) 2진수 |

#### [7-1-5] 명령어 실행의 기본 모델

CPU 는 다음 순서로 명령어를 처리한다.

1. RIP 가 가리키는 명령어 인출(fetch)
2. 디코딩(decode)
3. 피연산자 읽기
4. 연산 수행
5. 결과 저장
6. RIP 증가

점프/호출 명령어는 RIP 를 직접 변경한다.

#### [7-2] 데이터 이동

CPU 는 모든 연산을 레지스터에서만 수행한다.  
메모리에 저장된 값은 그대로는 연산할 수 없다.

따라서 항상 다음 순서를 따른다.

1. 메모리 → 레지스터로 값 이동
2. 레지스터에서 연산 수행
3. 결과를 다시 메모리로 저장

이 기본 동작을 수행하는 명령어가 mov 이다.

#### [7-2-1] mov 명령어의 의미

mov <dest>, <src>

의미는 단순하다.

소스의 값을 목적지로 복사한다.

- 소스 값은 변하지 않는다.
- 목적지에 값이 덮어써진다.
- 두 피연산자의 크기는 반드시 같아야 한다.

예를 들어,

```
mov eax, ebx
```

이면

```
eax ← ebx
```

이다.

[7-2-2] 왜 메모리 → 메모리가 안되는가?

많이 헷갈리는 부분이다.

```
mov [a], [b] ; 불가능
```

안되는 이유는

CPU 는 연산을 레지스터를 중심으로 수행하도록 설계되어 있다.

메모리는 CPU 내부 연산 장치(ALU)와 직접 연결되어 있지 않다.

따라서 데이터 이동은 항상 다음 구조를 따른다.

```
메모리 → 레지스터 → 메모리
```

즉, 레지스터가 반드시 중간 매개체가 된다.

올바른 형태는 다음과 같다.

```
mov eax, [b]
```

```
mov [a], eax
```

[7-2-3] 목적지는 즉시값이 될 수 없다.

```
mov 10, eax ; 불가능
```

이유는 간단하다.

즉시값은 저장 공간이 아니라 단순한 상수이기 때문이다.

목적지는 반드시

- 레지스터
- 또는 메모리

여야 한다.



#### [7-2-4] 크기 일치 규칙

두 피연산자는 반드시 같은 크기여야 한다.

```
mov eax, ebx    ; 32 비트  
mov ax, bx      ; 16 비트  
mov al, bl      ; 8 비트
```

다른 크기를 직접 섞을 수 없다.

CPU 는 “자동 형 변환”을 해주지 않는다.

#### [7-2-5] 32 비트 레지스터 쓰기의 특별 동작

x86-64 에서 매우 중요한 규칙이 하나 있다.

32 비트 레지스터에 값을 쓰면  
대응하는 64 비트 레지스터의 상위 32 비트가 0 으로 초기화된다.

예를 보자,

```
mov rcx, -1
```

이때 rcx의 값은

```
0xFFFFFFFFFFFFFFFF
```

이다.

이후,

```
mov eax, 100  
mov ecx, eax
```

실행 후 rcx 는

```
0x0000000000000064
```

가 된다.

이러한 이유는

ecx 는 32 비트 레지스터이므로 64 비트 레지스터 rcx 의 하위 32 비트를 덮어쓴다.

그리고 x86-64 설계 규칙에 의해 상위 32 비트는 자동으로 0 이 된다.

이 규칙은 32 비트 정수 레지스터에 쓸 때만 적용된다.

8 비트나 16 비트 쓰기에는 적용되지 않는다.

### [7-2-6] 데이터 선언과 CPU 의 관계

다음 선언을 보자

```
dValue dd 0
```

이것은

- 4 바이트 메모리 확보
- 그 주소에 dValue 라는 이름 부여

라는 의미다.

중요한 점은 이것이 어셈블러 단계의 정보라는 것이다.

프로그램이 실행되면 CPU 는

- 이 변수가 몇 바이트인지
- 어떤 타입인지

전혀 모른다.

CPU 는 오직 “주소”와 “명령어 포함된 크기 정보”만 본다.

### [7-2-7] 메모리 저장 시 크기 지정이 필요한 이유

다음 코드를 생각해보자.

```
mov [dValue], 27
```

이 명령은 모호하다.

CPU 는 27 을

- 1 바이트로 쓸지
- 2 바이트로 쓸지
- 4 바이트로 쓸지
- 8 바이트로 쓸지

알 수 없다.

그래서 반드시 크기를 명시해야 한다.

```
mov dword [dValue], 27
```

의미

- dValue 가 가리키는 주소에
- 4 바이트 크기로
- 27 을 저장

여기서 dword 는 CPU 가 아니라 어셈블러에게 주는 정보다.

### [7-2-8] 크기 생략이 가능한 이유

다음은 가능하다.

```
mov eax, [dNum]
```

가능한 이유:

- eax 는 32 비트 레지스터
- 따라서, 메모리도 32 비트로 읽는 것이 자동으로 결정됨

마찬가지로,

```
mov [dAns], eax
```

도 가능하다.

하지만 학습 단계에서는 명시하는 것이 더 좋다.

### [7-2-9] 실전 예제

데이터 선언:

```
dValue dd 0
bNum db 42
wNum dw 5000
dNum dd 73000
qNum dq 73000000
bAns db 0
wAns dw 0
dAns dd 0
qAns dq 0
```

다음 연산을 수행한다고 하자.

```
dValue = 27
bAns = bNum
wAns = wNum
dAns = dNum
qAns = qNum
```

구현:

```
mov dword [dValue], 27

mov al, byte [bNum]
mov byte [bAns], al

mov ax, word [wNum]
mov word [wAns], ax
```

```
mov eax, dword [dNum]
mov dword [dAns], eax

mov rax, qword [qNum]
mov qword [qAns], rax
```

### [7-3] Addresses and Values(주소 vs 값)

어셈블리를 배우면서 가장 많이 헷갈리는 부분이 바로 이것이다.

변수 이름은 “값”이 아니라 “주소”다.

그리고

대괄호 []는 “그 주소에 들어 있는 값”을 의미한다.

이 차이를 정확히 이해하지 못하면

코드는 돌아가는데 왜 그런지 이해하지 못하는 상황이 계속 발생한다.

#### [7-3-1] 대괄호 []의 의미

x86-64 에서 메모리에 접근하는 유일한 방법은 대괄호를 사용하는 것이다.

- []를 쓰면 → 메모리 접근(값)
- []를 쓰지 않으면 → 주소 자체

이것은 문법 차이가 아니라 의미 차이다. 그래서 어셈블리는 오류를 내지 않는다.

둘 다 “문법적으로는” 맞기 때문이다.

이 점이 초보자를 특히 혼란스럽게 만든다.

#### [7-3-2] 예제로 보는 주소 vs 값

다음 선언이 있다고 가정하자.

```
wNum dw 5000
```

이 변수는 메모리 어딘가에 2 바이트 공간을 차지하고 있고,  
그 위치에 5000 이 저장되어 있다.

[1] 값에 접근하는 경우

```
mov rax, word [wNum]
```

의미는 다음과 같다.

1. wNum 이 가리키는 주소로 이동
2. 그 주소에 있는 2바이트 값을 읽음
3. 그 값을 rax 에 저장

즉,

```
rax ← (wNum 이 가리키는 주소의 내용)
```

[2] 주소에 접근하는 경우

```
mov rax, wNum
```

이 명령은 전혀 다르다.

이번에는 대괄호가 없다.

따라서 의미는

```
rax ← wNum 의 주소
```

즉, 메모리를 읽지 않는다.

단순히 “위치 값(주소)”을 rax 에 넣는다.

[7-3-3] 왜 어셈블러는 에러를 내지 않는가?

두 경우 모두 논리적으로 맞기 때문이다.

- 주소도 정수 값이다.
- 메모리 값도 정수 값이다.

CPU 입장에서는 둘 다 그냥 숫자다.

따라서 문법 오류가 아니다.

하지만 의미는 완전히 다르다.

[7-3-4] lea 명령어(Load Effective Address)

주소가 필요할 때는 lea 명령어를 사용할 수도 있다.

형식:

```
lea <reg64>, <mem>
```

의미:

<mem>의 “주소 계산 결과”를 레지스터에 저장한다.

메모리 내용은 읽지 않는다.

### [1] 예제 1

```
lea rcx, byte [bVar]
```

의미:

```
rcx ← bVar 의 주소
```

여기서 중요한 점:

- 대괄호가 있어도
- 메모리를 읽지 않는다.

왜냐하면 lea 는 “주소 계산”명령이기 때문이다.

### [2] 예제 2

```
lea rsi, dword [dVar]
```

이 역시

```
rsi ← bVar 의 주소
```

메모리 값을 읽는 것이 아니다.

### [7-3-5] mov 와 lea 의 차이

비슷해 보이지만 내부 동작은 완전히 다르다.

```
mov rax, [dVar]
```

- 메모리 접근 발생
- 실제 값을 읽음

```
lea rax, [dVar]
```

- 메모리 접근 없음
- 주소 계산만 수행

### [7-3-6] 직관적으로 이해하기

c 언어와 비교하면 이해가 쉽다.

```
mov rax, wNum      ; rax = &wNum  
mov rax, [wNum]    ; rax = wNum
```

즉,

- 변수 이름 = 주소
- [변수 이름] = 그 주소에 들어 있는 값

#### [7-4] 변환 명령어(Conversion Instructions)

어셈블리에서 데이터의 크기(size)는 단순한 형식 문제가 아니라 연산 결과에 직접적인 영향을 주는 핵심 요소이다.

예를 들어,

- 1 바이트 값을 4 바이트 연산에 사용하려 하거나
- 4 바이트 값을 1 바이트 메모리에 저장하려는 경우

크기가 맞지 않으면 의도와 다른 결과가 발생한다.

따라서, 데이터 크기 변환(conversion)은 연산의 정확성을 유지하기 위한 필수 과정이다. 변환은 크게 두 가지로 나뉜다.

1. 축소 변환(Narrowing)
2. 확장 변환(Widening)

##### [7-4-1] 축소 변환(Narrowing Conversion)

축소 변환은 큰 크기의 데이터를 작은 크기로 변환하는 것을 의미한다.

예:

- word → byte
- dword → word
- qword → dword

##### [1] 축소 변환의 특징

축소 변환에는 특별한 명령어가 필요하지 않다.

이유는 큰 레지스터의 하위 부분(lower portion)에 직접 접근하면 되기 때문이다.

예를 들어,

- rax 는 64 비트
- eax 는 32 비트
- ax 는 16 비트
- al 는 8 비트

즉, 하위 부분을 사용하면 자동으로 축소 효과가 발생한다.

## [2] 예제 1: 안전한 축소

```
mov rax, 50  
mov byte [bVal], al
```

- rax = 50(0x32)
- al = rax 의 하위 8 비트
- bVal ← 0x32

50 은 1 바이트 범위(0~255)에 포함되므로 문제 없이 정확한 값이 저장된다.

## [3] 예제 2: 위험한 축소

```
mov rax, 500  
mov byte [bVal], al
```

- 500 = 0x01F4

하지만 al 은 하위 8 비트만 취한다.

```
0x01F4 → 0xF4
```

결과:

```
bVal = 0xF4(244)
```

원래 값 500 은 사라지고, 하위 8 비트만 남는다.  
이것을 잘림(truncation)이라고 한다.

중요한점은 어셈블러는 어떠한 경고도 하지 않는다.  
CPU 는 단순히 “하위 비트만 사용”할 뿐이며, 값의 의미를 고려하지 않는다.  
즉, 축소 변환의 정확성은 전적으로 프로그래머의 책임이다.

## [7-4-3] 확장 변환(Widening Conversion)

확장 변환은 작은 크기의 데이터를 큰 크기로 변환하는 것이다.

예:

- ```
- byte → word  
- word → dword  
- dword → qword
```

축소와 달리, 확장은 단순하지 않다.  
왜냐하면, 새로 생기는 상위 비트를 어떻게 채울 것인가를 결정해야 하기 때문이다.



이때 반드시 고려해야 할 요소가 있다.

- 값이 부호 없는 값인가?(unsigned)
- 값이 부호 있는 값인가?(signed)

이 구분에 따라 확장 방식이 달라진다.

### [1] 부호 없는 확장(Zero Extension)

부호 없는 값은 항상 0 이상이다.

따라서 확장 시 상위 비트는 반드시 0 으로 채워야 한다.

이것을 Zero Extension 이라고 한다.

#### [1-1] 수동 방식

```
mov al, 50  
mov rbx, 0  
mov bl, al
```

과정:

1. rbx 를 0 으로 초기화
2. 하위 8 비트에 값 복사

결과:

```
rbx = 50
```

상위 비트는 모두 0 이다.

#### [1-2] movzx 명령어

이 과정을 한 번에 수행하는 명령어:

```
movzx <dest>, <src>
```

의미: src 를 dest 로 이동하면서 상위 비트를 0 으로 채운다.

```
movzx eax, byte [bVal]
```

bVal = 0xF4 라고 가정하자.

결과:

```
eax = 0x000000F4
```

십진수로 244.

상위 24 비트가 모두 0 으로 채워진다.

### [1-3] 허용 형태

```
movzx reg16, op8  
movzx reg32, op8  
movzx reg32, op16  
movzx reg64, op 8  
movzx reg64, op16
```

주의:

- 두 피연산자가 모두 메모리일 수 없음
- 목적지는 반드시 레지스터
- 즉시값 사용 불가
- dword → qword 직접 변환은 movzx 로 불가

### [1-4] 중요한 자동 규칙

32 비트 레지스터에 값을 쓰면

64 비트 레지스터의 상위 32 비트는 자동으로 0 이된다.

예:

```
mov eax, 5
```

→ rax = 0x0000000000000005

크기가 확장되기 때문에, 상위 비트(upper-order bits)는 원래 값의 부호(sign)에 따라 설정되어야 한다. 따라서 데이터 타입이 부호 있는 값(signed)인지, 부호 없는 값(unsigned)인지를 반드시 알고 있어야 하며, 이에 맞는 적절한 처리 과정이나 명령어를 사용해야 한다.

### [2] 부호 있는 확장(Sign Extension)

부호 있는 값은 양수 또는 음수가 될 수 있다.

음수는 2의 보수 형태로 표현되며,

최상위 비트(MSB)가 1 이면 음수이다.

확장 시, 최상위 비트를 최상위 비트 전체에 복사해야 한다.

이것을 Sign Extension 이라고 한다.

#### [2-1] 예제 1

ax 에 -7 이 저장되어 있다고 가정하자.

```
-7 = 0xFFFF9
```

최상위 비트 = 1(음수)

이를 `eax` 로 확장하면

0xFFFFFFFF9

상위 16 비트가 전부 1로 채워진다.

음수 의미가 유지된다.

## [2-2] `movsx` / `movsxd`

보다 일반적인 명령어:

`movsx <dest>, <src>`

`movsxd <dest>, <src>`

### `movsx`

작은 값을 큰 레지스터로 이동하면서 부호 비트를 확장한다.

### `movsxd`

`dword` → `qword` 전용 부호 확장

## [2-3] 예제 2

`movsx, eax, byte [bVal]`

`bVal = 0xF4` 라고 하면 `0xF4` 는 signed 8 비트에서 -12 다.

결과:

`eax = 0xFFFFFFFF4(-12)`

## [3] A 레지스터 전용 확장 명령어

옛날 x86 방식으로, A 레지스터 전용 명령어들도 존재한다.

| 명령어               | 설명                                      |
|-------------------|-----------------------------------------|
| <code>cbw</code>  | <code>al</code> → <code>ax</code>       |
| <code>cwd</code>  | <code>ax</code> → <code>dx:ax</code>    |
| <code>cwde</code> | <code>ax</code> → <code>eax</code>      |
| <code>cdq</code>  | <code>eax</code> → <code>edx:eax</code> |
| <code>cdqe</code> | <code>eax</code> → <code>rax</code>     |
| <code>cqo</code>  | <code>rdx:rax</code>                    |

이들은 모두 A 레지스터에서만 동작한다.

## [4] `movzx` vs `movsx` 정리

### [4-1] 공통점

- 작은 데이터를 큰 레지스터로 확장
- 목적지는 반드시 레지스터
- 메모리 → 메모리 불가

#### [4-2] 차이

movzx

- 상위 비트를 0 으로 채움
- unsigned 값에 사용

movsx

- 부호 비트를 복사
- signed 값에 사용

#### [4-3] 같은 값 0xF4 의 차이

movzx eax, byte [bVal]

→  $eax = 0x000000F4(244)$

movsx eax, byte [bVal]

→  $eax = 0xFFFFFFFFF4(-12)$

같은 비트 패턴이 해석 방식에 따라 완전히 다른 숫자가 된다.

## [7-5] 정수 산술 명령어

정수 산술 명령어는

CPU가 정수 값에 대해 수행하는 기본 연산을 담당한다.

- 덧셈
- 뺄셈
- 곱셈
- 나눗셈

### [7-5-1] 덧셈(add)

#### [1] 기본 형식

```
add <dest>, <src>
```

동작:

```
<dest> = <dest> + <src>
```

- 결과는 dest에 저장된다.
- 기존 dest 값은 덮어쓰인다.
- src 값은 변경되지 않는다.

#### [2] 피연산자 규칙

1. dest와 src는 반드시 같은 크기여야 한다.
2. dest는 즉시값(immediate)이 될 수 없다.
3. 두 피연산자가 모두 메모리일 수 없다.
4. 메모리 → 메모리 연산이 필요하면 레지스터를 거쳐야 한다.

#### [3] 예제 데이터

```
bNum1 db 42  
bNum2 db 73  
bAns db 0
```

```
wNum1 dw 4321  
wNum2 dw 1234  
wAns dw 0
```

```
dNum1 dd 42000  
dNum2 dd 73000  
dAns dd 0
```

```
qNum1 dq 42000000  
qNum2 dq 73000000
```

```
qAns dq 0
```

#### [4] 바이트 덧셈 예제

```
mov al, byte [bNum1]
add al, byte [bNum2]
mov byte [bAns], al
```

#### 해석

1. bNum1 값을 al 에 로드
2.  $al = al + bNum2$
3. 결과를 bAns 에 저장

메모리끼리 직접 더할 수 없기 때문에 반드시 레지스터를 사용한다.

#### [5] 워드 / 더블워드 / 퀴드워드

모든 크기에서 구조는 동일하다.

```
mov ax, word [wNum1]
add ax, word [wNum2]
mov word [wAns], ax

mov eax, dword [dNum1]
add eax, dword [dNum2]
mov dword [dAns], eax

mov rax, qword [qNum1]
add rax, qword [qNum2]
mov qword [qAns], rax
```

연산 방식은 동일하고 크기만 다르다.

#### [6] 타입 지정 생략 가능성

예:

```
mov al, [bNum1]
```

처럼 크기가 명확하면 byte, word 등을 생략할 수 있다.

하지만 문서화 목적과 가독성을 위해 명시하는 것이 좋은 습관이다.

## [7-5-2] 증가 명령어(inc)

### [1] 형식

```
inc <operand>
```

동작:

```
<operand> = <operand> + 1
```

이는 다음과 정확히 동일하다.

```
add <operand>, 1
```

### [2] 예제

```
inc rax  
inc byte [bNum]  
inc word [wNum]  
inc dword [dNum]  
inc qword [qNum]
```

메모리 피연산자는 반드시 크기를 명시해야 한다.

### [3] add vs inc 차이

수학적으로 동일하지만, 플래그 처리에서 차이가 있다.

- add 는 Carry Flag(CF)를 변경한다.
- inc 는 CF 를 변경하지 않는다.

따라서 다중 정밀도 연산에서는 inc 대신 add 를 사용하는 경우가 많다.

## [7-5-3] 캐리를 포함한 덧셈(adc)

### [1] 필요한 이유

레지스터는 64 비트까지 표현이 가능하다.

그렇다면 128 비트는 어떻게 더할까?

64 비트씩 나누어 계산한다.

이때 하위 부분에 발생한 캐리(carry)를 상위 부분 계산에 포함해야 한다.

이를 위한 명령어가 adc(add with carry)이다.

### [2] 형식

```
adc <dest>, <src>
```

동작:

```
<dest> = <dest> + <src> + CF
```

여기서 CF 는 Carry Flag 이다.

### [3] 자리올림 개념 복습

|      |
|------|
| 17   |
| + 25 |
| ---- |
| 42   |

1.  $7 + 5 = 12 \rightarrow 2$  저장, 캐리 1 발생
2.  $1 + 2 + 1 = 4$

CPU 도 동일한 방식으로 동작한다.

### [4] 매우 중요한 규칙

adc 는 반드시 add 바로 다음에 와야한다.

이유는 다른 명령어가 실행되면 RFLAGS 레지스터의 CF 값이 바뀔 수 있기 때문이다.

### [5] 128 비트 덧셈 예제

데이터 선언

|                                 |
|---------------------------------|
| dquad1 ddq 0x1A0000000000000000 |
| dquad2 ddq 0x2C0000000000000000 |
| dqSum ddq 0                     |

각 변수는 128 비트이다.

64 비트 CPU 에서는 두 개의 64 비트 레지스터로 나누어 처리한다.

### [1 단계] 값 로드

|                             |      |
|-----------------------------|------|
| mov rax, qword [dquad1]     | ;LSQ |
| mov rdx, qword [dquad1 + 8] | ;MSQ |

- rax = 하위 64 비트
- rdx = 상위 64 비트

+8 은 8 바이트(64 비트) 이동을 의미한다.

### [2 단계] 하위 부분 덧셈

|                         |
|-------------------------|
| add rax, qword [dquad2] |
|-------------------------|

- LSQ 끼리 더함
- 캐리가 발생하면  $CF = 1$



[3 단계] 상위 부분 덧셈

```
adc rdx, qword [dquad2 + 8]
```

- MSQ 끼리 더함
- 이전 캐리(CF)를 포함

[4 단계] 결과 저장

```
mov qword [dqSum], rax  
mov qword [dqSum+8], rdx
```

이렇게 하면 128 비트 결과가 정확히 계산된다.

#### [7-5-4] 뺄셈

정수 뺄셈은 두 피연산자의 차를 계산하여 그 결과를 목적지에 저장하는 연산이다.

##### [1] 기본 형식

```
sub <dest>, <src>
```

동작:

```
<dest> = <dest> - <src>
```

즉,

- 목적지 피연산자에서
- 소스 피연산자의 값을 빼고
- 그 결과를 목적지에 저장한다.

이때,

- 기존 목적지 값은 결과로 덮어쓰인다.
- 소스 피연산자는 변경되지 않는다.

##### [2] 피연산자 규칙

1. 목적지와 소스는 반드시 같은 크기여야 한다.
2. 목적지 피연산자는 즉시값(immediate)이 될 수 없다.
3. 두 피연산자 모두 메모리일 수 없다. (레지스터를 거쳐야함)
4. 64 비트 즉시값은 직접 사용할 수 없다.(부호 확장된 32 비트 즉시값만 허용)

##### [3] 예제 데이터 선언

```
bNum1 db 73
```

```
bNum2 db 42
```

```
bAns db 0
```

```
wNum1 dw 1234
```

```
wNum2 dw 4321
```

```
wAns dw 0
```

```
dNum1 dd 73000
```

```
dNum2 dd 42000
```

```
dAns dd 0
```

```
qNum1 dq 73000000
```

```
qNum2 dq 42000000
```

```
qAns dq 0
```

#### [4] 기본 뺄셈 예제

다음 연산을 수행한다고 가정하자.

```
bAns = bNum1 - bNum2
wAns = wNum1 - wNum2
dAns = dNum1 - dNum2
qAns = qNum1 - qNum2
```

##### [4-1] 바이트 단위 연산

```
mov al, byte [bNum1]
sub al, byte[bNum2]
mov byte [bAns], al
```

동작 과정

1. bNum1 을 al 에 로드
2.  $al = al - bNum2$
3. 결과를 bAns 에 저장

메모리 간 직접 연산은 불가능하므로 반드시 레지스터를 사용한다.

##### [4-2] 워드 단위 연산

```
mov ax, word [wNum1]
sub ax, word [wNum2]
mov word [wAns], ax
```

##### [4-3] 더블워드 연산

```
mov eax, dword [dNum1]
sub eax, dword [dNum2]
mov dword [dAns], eax
```

##### [4-4] 쿼드 워드 연산

```
mov rax, qword [qNum1]
sub rax, qword [qNum2]
mov qword [qAns], rax
```

#### [5] 자료형 지정 생략 가능 여부

예를 들어,

```
mov al, [bNum1]
```

처럼 레지스터가 크기를 명확히 정의해주면 byte, word 등의 타입을 생략할 수 있다.  
하지만 코드의 명확성과 일관성을 위해 명시적으로 작성하는 것이 권장된다.

## [7-5-5] 감소 명령어(dec)

### [1] 기본 형식

```
dec <operand>
```

동작:

```
<operand> = <operand> - 1
```

이는 다음과 동일하다.

```
sub <operand>, 1
```

### [2] 예제

```
dec rax  
dec byte [bNum]  
dec word [wNum]  
dec dword [dNum]  
dec qword [qNum]
```

메모리 피연산자를 사용할 경우 반드시 크기를 명시해야 한다.

### [3] sub vs dec 의 차이

수학적 결과는 동일하지만 플래그 처리에서 차이가 있다.

- sub 는 Carry Flag(CF)를 변경한다.
- dec 는 Carry Flag(CF)를 변경하지 않는다.

따라서 다중 정밀도 연산에서는 dec 대신 sub 를 사용하는 것이 일반적이다.

### [4] 부호 있는 / 부호 없는 뺄셈

sub 명령어는

signed, unsigned 구분 없이 동일한 이진 연산을 수행한다.

CPU 는 단순히 비트를 계산할 뿐이며,

- signed 해석은 Overflow Flag(OF)로 판단
- unsigned 해석은 Carry Flag(CF)로 판단

한다.

데이터의 해석과 의미는 전적으로 프로그래머의 책임이다.

## [7-5-6] 정수 곱셈

곱셈은 두 정수를 곱하여 결과를 생성하는 연산이다.

그러나 덧셈, 뺄셈과 달리, 곱셈은 결과의 크기가 두 배가 될 수 있다는 점이 매우 중요하다.

### [1] 결과 크기 원리

$n \text{ 비트} \times n \text{ 비트} \rightarrow 2n \text{ 비트 결과}$

| 연산 크기       | 결과 크기 |
|-------------|-------|
| 8비트 × 8비트   | 16비트  |
| 16비트 × 16비트 | 32비트  |
| 32비트 × 32비트 | 64비트  |
| 64비트 × 64비트 | 128비트 |

이 때문에 곱셈은 레지스터 쌍을 사용하는 특별한 구조를 가진다.

### [2] 부호 없는 곱셈 - mul

#### [2-1] 기본 형식

mul <src>

- 단일 피연산자 형식만 존재
- <src>는 레지스터 또는 메모리
- 즉시값(immediate) 사용 불가

#### [2-2] 암묵적 피연산자(A 레지스터)

mul 은 항상 A 레지스터를 자동으로 사용한다.

| 연산 크기 | 실제 계산     | 결과 저장   |
|-------|-----------|---------|
| 8비트   | AL × src  | AX      |
| 16비트  | AX × src  | DX:AX   |
| 32비트  | EAX × src | EDX:EAX |
| 64비트  | RAX × src | RDX:RAX |

즉,

RDX:RAX = RAX × src

여기서 : 는 상위:하위를 의미한다.

#### [2-3] 왜 이렇게 복잡한가?

이 구조는 초기 x86 레지스터 아키텍처의 유산(legacy)이다.

하위 호환성을 유지하기 위해 현재까지 유지되고 있다.

## [2-4] 예제

### 8 비트 곱셈

```
mov al, [bNumA]
mul al
mov [wAns], ax
```

결과는 ax 에 저장된다.

### 16 비트 곱셈

```
mov ax, [wNumA]
mul word [wNumB]
mov word [dAns2], ax
mov word [dAns2 + 2], dx
```

결과는 DX:AX 에 저장된다.

### 64 비트 곱셈(128 비트 결과)

```
mov rax, [qNumA]
mul qword [qNumB]
mov [dqAns4], rax
mov [dqAns4 + 8], rdx
```

128 비트 결과는 RDX:RAX 에 저장된다.

### mul 의 핵심 특징

- unsigned 전용
- A 레지스터 만드시 사용
- 결과는 항상 전체 크기 생성
- 결과는 레지스터 쌍에 저장

### [3] 부호 있는 곱셈 - imul

imul 은 signed 정수를 처리한다.  
mul 보다 훨씬 유연하다.

#### [3-1] imul 의 세 가지 유형

##### [3-1-1] 단일 피연산자 형식

imul <src>

mul 과 동일한 구조지만 signed 해석을 한다.

| 연산 크기 | 결과            |
|-------|---------------|
| 8비트   | AX = AL × src |
| 16비트  | DX:AX         |
| 32비트  | EDX:EAX       |
| 64비트  | RDX:RAX       |

전체 결과가 필요할 때 사용한다.

##### [3-1-2] 두 피연산자 형식

imul <dest>, <src/imm>

동작:

dest = dest × src

특징:

- 목적지는 반드시 레지스터
- 즉시값 사용 가능
- 결과는 목적지 크기로 잘림(truncate)
- 바이트 목적지 레지스터는 지원하지 않음

예제:

```
mov ax, [wNumA]
imul ax, -13
mov [wAns1], ax
```

결과는 AX 크기(16 비트)로 잘려 저장한다.

##### [3-1-3] 세 피연산자 형식

imul <dest>, <src>, <imm>

동작:

dest = src × imm

특징:

- 원본 src 를 보존할 수 있음
- 결과를 다른 레지스터에 저장 가능
- 즉시값은 반드시 32 비트 이하

예제:

```
mov rcx, [qNumA]
imul rbx, rcx, 7096
mov [qAns1], rbx
```

장점:

- rcx 보존
- 결과를 rbx 에 저장

#### [4] 결과 잘림(truncation)의 의미

예:

32 비트 × 32 비트 → 실제 결과는 64 비트

그러나

```
imul eax, [dNumB]
```

결과는 EAX(32 비트)에만 저장된다.

상위 32 비트는 버려진다.

이는 C 언어에서 :

```
int a = b * c;
```

와 동일한 개념이다.

#### [5] 전체 결과가 필요한 경우

전체 64 비트 결과가 필요하다면:

```
imul <src> ;단일 피연산자 형식
```

을 사용해야 한다.

#### [6] 작은 자료형 결과 사용 시 주의

곱셈 결과가 더 큰 자료형으로 생성되더라도 값이 범위 내에 있다면 하위 비트만 사용 가능하다.

하지만 범위를 초과하면 오버플로우가 발생한다.



#### [7] mul vs imul 비교

| 구분       | mul      | imul              |
|----------|----------|-------------------|
| 곱셈 종류    | unsigned | signed            |
| 음수 처리    | 불가       | 가능                |
| 단일 피연산자  | 있음       | 있음                |
| 2·3 피연산자 | 없음       | 있음                |
| 즉시값 사용   | 불가       | 가능                |
| 결과 저장    | 레지스터 쌍   | 레지스터 쌍 또는 단일 레지스터 |
| 결과 크기    | 항상 전체    | 전체 또는 잘림          |
| 목적지 지정   | 불가       | 가능                |

#### [8] 오버플로우 판단

mul / imul 단일 형식에서:

- 상위 레지스터가 0 이면 → overflow 없음
- 상위 레지스터가 0 이 아니면 → overflow 발생

2·3 피연산자 형식에서는 잘려나간 비트가 존재하면 OF/CF 가 설정된다.

#### [9] 전체 요약

1. 곱셈은 결과가 두 배 크기
2. mul 은 unsigned 전용, A 레지스터 고정
3. imul 은 signed 전용, 매우 유연
4. 단일 형식은 전체 결과 생성
5. 2·3 피연산자 형식은 결과가 잘림
6. 레거시 설계로 인해 A/D 레지스터 쌍 사용

#### [7-5-4] 정수 나눗셈

정수 나눗셈은 두 정수를 나누어 몫(quotient)과 나머지(remainder)를 생성한다.

부호 있는 정수와 부호 없는 정수는 나눗셈 규칙이 다르기 때문에 서로 다른 명령어를 사용한다.

|            |                       |
|------------|-----------------------|
| div <src>  | ; 부호 없는 나눗셈(unsigned) |
| idiv <src> | ; 부호 있는 나눗셈(signed)   |

##### [1] 나눗셈의 기본 개념

정수 나눗셈은 다음 관계를 만족한다.

|                    |
|--------------------|
| 피제수 = 제수 × 몫 + 나머지 |
|--------------------|

중요한 점은:

나눗셈에서는 피제수(dividend)가 제수(divisor)보다 더 큰 크기를 가져야 한다.

- |                                                                                                            |
|------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>- 피제수: 나눗셈에서 나눔을 당하는 수</li> <li>- 제수: 나눗셈에서 어떠한 수를 나누는 수</li> </ul> |
|------------------------------------------------------------------------------------------------------------|

크기 규칙

| 제수 크기 | 피제수 크기 |
|-------|--------|
| 8비트   | 16비트   |
| 16비트  | 32비트   |
| 32비트  | 64비트   |
| 64비트  | 128비트  |

즉,

- |                                                                                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>- 8 비트로 나누려면 AX 가 필요</li> <li>- 16 비트로 나누려면 DX:AX 필요</li> <li>- 32 비트로 나누려면 EDX:EAX 필요</li> <li>- 64 비트로 나누려면 RDX:RAX 필요</li> </ul> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

이 구조는 곱셈과 동일한 레지스터 설계 때문이다.

##### [2] 레지스터 사용 구조

나눗셈은 항상 A 레지스터를 중심으로 동작한다.

| 연산 크기 | 피제수     | 몫   | 나머지 |
|-------|---------|-----|-----|
| 8비트   | AX      | AL  | AH  |
| 16비트  | DX:AX   | AX  | DX  |
| 32비트  | EDX:EAX | EAX | EDX |
| 64비트  | RDX:RAX | RAX | RDX |

피제수 설정을 잘못하면 Divide Error 예외가 발생한다.

나눗셈 오류의 대부분은 상위 레지스터(D 레지스터) 초기화 실패 때문이다.

### [3] 피제수 설정 방법

#### [3-1] 부호 없는 나눗셈

상위 부분은 항상 0 으로 설정한다.

예(32 비트 나눗셈):

```
mov eax, [value]
xor edx, edx    ; 상위 32 비트 = 0
div ecx
```

#### [3-2] 부호 있는 나눗셈

상위 부분은 부호 확장으로 설정해야 한다.

이를 위한 전용 명령어가 존재한다.

| 명령어 | 기능            |
|-----|---------------|
| cbw | AL → AX       |
| cwd | AX → DX:AX    |
| cdq | EAX → EDX:EAX |
| cqo | RAX → RDX:RAX |

예 (64 비트):

```
mov rax, [value]
cqo
idiv rbx
```

cqo 는 RAX 의 부호 비트를 RDX 전체로 확장한다.

### [4] 0 으로 나누면?

Divide Error 예외 발생(프로그램 중단)

0 으로 나누지말자

### [5] 바이트 나눗셈 예제(Unsigned)

bAns1 = bNumA/3

```
mov al, [bNumA]
xor ah, ah
mov b1, 3
div bl
mov [bAns1], al
```

bAns2 = bNumA / bNumB  
bRem2 = bNumA % bNumB

```
mov al, [bNumA]
xor ah, ah
div byte [bNumB]
mov [bAns2], al
mov [bRem2], ah

bAns3 = (bNumA * bNumC)/bNumB
mov al, [bNumA]
mul byte [bNumC]      ; AX 에 결과
div byte [bNumB]      ; AX 를 bNumB 로 나눔
mov [bAns3], al
```

#### [6] 워드 나눗셈(Unsigned)

wAns1 = wNumA / 5  
mov ax, [wNumA]

```
xor, dx, dx
mov bx, 5
div bx
mov [wAns1], ax
```

wAns2 = wNumA / wNumB  
wRem2 = wNumA % wNumB

```
mov ax, [wNumA]
xor dx, dx
div word [wNumB]
mov [wAns2], ax
mov [wRem2], dx
```

#### [7] 더블워드 나눗셈(Signed)

dAns1 = dNumA / 7

```
mov eax, [dNumA]
cdq
mov ecx, 7
idiv ecx
mov [dAns1], eax
```

dAns2 = dnumA / dNumB  
dRem2 = dNumA % dNumB

```
mov eax, [dNumA]
cdq
idiv dword [dNumB]
mov [dAns2], eax
mov [dRem2], edx
```

[8] 쿼드워드 나눗셈(signed)

qAns1 = qNumA / 9

```
mov rax, [qNumA]
cqo
mov rbx, 9
idiv rbx
mov [qAns1], rax
```

qAns2 = qNumA / qNumB

qRem2 = qNumA % qNumB

```
mov rax, [qNumA]
cqo
idiv qword [qNumB]
mov [qAns2], rax
mov [qRem2], rdx
```

[9] div/idiv 요약표

| 크기   | 명령어        | 피제수              | 제수    | 몫   | 나머지 |
|------|------------|------------------|-------|-----|-----|
| 8비트  | div r/m8   | AX               | r/m8  | AL  | AH  |
| 8비트  | idiv r/m8  | AX<br>(부호확장)     | r/m8  | AL  | AH  |
| 16비트 | div r/m16  | DX:AX            | r/m16 | AX  | DX  |
| 16비트 | idiv r/m16 | DX:AX<br>(부호확장)  | r/m16 | AX  | DX  |
| 32비트 | div r/m32  | EDX:EAX          | r/m32 | EAX | EDX |
| 32비트 | idiv r/m32 | EDX:EAX<br>(cdq) | r/m32 | EAX | EDX |
| 64비트 | div r/m64  | RDX:RAX          | r/m64 | RAX | RDX |
| 64비트 | idiv r/m64 | RDX:RAX<br>(cqo) | r/m64 | RAX | RDX |

[10] 자주 발생하는 오류

1. 상위 레지스터 초기화 안 함 → Divide Error
2. 부호 확장 안 함 → 잘못된 결과
3. 몫이 레지스터 범위 초과 → Divide Error
4. 즉시값을 제수로 사용 → 문법 오류

[11] 핵심 정리

1. 나눗셈은 항상 A 레지스터 기반
2. 피제수는 두 배 크기 필요
3. unsigned → 상위 0
4. signed → cdq / cqo 필수
5. 몫은 A 레지스터
6. 나머지는 D 레지스터
7. 0 으로 나누면 예외 발생

## [7-6] 논리 명령어(Logical Instructions)

논리 명령어는 비트 단위(bitwise)로 연산을 수행한다.

- 산술 연산(add, sub 등)은 “값”을 계산하는 연산이고
- 논리 연산은 “비트 패턴”을 조작하는 연산이다.

CPU는 결국 모든 데이터를 2 진수로 처리하므로,  
논리 연산은 매우 근본적인 연산이다.

대표적인 명령어는

- AND
- OR
- XOR
- NOT

이들은 조건 검사, 마스크 처리, 비트 설정/제거, 플래그 조작 등에서 매우 자주 사용된다.

### [7-6-1] 논리 연산

진리표(1 비트 기준)

| A | B | AND | OR | XOR |
|---|---|-----|----|-----|
| 0 | 0 | 0   | 0  | 0   |
| 0 | 1 | 0   | 1  | 1   |
| 1 | 0 | 0   | 1  | 1   |
| 1 | 1 | 1   | 1  | 0   |

NOT 은 단항 연산:

| A | NOT A |
|---|-------|
| 0 | 1     |
| 1 | 0     |

#### [1] AND 명령어

##### [1-1] 형식

and <dest>, <src>

##### [1-2] 가능한 형태:

and <dest>, <src>  
and <reg>, <reg>  
and <reg>, <imm8/16/32>  
and <reg>, <mem>  
and <mem>, <reg>  
and <mem>, <imm8/16/32>

### [1-3] 동작

```
dest = dest & src
```

각 비트를 비교하여 둘 다 1 일 때만 1 이 된다.

### 예제 분석

```
mov eax, 11010110b  
and eax, 00001111b
```

### 비트별 계산

```
  11010110  
&00001111  
-----  
  00000110
```

AND 는 “공통된 1 만 남기는 연산”이다.

마스크가 1 인자리만 남고, 나머지는 0 이 된다.

→ AND 는 비트를 걸러내는 필터 역할을 한다.

### [1-4] 주요 특징

1. 두 피연산자 모두 메모리일 수 없음
2. 목적지는 즉시값 불가
3. 64 비트 즉시값은 직접 사용 불가

### [1-5] 플래그 영향

- ZF: 결과가 0 이면 1
- SF: 결과의 최상위 비트 복사
- PF: 하위 바이트에서 1 비트 개수가 짝수이면 1
- CF/OF: 0 으로 클리어

AND 는 캐리/오버플로우 개념이 없다.

### [1-6] 대표 활용: 마스크(mask)

특정 비트만 남기기

```
and eax, 0xFF
```

하위 8 비트만 유지

특정 비트 제거

```
and eax, 0xFFFFF0
```

최하위 비트 제거



## [2] OR 명령어

### [2-1] 형식

```
or <dest>, <src>
```

### [2-2] 동작

```
dest = dest | src
```

### [2-3] 플래그 영향

- ZF, SF, PF 설정
- CF, OF : 0 으로 클리어

### [2-4] 대표 활용: 비트 설정

특정 비트를 1 로 만들기

```
or eax, 1
```

최하위 비트를 강제로 1 로 설정

여러 비트 켜기

```
or eax, 0b1010
```

1 번, 3 번 비트 설정

## [3] XOR 명령어

### [3-1] 형식

```
xor <dest>, <src>
```

### [3-2] 동작

```
dest = dest ^ src
```

같으면 0, 다르면 1

### [3-3] 플래그 영향

- ZF, SF, PF 설정
- CF, OF: 0 으로 클리어

### [3-4] 매우 중요한 패턴

레지스터 0 으로 초기화

```
xor eax, eax
```

이는 다음보다 효율적이다.

```
mov eax, 0
```

이유:

- 더 짧은 기계어
- 실행 속도 빠름
- 의존성 제거 효과

비트 토글(toggle)

```
xor eax, 1
```

최하위 비트 반전

[4] NOT 명령어

[4-1] 형식

```
not <operand>
```

[4-2] 동작

```
operand = ~operand
```

모든 비트 반전(1 의 보수)

[4-3] 특징

- 단항 연산
- 플래그에 영향 없음

[4-4] 예:

```
not eax
```

모든 비트 반전

[7-7] 논리 연산 vs 산술 연산 차이

| 구분    | 논리 연산   | 산술 연산    |
|-------|---------|----------|
| 캐리 개념 | 없음      | 있음       |
| CF/OF | 0으로 초기화 | 연산 결과 반영 |
| 목적    | 비트 조작   | 수치 계산    |

[7-7-1] 실전 활용 예제

[1] 짝수/홀수 판별

```
test eax, 1
```

또는

```
and eax, 1
```

최하위 비트가 1 이면 홀수

#### [1-1] 원리

모든 정수의 최하위 비트(LSB):

| 값 | 2진수 | LSB |
|---|-----|-----|
| 6 | 110 | 0   |
| 7 | 111 | 1   |

#### [2] 특정 비트 검사

```
and eax, 0x8  
jz not_set
```

#### [3] 플래그 설정용 OR

```
or eax, 0x4
```

#### [4] 특정 비트 끄기

```
and eax, ~0x4
```

#### [7-8] RFLAGS 레지스터란?

x86-64 CPU 에서 RFLAGS 레지스터가 있다.

이 안에는 연산 결과에 대한 상태 정보가 비트 형태로 저장된다.

즉, 연산을 하면 CPU가 자동으로 플래그를 설정한다.

그 다음 조건 점프(jz, jc 등)가 그 플래그를 보고 분기한다.

우리가 반드시 알아야 할 것은 다음 6 개다.:

- CF(Carry Flag)
- ZF(Zero Flag)
- SF(Sign Flag)
- OF(Overflow Flag)
- PF(Parity Flag)
- AF(Auxiliary Carry Flag)

실전에서는 CF, ZF, SF, OF 가 핵심이다.

### [7-8-1] CF - Carry Flag(자리올림 플래그)

#### [1] 언제 설정되는가

- 덧셈(add): 자리 올림이 발생하면 1
- 뺄셈(sub): 빌림(borrow)이 발생하면 1

#### [2] 예시(8 비트 기준)

```
mov al, 255    ;1111 1111
add al, 1
```

결과:

```
 11111111
+00000001
-----
00000000 ← 자리 넘침 발생
→ CF = 1
```

#### [3] 의미

CF 는 unsigned 연산의 overflow 검사에 사용된다.

### [7-8-2] ZF - Zero Flag(제로 플래그)

#### [1] 언제 설정되는가

연산 결과가 0 이면 1

#### [2] 예시

```
mov eax, 5
sub eax, 5
```

결과 = 0 → ZF = 1

#### [3] 어디에 쓰는가?

```
jz label
```

→ ZF = 1 이면 점프

같은 값인지 비교할 때 핵심

### [7-8-3] SF - Sign Flag(부호 플래그)

#### [1] 언제 설정되는가

결과의 최상위 비트를 복사한다.

- 최상위 비트 = 1 → 음수
- 최상위 비트 = 0 → 양수

(2의 보수 표현 기준)

#### [2] 예시(32 비트)

```
mov eax, -1
```

-1 = 11111111 11111111 11111111 11111111

SF = 1

#### [3] 어디에 쓰는가?

signed 비교 연산에서 사용

### [7-8-4] OF - Overflow Flag(부호 오버플로우)

#### [1] 언제 설정되는가?

signed 범위를 벗어나면 1

#### [2] 예시(8 비트 signed)

최대값: 127(0111 1111)

```
mov al, 127  
add al, 1
```

결과

10000000 → -128

부호가 갑자기 바뀜 → OF = 1

#### [3] CF 와 OF 차이

| 플래그 | 기준                |
|-----|-------------------|
| CF  | unsigned overflow |
| OF  | signed overflow   |

### [7-8-5] PF - Parity Flag

#### [1] 언제 설정되는가?

결과의 하위 8 비트에서 1의 개수가 짝수이면 1

#### [2] 예시

|          |
|----------|
| 00000011 |
|----------|

1이 두 개 → 짝수 → PF = 1

#### [3] 어디에 쓰는가?

거의 사용되지 않음(레거시 용도)

### [7-8-6] AF - Auxiliary Carry

- BCD 연산용

- 현대 프로그래밍에서는 거의 사용 안함

### [7-9] Shift Operations(시프트 연산)

시프트 연산은 피연산자 내부의 비트들을 왼쪽 또는 오른쪽으로 이동시키는 연산이다.

비트를 시프트하는 일반적인 이유는 다음과 같다.

- 특정 목적을 위해 일부 비트만 분리하기 위해
- 2의 거듭제곱에 대한 곱셈 또는 나눗셈을 수행하기 위해

모든 비트는 한 위치씩 이동한다.

피연산자 범위를 벗어나 밀려난 비트는 사라지며, 반대쪽에는 0으로 채워진다.

#### [7-9-1] Logical Shift(논리 시프트)

논리 시프트는 소스 레지스터의 모든 비트를 지정된 비트 수만큼 이동시키고, 그 결과를 목적지 레지스터에 저장하는 비트 연산자이다.

비트는 왼쪽 또는 오른쪽으로 이동할 수 있다.

소스 피연산자의 모든 비트는 지정된 수만큼 이동하며,  
비어 있는 자리는 0으로 채워진다.

논리 시프트는 피연산자를 “숫자”로 취급하는 것이 아니라  
단순한 비트들의 나열로 취급한다.

### [1] 예시 설명

- 23 을 1 비트 논리 오른쪽 시프트 → 2 로 나누기 → 결과 11
- 13 을 2 비트 논리 왼쪽 시프트 → 4 를 곱하기 → 결과 52

### [2] 논리 시프트 명령어

#### [2-1] SHL(Shift Left Logical)

shl <dest>, <imm8>

shl <dest>, cl

목적지 피연산자를 왼쪽으로 논리 시프트한다.  
오른쪽 빈 자리는 0 으로 채워진다.

- <imm> 또는 cl 레지스터 값은 1~64 사이
- 목적지는 즉시값 불가
- 즉시값은 8 비트만 허용

예:

shl ax, 8

shl rcx, 32

shl eax, cl

shl qword [qNum], cl

#### [2-2] SHR(Shift Right Logical)

shr <dest>, <imm8>

shr <dest>, cl

목적지 피연산자를 오른쪽으로 논리 시프트한다.  
왼쪽 빈 자리는 0 으로 채워진다.

- <imm> 또는 cl 값은 1~64
- 목적지는 즉시값 불가
- 즉식맞은 8 비트만 허용

예:

shr ax, 8

shr rcx, 32

shr, eax, cl

shr qword [qNum], cl

#### [2-3] 논리 시프트의 의미

- SHR → unsigned 나눗셈(2 의 거듭제곱으로 나눔)
- SHL → unsigned 곱셈(2 의 거듭제곱을 곱함)

## [7-9-2] Arithmetic Shift(산술 시프트)

산술 시프트 역시 비트 연산이지만,  
특히 부호 있는 정수(signed) 처리를 위해 사용된다.

### [1] Arithmetic Right Shift(SAR)

오른쪽 산술 시프트는  
오른쪽으로 비트를 이동시키되,

왼쪽의 빈 자리는 원래의 최상위 비트(부호 비트)로 채운다.  
이를 부호 확장(sign extension)이라고 한다.

즉,

- 양수 → 0 채움
- 음수 → 1 채움

### [2] Arithmetic Left Shift(SAL)

왼쪽 산술 시프트는  
왼쪽으로 이동하고 오른쪽은 0 으로 채운다.

부호 비트는 유지되지 않는다.  
실질적으로 SHL 과 동일하게 동작한다.

### [3] 산술 시프트의 특징

- SAR → 부호 유지
- SAL → SHL 과 동일
- SAR 는 나눗셈처럼 보이지만 실제 divide 명령어와는 다르게 음수에서 내림(round down) 방식으로 동작한다.

### [4] 산술 시프트 명령어

#### [4-1] SAL

```
sal <dest>, <imm8>  
sal <dest>, cl
```

왼쪽 산술 시프트  
오른쪽 0 채움

#### [4-2] SAR

```
sar <dest>, <imm8>  
sar <dest>, cl
```

오른쪽 산술 시프트  
왼쪽은 부호 비트로 채움



### [7-9-3] Rotate Operations(회전 연산)

회전 연산은 시프트와 유사하지만,  
밀려난 비트를 반대쪽으로 다시 넣는다.

예:

10010110 를

- 오른쪽으로 1 회전 → 01001011
- 왼쪽으로 1 회전 → 00101101

#### [1] ROL(Rotate Left)

rol <dest>, <imm8>

rol <dest>, cl

왼쪽 회전

#### [2] ROR(Rotate Right)

ror <dest>, <imm8>

ror <dest>, cl

오른쪽 회전

### [7-10] Control Instructions(제어 명령어)

프로그램 제어는 다음과 같은 구조를 의미한다.

- IF 문
- 반복문(loop)

고급 언어의 IF-THEN-ELSE 구조는  
어셈블리 수준에서는 존재하지 않는다.

어셈블리는 다음만 제공한다.

- 무조건 점프
- 조건 점프

조건 분기와 반복은 모두 점프 명령어를 이용해 구현된다.

### [7-10-1] Labels(레이블)

레이블은 점프 명령어의 목적이다.

예:

```
loopStart:
last:
```

레이블은:

- 문자로 시작
- 문자, 숫자, “\_”사용 가능
- 콜론(:)으로 끝남
- yasm 에서는 대소문자 구분
- 한 번만 정의 가능

레이블은 프로그램 내 특정 위치를 나타내는 이름이다.

### [7-10-2] Unconditional Control Instructions(무조건 제어 명령어)

무조건 제어 명령어는 프로그램 내의 특정 위치(레이블)로 조건 없이 점프한다.

점프 대상이 되는 레이블은:

- 정확히 한 번만 정의되어야 하며
- 점프 명령어가 있는 위치에서 접근 가능하고
- 범위(scope)내에 있어야 한다.

#### [1] 무조건 점프 명령어

| 명령어         | 설명          |
|-------------|-------------|
| jmp <label> | 지정된 레이블로 점프 |

※ 레이블은 반드시 정확히 한 번 정의되어야 한다.

예시:

```
jmp startLoop
jmp ifDone
jmp last
```

### [7-10-3] Conditional Control Instructions(조건 제어 명령어)

조건 제어 명령어는 비교 결과에 따라 점프를 수행한다.

이는 기본적인 IF 문 기능을 제공한다.

#### [1] 비교 수행 과정

비교를 위해서는 두 단계가 필요하다.

1. cmp 명령어
2. 조건 점프 명령어

#### [2] 동작 방식

- cmp 명령어는 두 피연산자를 비교하고 그 결과를 rFlag 레지스터에 저장한다.
- 조건 점프 명령어는 rFlag 레지스터 내용을 기반으로 점프하거나 점프하지 않는다.

중요

- cmp 바로 다음에 조건 점프가 와야 한다.

만약 그 사이에 다른 명령어가 들어가면 rFlag 값이 변경되어 조건이 올바르게 판단되지 않을 수 있다 .

#### [3] cmp 명령어 형식

cmp <op1>, <op2>

- <op1>과 <op2>는 변경되지 않는다.
- 두 피연산자는 반드시 같은 크기여야 한다.
- 둘 중 하나는 메모리 피연산자일 수 있지만, 둘 다 메모리는 불가
- <op1>은 즉시값(immediate)이 될 수 없다.
- <op2>는 즉시값이 될 수 있다.

#### [4] 조건 점프 명령어 종류

##### [4-1] 공통(signed/unsigned 동일)

je <label> ; <op1> == <op2>  
jne <label> ; <op1> != <op2>

##### [4-2] Signed 비교

jlt <label> ; signed, <op1> < <op2>  
jle <label> ; signed, <op1> <= <op2>  
jgt <label> ; signed, <op1> > <op2>  
jge <label> ; signed, <op1> >= <op2>

#### [4-3] Unsigned 비교

|             |                            |
|-------------|----------------------------|
| jb <label>  | ; unsigned, <op1> < <op2>  |
| jbe <label> | ; unsigned, <op1> <= <op2> |
| ja <label>  | ; unsigned, <op1> > <op2>  |
| jae <label> | ; unsigned, <op1> >= <op2> |

#### [5] 예제 1(Signed 데이터)

의사코드

|                                         |
|-----------------------------------------|
| if(currNum > myMax)<br>myMax = currNum; |
|-----------------------------------------|

데이터 선언

|                            |
|----------------------------|
| currNum dq 0<br>myMax dq 0 |
|----------------------------|

구현 코드

|                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------------|
| mov rax, qword [currNum]<br>cmp rax, qword [myMax] ; if currNum <= myMax<br>jle notNewMax ; 새 최대값 설정 건너뛰<br>mov qword [myMax], rax |
|------------------------------------------------------------------------------------------------------------------------------------|

notNewMax:

#### [5-1] 설명

IF 문 논리가 뒤집혀 있다.

어셈블리는

- |                                                                             |
|-----------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>- 점프하거나</li><li>- 점프하지 않는다.</li></ul> |
|-----------------------------------------------------------------------------|

따라서 원래 IF 조건이 거짓일 때

코드를 실행하지 않기 위해 조건이 거짓이면 점프해서 건너뛰다.

## [6] 예제 2(IF-ELSE 구조)

의사코드

```
if(x != 0) {  
    ans = x / y;  
    errFlg = FALSE;  
} else {  
    ans = 0;  
    errFlg = TRUE;  
}
```

데이터 선언

```
TRUE equ 1  
FALSE equ 0  
  
x dd 0  
y dd 0  
ans dd 0  
errFlg db FLASE
```

구현 코드

```
cmp dword [x], 0  
je doElse      ; <op1> == <op2>  
cdq            ; EAX → EDX:EAX  
idiv dword [y]  
mov dword [ans], eax  
mov byte [errFlg], FALSE  
jmp skipElse  
  
doElse:  
mov dword [ans], 0  
mov byte [errFlg], TRUE  
  
skipElse:
```

#### [6-1] 설명

```
- cmp [x], 0
- je doElse → x가 0이면 Else로 점프
```

x ≠ 0이면:

```
- eax ← x
- cdq → 부호 확장
- idiv y → signed 나눗셈
- 결과는 eax에 저장
- errFlag = FALSE
- jmp skipElse → else 부분 건너뛰기
```

#### [6-2] 중요한 점

- signed 데이터이므로 idiv 사용
- 나눗셈 전에 cdq 필요 (eax → edx:eax 확장)
- idiv 실행 시 edx가 자동으로 변경됨
- 따라서 edx(또는 rdx)에 이전 값이 있었다면 덮어쓰인다.

#### [7-10-4] Jump Out of Range(점프 범위 초과)

조건 점프의 대상 레이블은 short-jump라고 불린다.

이는 조건 점프 명령어로부터 ±128 바이트 이내에 레이블이 있어야 한다는 의미이다.

일반적으로 이 제한은 문제가 되지 않지만,

아주 큰 루프에서는 어셈블러가 다음과 같은 오류를 발생시킬 수 있다.

```
jump out-of-range
```

반면에 무조건 점프(jmp)는 범위 제한이 없다.

#### [1] 오류 발생 예시

```
cmp rcx, 0
jne startOfLoop
```

만약 startOfLoop 레이블이 멀리 떨어져 있다면 “jump out-of-range” 오류가 발생할 수 있다.

### [1-1] 해결 방법

논리를 뒤집고, 긴 점프는 jmp 로 처리하면 된다.

```
cmp rcx, 0
je endOfLoop
jmp startOfLoop
endOfLoop:
```

이 코드는:

- 조건 점프는 가까운 레이블로
- 먼 점프는 무조건 점프로 처리

하면 동일한 기능을 수행한다.

### [2] 조건 점프 명령어 요약

#### [2-1] cmp

```
cmp <op1>, <op2>
```

- <op1>과 <op2> 비교
- 결과는 rFlag 레지스터에 저장
- 피연산자는 변경되지 않음
- 두 피연산자가 모두 메모리일 수 없음
- <op1>은 즉시값 불가
- <op2>는 64 비트 즉시값 불가

예시

```
cmp rax, 5
cmp ecx, edx
cmp ax, word [wNum]
```

#### [2-2] je

```
je <label>
```

이전 cmp 결과에 따라 <op1> == <op2> 이면 점프

레이블은 정확히 한 번 정의되어야 한다.

예:

```
cmp rax, 5,
je wasEqual
```

#### [2-3] jne

```
jne <label>
```

<op1> != <op2>이면 점프

예:

```
cmp rax, 5  
jne wasNotEqual
```

### [3] Signed 조건 점프

#### [3-1] jl

```
jl <label>
```

signed 비교에서 <op1> < <op2>이면 점프

예:

```
cmp rax, 5  
jl wasLess
```

#### [3-2] jle

```
jle <label>
```

signed 비교에서 <op1> <= <op2> 이면 점프

예:

```
cmp rax, 5  
jle wasLessOrEqual
```

#### [3-3] jg

```
jg <label>
```

signed 비교에서 <op1> > <op2>이면 점프

예:

```
cmp rax, 5  
jg wasGreater
```

#### [3-4] jge

```
jge <label>
```

signed 비교에서 <op1> >= <op2>이면 점프

예:

```
cmp rax, 5  
jge wasGreaterOrEqual
```



#### [4] Unsigned 조건 점프

##### [4-1] jb

jb <label>

unsigned 비교에서 <op1> < <op2>이면 점프

예:

cmp rax, 5

jb wasLess

##### [4-2] jbe

jbe <label>

unsigned 비교에서 <op1> <= <op2>이면 점프

예:

cmp rax, 5

jbe wasLessOrEqual

##### [4-3] ja

ja <label>

unsigned 비교에서 <op1> > <op2>이면 점프

예:

cmp rax, 5

ja wasGreater

##### [4-4] jae

jae <label>

unsigned 비교에서 <op1> >= <op2>이면 점프

예:

cmp rax, 5

jae wasGreaterOrEqual

### [7-10-5] Iteration(반복)

앞에서 설명한 기본 제어 명령어들은 반복(loop)을 구현하는 수단을 제공한다.  
기본적인 반복 구조는 다음과 같이 구현할 수 있다.

- 반복 횟수를 세는 카운터(counter)
- 루프의 상단 또는 하단에서
- cmp 와 조건 점프를 이용해 반복 여부를 결정한다.

#### [1] 예제: 홀수의 합 구하기

다음과 같은 선언이 있다고 가정하자.

```
lpCnt dq 15  
sum dq 0
```

아래 코드는 1 부터 30 까지의 홀수들의 합을 계산한다.

```
mov rcx, qword [lpCnt]      ; 루프 카운터  
mov rax, 1                  ; 현재 홀수 값  
  
sumLoop:  
add qword [sum], rax        ; 현재 홀수를 합에 더함  
add rax, 2                  ; 다음 홀수로 증가  
dec rcx                     ; 루프 카운터 감소  
cmp rcx, 0  
jne sumLoop
```

#### [1-1] 동작 설명

- rcx: 반복 횟수(15 번)
  - rax: 현재 홀수 값(1 부터 시작)
  - 매 반복마다
    - 현재 홀수를 sum 에 더하고
    - 2 를 더해 다음 홀수로 이동
    - rcx 감소
    - rcx ≠ 0 이면 다시 루프
- 이 예제는 여러 방법 중 하나일 뿐이다.

#### [2] loop 명령어

rcx 를 카운터로 사용하는 방식은  
정해진 횟수만큼 반복할 때 매우 유용하다.

이를 더 간단히 처리해주는 특별한 명령어가 있다.

```
loop <label>
```

## [2-1] 동작 방식

loop 명령어는 내부적으로 다음을 수행한다.

1. rcx 감소
2. rcx 를 0 과 비교
3. rcx  $\neq$  0 이면 지정된 레이블로 점프

레이블은 반드시 한 번만 정의되어야 한다.

## [3] 동등한코드 비교

### [3-1] Code Set 1

```
loop <label>
```

### [3-2] Code Set 2

```
dec rcx  
cmp rcx, 0  
jne <label>
```

두 코드는 동일한 기능을 수행한다.

### [3-3] 이전 예제를 loop 로 다시 작성

```
mov rcx, qword [maxN]      ; 루프 카운터  
mov rax, 1                  ; 현재 홀수  
  
sumLoop:  
add qword [sum], rax  
add rax, 2  
loop sumLoop
```

이 코드 역시 이전 코드와 동일한 결과를 낸다.

## [4] 주의사항

### [4-1] rcx 초기화 필수

loop 는

- 먼저 rcx 를 감소
- 그 후 0 과 비교

만약 rcx 를 초기화하지 않으면 예상치 못한 횟수만큼 반복될 수 있다.

이 경우 디버깅이 매우 어렵다.

### [4-2] rcx 에만 사용 가능

loop 명령어는

- 오직 rcx 레지스터만 사용 가능
- 반드시 감소 방향 반복(count-down)만 가능

#### [4-3] 중첩 루프 문제

내부 루프와 외부 루프 모두에서 loop 를 사용하면 rcx 충돌이 발생한다.

이 경우:

- 내부 루프에서 rcx 를 저장(push)
- 사용 후 복원(pop)

과 같은 추가 조치가 필요하다.

#### [7-11] Example Program – Sum of Squares(제곱합 구하기 예제 프로그램)

다음은 1 부터 n 까지의 제곱합을 구하는 완전한 예제 프로그램이다.

예를 들어, n = 10 일 때

$$1^2 + 2^2 + \dots + 10^2 = 385$$

이 예제의 main 은 위 예시와 맞추기 위해 n 값을 10 으로 초기화 한다.

#### [7-11-1] 전체 프로그램

```
; Simple example program to compute the
; sum of squares from 1 to n.
; *****

; =====
; Data declarations
; =====
section .data

; -----
; Define constants
SUCCESS equ 0          ; Successful operation
SYS_exit equ 60        ; call code for terminate

; -----
; Define Data
n dd 10
sumOfSquares dq 0
```

### [1] 데이터 영역 설명

| 항목           | 설명                 |
|--------------|--------------------|
| SUCCESS      | 프로그램 정상 종료 코드      |
| SYS_exit     | 리눅스 시스템콜 번호 (exit) |
| n            | 반복 상한값 (32비트 정수)   |
| sumOfSquares | 결과 저장용 64비트 변수     |

```
; *****
;
section .text
global _start

_start:
```

### [2] 제곱합 계산 부분

```
; -----
; Compute sum of squares from 1 to n (inclusive).
; Approach:
; for (i=1; i<=n; i++)
;     sumOfSquares += i^2;
```

C 코드로 표현하면:

```
for (i = 1; i <= n; i++)
    sumOfSquares += i * i;
```

### [3] 레지스터 초기화

```
mov rbx, 1          ; i = 1
mov ecx, dword [n]  ; 반복 횟수
```

의미

- rbx: 현재 i 값
  - ecx: 반복 카운터(n)
- (loop 명령어는 rcx 를 사용하므로 ecx 에 로드함)

### [4] 루프 본문

```
sumLoop:
mov rax, rbx        ; rax = i
mul rax             ; rax = rax * rax → i^2
add qword [sumOfSquares], rax
inc rbx             ; i++
loop sumLoop
```

#### [5] 프로그램 종료

```
last:
mov rax, SYS_exit
mov rdi, SUCCESS
syscall
```

#### [6] 구조적 특징 분석

loop 사용 이유

- 반복 횟수가 정해져 있음
- rcx 감소 기반 반복

rbx 사용 이유

- 일반 용도 레지스터
- i 값 보관

rax 사용 이유

- mul 명령어가 rax 를 기본 피연산자로 사용

#### [7] 디버깅

디버거를 사용하여

- sumOfSquares 메모리 값 확인
- rax/rbx/rcx 변화 추적

하면 정상 실행 여부를 검증할 수 있다.